فروردین | ۸ | 28 Mar 2021
یکشنبه | ۱۴ شعبان ۱۴۴۲

شبکه های عصبی - دکتر صفابخش - پاییز ۱۴۰۰

جلسه اول - ۱۴۰۰/۶/۲۸

ایمیل درس: safacourses1400@gmail.com

بارم بندی: کار مطالعاتی: ٪۲۰ پروژه ها: ٪۵۰ امتحان پایانترم: ٪۳۰

حداقل ۵ مقاله، حداقل ۵۰ صفحه، سعی کنید مقالات ژورنال انتخاب کنید نه کنفرانس

مقدمه (Introduction)

اسامی مختلف شبکه های عصبی: Neural Networks, Neural Nets, NN

Artificial Neural Networks, ANN, Connectionist Models,

parallel Distributed Processing (PDP) systems,

Neuromorphic systems, Neural Computing, Neurocomputing

Deep Neural Networks, Deep Learning

← سوال: شبکه های عصبی چه چیزی هستند؟

یک جواب خیلی کلی برای این سوال این است که شبکه های عصبی تلاش انسان است برای ایجاد کردن امکان دستیابی برای تقلید از عملکرد مغز انسان.

← یک ویژگی مغز انسان این است که هم زمان می تواند کارهای متعددی را انجام بدهد و می تواند از روی اطلاعات مجمل یا ناقص یا متناقض نتیجه گیری کند. هم چنین در انجام کارهای طبیعی مثل بینایی، خیلی بهتر از ماشین های امروزی کار می کند. مثال های دیگر: زبان، بازیابی اطلاعات، برنامه ریزی، مغز انسان در همه این موارد بهتر از ماشین ها عمل می کند. به همین دلیل علاقه داریم که در ماشین ها از عملکرد مغز انسان تقلید کنیم.

Scanned with CamScanner

۱۴۰۰

روش های رایانش عصبی (Neural Computing) یک فرق اساسی با روش های کلاسیک حل مسئله دارند. در روش های سنتی باید یک الگوریتم ابداع کنیم و آن را پیاده سازی کنیم اما در رایانش عصبی نیاز به داشتن الگوریتم نداریم، هم چنین نیازی به پیاده سازی هم نداریم چون گاهی اوقات الگوریتم یافت می شود اما پیاده سازی آن سخت است. روش آن به این صورت است که با تعامل با محیط مسئله یادمی گیرد که پاسخی که شما انتظار دارید را پیدا کند.

نکته دیگری که رایانش عصبی را ارزشمند می کند این است که در روش های سنتی اگر آنتی در هر بخش از سیستم رخ دهد، سیستم از کار می افتد اما در شبکه های عصبی اگر بخشی از شبکه (چار مشکل شود (مثلا گره ای از دست برود) شبکه به کار خودش ادامه می دهد فقط ممکن است نتیجه ای که شبکه می دهد بدتر شود. این ویژگی فوق العاده مهم است به خصوص در سیستم های حساس. به این ویژگی gracefull degradation (خراب شدن آبرومندانه) گفته می شود.

پیاده سازی شبکه های عصبی

① شبیه سازی نرم افزاری ← کتابخانه های موجود

② پیاده سازی سخت افزاری ← مزیت این روش ها سرعت بیشتر نسبت به شبیه سازی های نرم افزاری است، هم چنین می تواند به عنوان اجزایی در شبکه های پیچیده مورد استفاده قرار بگیرند

Scanned with CamScanner

جلسه سوم - ۱۴۰۰/۷/۴

وقتی کندیک نورون فعال می‌شود این فعال شدن چگونه بر نورون‌های دیگر تأثیر می‌گذارد؟

وقتی پتانسیل به سیناپس می‌رسد کیسه‌های مواد انتقال دهنده به سمت شکاف سیناپسی می‌روند و مواد خود را تخلیه می‌کنند و این مواد بعد می‌کنند در دیواره‌ی شکاف و حلول می‌روند و رسیدن به نورون پس سیناپسی

مولکول‌های گیرنده در سلول پس سیناپسی منتظر دریافت ماده انتقال دهنده‌اند

اگر نوع انتقال دهنده تحریک یا فاداشته باشد پتانسیل استراحت سلول پس سیناپسی به سمت صفر می‌رود (Depolarization) اختلاف پتانسیل دو سوی غشا دارین سلول کاهش می‌یابد در نتیجه سلول آماده‌ی تشکیل شدن می‌شود

یک زمان خاص اگر تحریکات مختلف از دوردل‌های مختلف نورون وارد شود، اثر آن‌ها با هم جمع می‌شود و اگر از یک حد آستانه بیشتر شود سلول فعال می‌شود

اگر ماده منتقل کننده در ناحیه پس سیناپسی تجمع پیدا کند سلول به یک حالتی می‌رود که به آن حد اکثر تحریک پذیری یا دپولاریزاسیون ثابت می‌گویند. مواد پیوسته جریان دارد از این سلول به سلول‌های بعدش. دپولاریزاسیون طبیعت و پلکول‌ها تحریک تشکیل می‌شود دارن می‌توانند تم ارجمند ایجاد کنند تا ادامه پیدا کند. چگونه؟ چون تحریک سلول پس سیناپسی ادامه پیدا می‌کند، سلول به آستانه‌ی تحریک می‌رسد و باز تحریک می‌شود و این تحریک‌ها می‌توانند ادامه پیدا کنند تا ادامه پیدا کند.

اگر مواد انتقال دهنده از نوع بازدارنده باشند (inhibitory): می‌خواهد مانع ایجاد تحریک شود) اختلاف پتانسیل را می‌خواهد افزایش دهد. پلاریزاسیون سلول پس سیناپسی زیادی می‌شود مانع را زیادی می‌کند برای فعال شدن سلول بعدی.

جمع جبری این پتانسیل‌ها اگر به حد آستانه برسد سلول فعال می‌شود.

درمان اطلاعات در مغز

مدارهای واگرا: یک نورون چند نورون بعدی را تحریک می‌کند دوباره هرکدام از آن‌ها چند نورون بعدی را و ... ساختار درختی ایجاد می‌شود. مثال: احساسات

مدارهای همگرا: چند نورون می‌خواهند یک نورون را تحریک کنند مثلا در کارهای حرکتی (motor)

مدارهای موازی: یک کار مانند، چند یک در آن‌ها وجود دارد. باعث می‌شود عمل تشخیص سرعت تشخیص بیشتر مثل فرمان قند

Scanned with CamScanner

مدار موازی پس از تخلیه ← به سری مسیرهای موازی که می خواهند یک نورون را فعال کنند.

مثال: ارسال هم زمان یک سری ضربات سریع

شبکه های عصبی مصنوعی (Artificial Neural Networks)

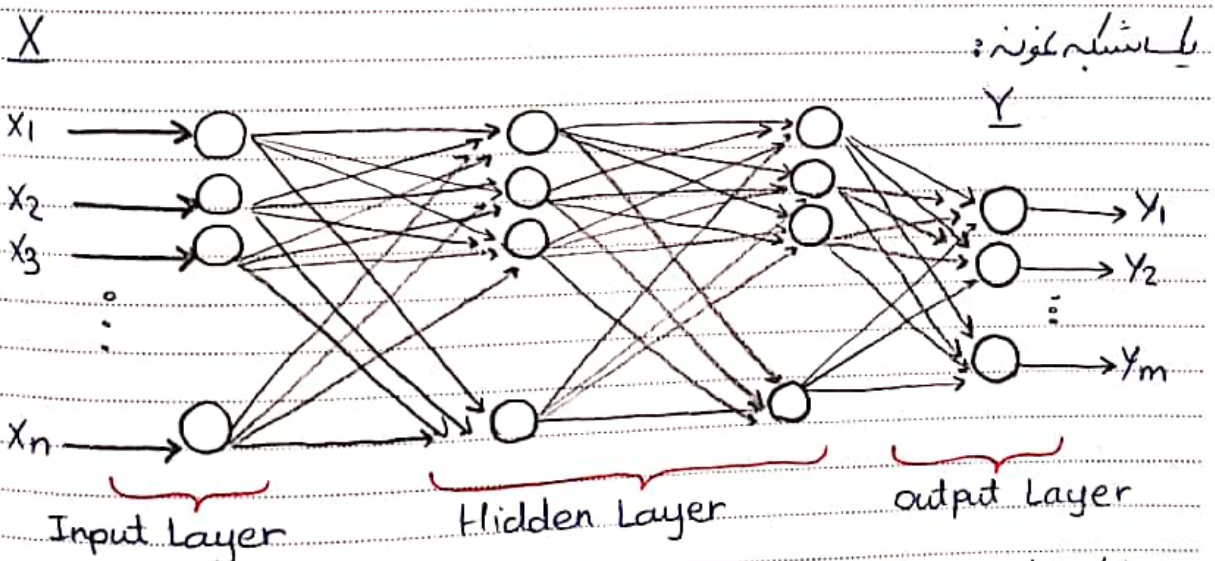
← یک شبکه عصبی یک ساختار موازی توزیع شده به شکل یک گراف جهت دار است. در این گراف هر گره نشان دهنده یک نورون یا واحد پردازشگر است (Neuron or processing unit)

← هر کدام از نورون ها فقط یک خروجی دارد ولی تعداد دلخواهی ورودی می تواند داشته باشند. خروجی هر نورون می تواند به تعداد دلخواهی نورون های دیگر برود. همه ی این خروجی ها با هم برابرند.

← هر ارتباط در این گراف یک جهت و یک وزن دارد. جهت نشان می دهد که در این ارتباط کدام گره روی کدام گره اثر می گذارد و مبدأ و مقصد را مشخص می کند و قوت آن را وزن نشان می دهد.

← گره ها گروه بندی می شوند به یک سری لایه هایی (در مورد مغز هم همین طور است)

یک شبکه نمونه:



همانطور که مشاهده می شود ساختار به گونه ای است که گره ها لایه لایه هستند

یک لایه هست که از خارج ورودی ها را قبول می کند که به آن لایه ورودی می گویند. نورون های این لایه سیگنال ها را از خارج می گیرند و به لایه های بعدی در شبکه می دهند.

Scanned with CamScanner

یک لایه هست که نتیجه شبکه را به بیرون می دهد. (لایه خروجی) و نورون های آن را نورون های خروجی می نامند

لایه هایی که بین این دو لایه قرار دارند لایه های مخفی (hidden layer) نامیده می شوند چون

نه از ورودی و نه از خروجی دیده نگاه می کنیم این لایه ها را می بینیم مخفی هستند.

معمولا ورودی ها را با x و خروجی ها را با y نشان می دهیم. معمولا تعداد بعد ورودی را با n و

تعداد بعد خروجی را با m نشان می دهیم.

هر نوع بودی می تواند به هر نوع نورون در لایه دیگر وصل باشد (مثلا از لایه اول میتونه به لایه چهارم وصل بشه)

فرض می کنیم که همه ی نورون های یک لایه شبیه هم هستن

هیچ تأخیری در نظر نمی گیریم (برخلاف مدارهای منطقی) از ورودی به خروجی.

عملکرد نورون ها ۲ حالت می تواند باشد:

① synchronous (سنکرون) episodical ← در گیرانی که داریم نورون ها در زمان های خاصی فعال می شوند

② Asynchronous (آسنکرون) continuous ← عملکرد نورون ها پیوسته است و هر وقت در ورودی تغییری

ایجاد شود بلافاصله اثر آن در خروجی ظاهر می شود

← می توان گفت که شبکه ما در این حالت مثل مدارهای منطقی است ! ...

← برای راحتی فرض می کنیم همه ی نورون های یک شبکه سنکرون یا آسنکرون است

* وزن ها در یک فرآیند یادگیری، یادگرفته می شوند. این یادگیری به چند چیز نیاز دارد:

① قانون یادگیری: شبکه با محیط تعامل می کند تا یاد بگیرد و این تعامل از طریق داده هایی که از محیط

وارد شبکه می شود صورت می گیرد.

← سلول های واقعی عملکرد بسیار پیچیده تری دارند. همیشه آسنکرون هستند. همیشه ترکیبی بی‌نظیر از

ورودی ها را در نظر می گیرند.

Scanned with CamScanner

← خروجی در نورون های واقعی ضربه یا دنباله ای از ضربه هاست درحالی که در شبکه عصبی مصنوعی سطح
بشکل سطح در نظر گرفته می شود. مثلا سطح اده

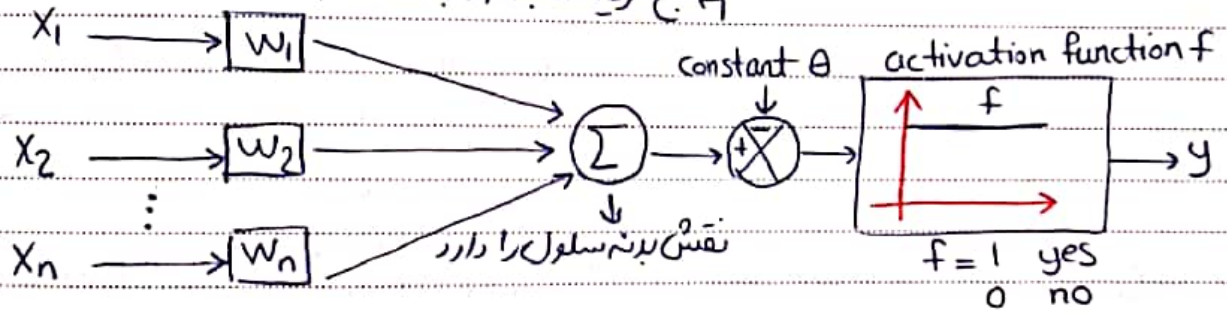
← در نورون های واقعی مدل مدار منطقی تاخیر وجود دارد و پیچیدگی های ای را طی می کند

← کلیه مدار در سیناپس می تواند به صورت اتفاقی عرض شود که تحت عوامل مختلف و خیلی قابل پیش بینی نیست.

← مدل McCulloch & Pitts :

اثر سیناپس را با وزن ها نشان می دهیم. اگر وزن مثبت باشد اثر تحریک کنندگی دارد و اگر منفی باشد اثر بازدارنده دارند.

↳ جمع تحریکات بدنه را با مقدار ثابت θ مقایسه می کند



اگر نتیجه تحریکات منهای θ مثبت باشد تابع فعال سازی ۱ می دهد در غیر اینصورت ۰ می دهد

$$w_1x_1 + w_2x_2 + \dots + w_nx_n - \theta \xrightarrow{f} y = f(w_1x_1 + \dots + w_nx_n - \theta)$$

$$y = f\left(\sum_{i=1}^{n} w_i x_i - \theta\right) = f\left(\sum_{i=0}^{n} w_i x_i\right) \qquad x_0 = 1, w_0 = -\theta$$

باید $\theta = w_0 x_0$ باشد به همین دلیل x_0 را ۱ می گیریم

در این مدل که متعلق به McCulloch و Pitts است تابع فعال سازی تابع پله است.

می توانیم این مدل را تغییراتی ایجاد کنیم:

۱) تابع فعال سازی را تغییر بدهیم

اسامی مختلفی برای توابع فعال سازی عنوان شده است:

- Activation function
- Squashing function
- Transfer function
- Gain function

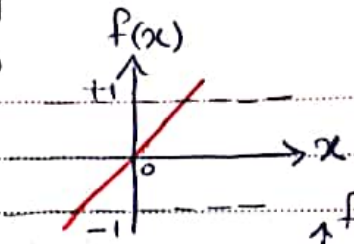
روز سلامتی

Scanned with CamScanner

انواع تابع فعال سازی :

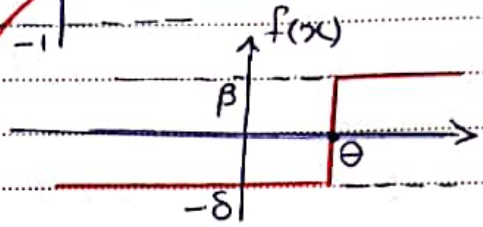
الف) تابع خطی

$$f(x) = a.x \quad \alpha > 0$$



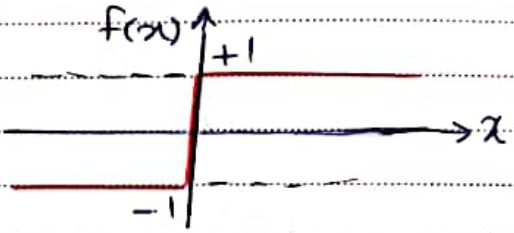
ب) تابع پله

$$f(x) = \begin{cases} \beta & x \geq \theta \\ -\delta & x < \theta \end{cases}$$



اگر $\beta = \delta = 1$ و $\theta = 0$ باشد تابع علامت ایجاد می شود:

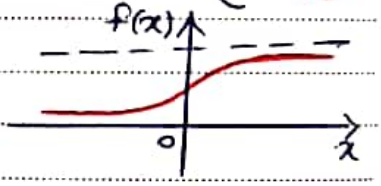
$$f(x) = \begin{cases} 1 & x \geq 0 \\ -1 & x < 0 \end{cases} = sgn(x)$$



مرفتش با تابع پله در این است که به جای صفر، ۱- داریم که کمتر عمل می کند.

ج) تابع Sigmoid

$$f(x) = \frac{1}{1+e^{-\alpha x}} \qquad \alpha > 0 \quad \alpha \simeq 1$$



به آن binary Sigmoid یا Logistic هم گفته می شود

مشتقش برحسب خودش قابل بیان است

$$f'(x) = \alpha f(x)\left[1 - f(x)\right]$$

① ایراد دارد ← از یک جایی به بعد هم در قسمت مثبت هم منفی به یک حالت ثابتی می رسد. چون مشتق این تابع در این نواحی کوچک است و این مشتق هست که در آموزش کاربرد دارد و خیلی کوچک می شود و آموزش نمی تواند ای خوب انجام بگیرد.

② ایراد دوم: نسبت به مبدا متقارن نیست.

د) تابع Bipolar Sigmoidal (دوقطبی) ← همان تابع قبلی است که scale شده و شیفت داده شده

$$f(x) = \frac{2}{1+e^{-\alpha x}} - 1 = \frac{1-e^{-\alpha x}}{1+e^{-\alpha x}} = \tanh\left(\frac{\alpha x}{2}\right)$$

$$f'(x) = \left(\frac{\alpha}{2}\right)\left[1 + f(x)\right]\left[1 - f(x)\right]$$

Scanned with CamScanner

جلسه چهارم - ۱۴۰۰/۷/۶

به طور کلی در توابع فعال سازی دو مقداری بهتر است که یک مقدار مثبت و یک مقدار منفی باشد تا اینکه یک مقدار مثبت و یکی صفر باشد. صفر محو لا مستقل سازی می شود.

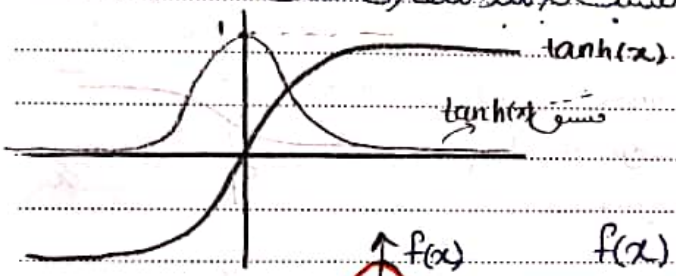
ادامه توابع فعال سازی:

ه) tanh Units

$$\tanh(x) = \frac{1-e^{-2x}}{1+e^{-2x}}$$

$$\tanh(x) = 2\sigma(2x) - 1$$

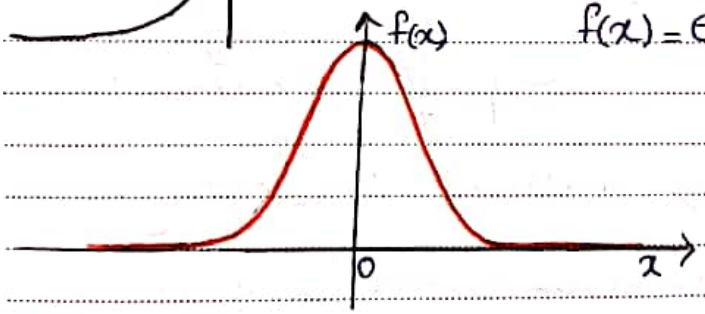
همان زیرگی های Bipolar Sigmoid را دارد، نسبت به مبدا متقارن است



و) تابع گاوسی

$$f(x) = e^{-x^2/\sigma^2}$$

مشتق تابع گوسی، باز هم گوسی می شود و از این جهت مطلوب است



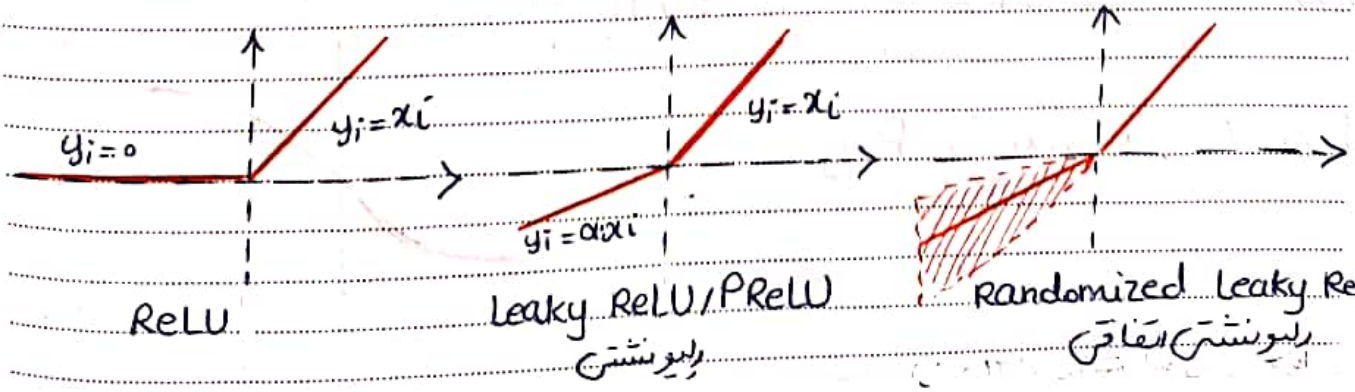
ز) ReLU

$$f(x) = \max(0, x)$$

$$f(x) = \max(x, \alpha x)$$

$$\begin{cases} f(x) = \alpha x & x < 0 \\ f(x) = x & x \geq 0 \end{cases}$$

$$\begin{cases} f(x_{ji}) = x_{ji} & x_{ji} \geq 0 \\ f(x_{ji}) = \alpha_{ji} x_{ji} & x_{ji} < 0 \end{cases}$$



ReLU

Leaky ReLU / PReLU
پرلو نشتی

Randomized Leaky ReLU
پرلو نشتی اتفاقی

Scanned with CamScanner

مزیت ReLU : مشکلی که sigmoid یا tanh داشت را کمتر دارد

هزینه محاسباتش کمتر است، همگرایی اش سریعتر است و پیاده سازی اش راحت تر است

مسئله ی به اشباع رفتن sigmoid ها در دو انتها که باعث می شد مشتق کوچک باشد و یادگیری کند شود را ندارد چون برای x های مثبت مشتق ۱ است

ایراد ReLU : بعضی واحدها از کار می میرند و آموزش در مورد آن ها غیرفعال می شود.

علتش این است که برخی وزن ها به گونه ای می شوند که نورون هیچ وقت دیگر فعال نمی شود

این نتیجه ی جریان مشتق های بزرگ در شبکه حین آموزش است

به خصوص اگر ضریب نرخ یادگیری بالا باشد تا حدود ۴۰ درصد از نورون های شبکه ممکن است

میرند به همین دلیل به سراغ leaky ReLU رفتند.

مردن نورون به این معناست وزن ها مقادیری پیدا می کنند که این نورون دیگر نمی تواند فعال شود

برای رفع این ایراد به سراغ Leaky ReLU رفتند که اجازه می دهد مقدار خروجی برای x های منفی

یک مقدار منفی کوچک پیدا کند (نشت کند)

Randomized ReLU : گفتنشیب leaky ReLU را به صورت رندم انتخاب کنیم ←

و این رندم کردن اثر بهتری در آموزش دارد.

α را از یک توزیع یکنواخت (uniform) در فاصله ۰ و ۱ انتخاب می کنند. در فرآیند آموزش، α هر بار با نمونه برداری

از چنین توزیعی انتخاب می شود و موقع تست تمام α های که در آموزش به کار رفته را میانگین گیری می کنند

و این میانگین را به صورت ثابت در زمان تست به کار می برند

$$\alpha_{ji} \sim U(l, u) \quad l<u \text{ and } l,u \in [0,1)$$

رندم کردن برای x های مثبت صورت نمی گیرد چون می خواهیم مشتق ۱ باشد

Scanned with CamScanner

Softmax (Layer)

کاری که امروزه در شبکه‌های عمیق انجام می‌دهیم این است که یک لایه Softmax قرار می‌دهیم در واقع مثل این است که روی خروجی های نورون ها یک عمل دیگر هم انجام می‌دهیم، به عنوان لایه Softmax به کار می‌برد. (در واقع لایه نیست)

علتش این است که در مسائل دسته‌بندی چند کلاسه می‌خواهیم به جای اینکه برای کلاس‌ها مختلف اعداد مختلف داشته باشیم، خروجی ها اعدادی بین ۰ و ۱ باشند که احتمال تعلق به آن کلاس را نشان دهند.

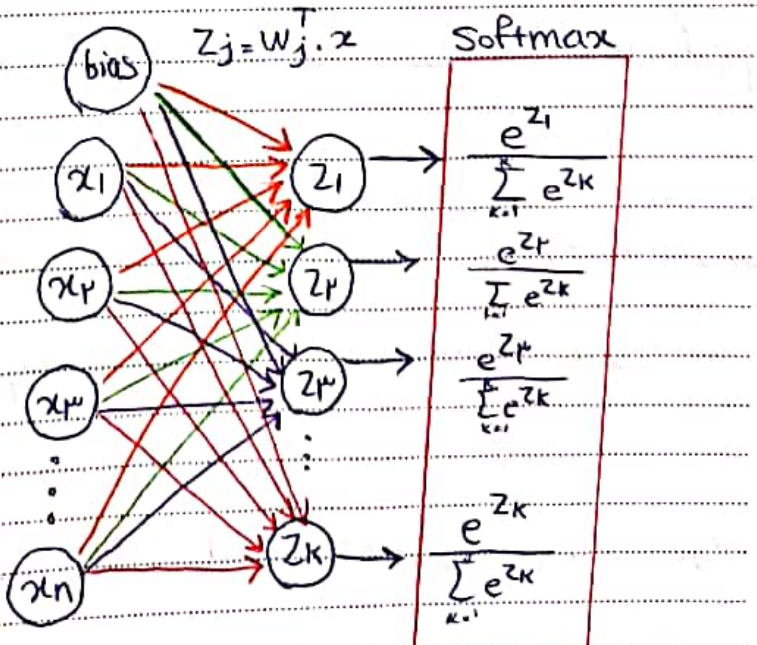
فرض کنید خروجی شبکه K بعدی است. Z ها خروجی اند و یک مقدار حقیقی تولید می‌کنند.

یک تابع $\sigma(z)$ تولید می‌کنیم که این کار را انجام می‌دهد:

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}} \qquad j = 1, 2, \dots, K$$

در واقع یک توزیع احتمال روی خروجی به ما می‌دهد و احتمال تعلق به هر کلاس را نشان می‌دهد.

$$Z = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ \vdots \\ z_K \end{bmatrix} = \begin{bmatrix} w_1^T \\ w_2^T \\ w_3^T \\ \vdots \\ w_K^T \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$$



Scanned with CamScanner

دومین تغییر:

تغییر دیگری که در شبکه های عمیق می توانیم بدهیم:

برای لایه های مختلف توابع فعال سازی مختلفی تعریف کنیم

سومین تغییر:

نورون را غیرخطی کنیم. ترکیبات غیرخطی ورودی ها را وارد کنیم:

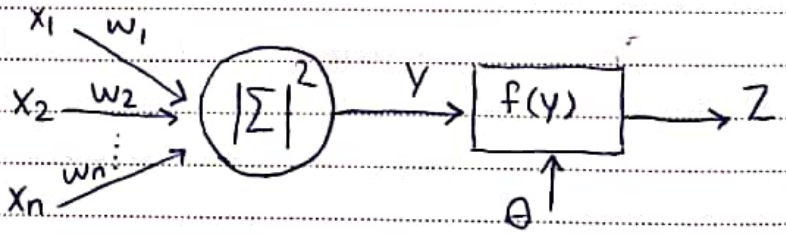
$$y = f\left(\sum_{i=0}^{n} w_i x_i + \sum_{k=1}^{n_i}\sum_{l=1}^{n_i} w_{kl}\, x_k x_l + \dots\right)$$

مثلا یک راه این است که جفت جفت xها را اضافه کنیم (درجه ۲)

چهارمین تغییر: نورون آبسترون شود

پنجمین تغییر: وزن ها اعداد حقیقی نباشند، اعداد مختلط باشند. در تئوری فقط قابل اجرا است.

$$w_i = a_i + j b_i$$



$$y = |w_1 x_1 + w_2 x_2 + \dots + w_n x_n|^2 = (a_1 x_1 + a_2 x_2 + \dots + a_n x_n)^2$$

$$+ (b_1 x_1 + b_2 x_2 + \dots + b_n x_n)^2$$

در مطلق به توان ۲ می شود قسمت حقیقی

به توان ۲ به اضافه ی قسمت موهومی به توان ۲

$$Z = f(y) = \frac{1}{1 + e^{-\alpha(y-\theta)}}$$

y می شود یک عبارت درجه ۲ از ورودی ها

Scanned with CamScanner

تغییر ششم: فعال شدن نورون تصادفی باشد نه تقلی

I ← ورودی خالص به نورون (مجموع $w_i x_i$ها) Y ← خروجی نورون

$P(Y)$ ← احتمال فعال شدن نورون

$$p(I)=\frac{1}{1+e^{-I/T}}$$

T ← پارامتری که میزان randomness را کنترل می کند

$$Y=\begin{cases}+1 & \text{با احتمال } P(I)\\ -1 & \text{با احتمال } 1-P(I)\end{cases}$$

تغییر هفتم: گره های گراف، نورون های فازی باشند. به ۳ صورت قابل انجام است:

① ورودی ها اعداد فازی باشند

② پارامترهای شبکه اعداد فازی باشند (مثلا وزن ها)

③ هم ورودی ها و هم پارامترها فازی باشند

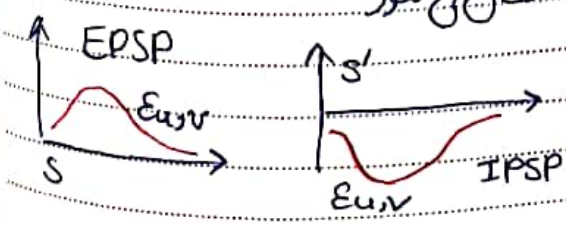
تغییر هشتم: خروجی نورون ها به جای ۰ و ۱، عربه باشد. به آن ها نورون های مربه ای یا spiking Neurons گفته می شود.

برای یک نورون پتانسیل در نظر می گیریم که آن را با P_V نشان می دهیم. از نورون طبیعی به خاطر داریم که P_V مجموع EPSP ها و IPSP هایی است که در آن لحظه به نورون می رسد.

اگر این پتانسیل (P_V) به مقدار آستانه θ برسد نورون فعال می شود

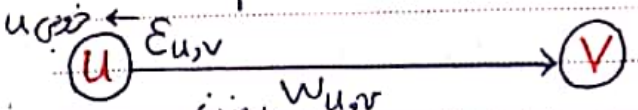
EPSP تاثیر کم کردن پتانسیل منفی دارند

IPSP ها « زیاد » « « « «



Scanned with CamScanner

فرض کنید در شبکه مان نورون های ضربه ای داریم و دو تا نورون u و v داریم که به شکل زیر



کم متصل است:

وقتی نورون u فعال می شود و شلیک می کند

تغییرات پتانسیل با زمان ظاهر می شود و این مقادیر در وزن $w_{u,v}$ ضرب می شوند و وارد می شوند

به نورون بعدی (v). معادله آن: $w_{u,v} \cdot \varepsilon_{u,v}(t-s)$

$s \leftarrow$ زمان شلیک

چون در زمان s شلیک کرده، اثری که وارد نورون v می شود $(t-s)$ است

w می تواند ثابت یا متغیر باشد

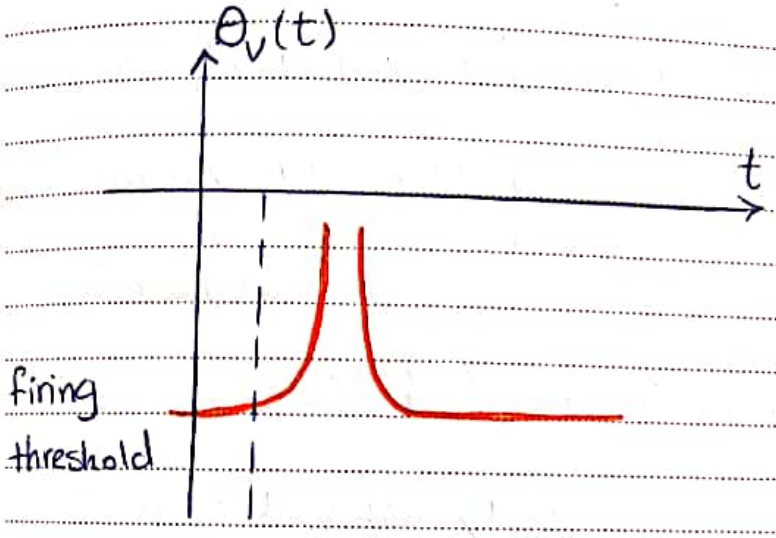
پتانسیل استراحت نورون حدود $-70mv$ است اگر به $-50mv$ برسد نورون فعال می شود

پس مجموع اثرهایی که به یک نورون می رسد اگر پتانسیل را به $-50mv$ یا بالاتر برساند فعال می شود

Scanned with CamScanner

وقتی که نورون شلیک می‌کند (نورون طبیعی)، تا یک مدت کوتاهی بعد از آن نمی‌تواند فعال شود و نمی‌تواند به تحریک ورودی حتی پاسخ بدهد چون در حال طی کردن دوره گذار به وجود آوردن ضربه در خروجی می‌باشد.

مدل آن به این صورت است:



در این تابع وقتی نورون شلیک می‌شود به‌طور ناگهانی θ افزایش پیدا می‌کند یعنی مقدار آستانه به حدی بالا می‌رود که هیچ تحریکی نمی‌تواند از آن عبور کند به همین دلیل نورون در این بازه فعال نمی‌شود بعد از سپری شدن این زمان کوتاه، مجددا θ کاهش پیدا می‌کند و به مقدار قبلی اش برمی‌گردد بنابراین این تابع، ناتوان شدن نورون بعد از شلیک را مدل می‌کند.

$$P_v(t) = \sum_u \sum_{s \in F_u} w_{u,v} \cdot \varepsilon_{u,v}(t-s)$$

u ها عام نورون‌هایی هستند که به نورون v وصل اند و خروجی شان وارد نورون v می‌شود

s ها زمان‌های شلیک نورون‌های u اند

مدلی که گفتیم، مدل نورون ضربه‌ای بود.

Scanned with CamScanner

دسته بندی نورون ها یا گره های گراف:

① نورون های نسل اول ← bipolar / binary ← یعنی خروجی شان یا ۰ و ۱ بود یا ۱- و ۱. مثال: پرسپترون، شبکه Hopfield، شبکه Boltzman

② نورون های نسل دوم ← نورون هایی که خروجی شان پیوسته است مثلا در شبکه های MLP، RBF و RNN چنین نورون هایی داریم.

③ نورون های نسل سوم ← نورون های ضربه ای، مثلا در شبکه های feed forward ضربه ای یا در شبکه های RBF ضربه ای. پیچیدگی بالایی دارند. عملکردشان بهتر از نورون های غیرضربه ای است

اگر فرض کنیم تغییرات خروجی نورون به طور پیوسته صورت می گیرد می توانیم آن را با یک معادله دیفرانسیل مرتبه اول مدل کنیم:

$$\frac{dy}{dt} = -g(y) + \sum_{j=0}^{n} w_j . x_j = -g(y) + I \qquad \boxed{I = \sum_{j=0}^{n} w_j . x_j}$$

Net Input

می خواهیم نشان بدهیم این مدل با مدل استاتیک که تابع دانستیم ها هنگ است:

در حالت تعادل: $dy/dt = 0$ چون خروجی ثابت است $\Rightarrow g(y) = I$

$$\Rightarrow \boxed{y = g^{-1}(I) = g^{-1}\left(\sum_{j=0}^{n} w_j . x_j\right) = f\left(\sum_{j=0}^{n} w_j . x_j\right)}$$

$$\Rightarrow \boxed{y = f\left(\sum_{j=0}^{n} w_j . x_j\right)}$$

همان رابطه ای است که برای نورون استاتیک داشتیم

به عنوان مثال اگر $f(x) = \tanh(\beta . x)$ آن گاه g که معکوس f است می شود:

$$g(x) = -\left(\frac{1}{\beta}\right)\left[\ln(1-x)/(1+x)\right]$$

Scanned with CamScanner

Network Connections (links)

اتصالات شبکه

گفتیم که یال ها جهت و وزن دارند. این وزن ها قرار است در یک فاز یادگیری یا آموزش، یاد گرفته شوند. تنها مجهولات ما در شبکه همین وزن ها هستند.

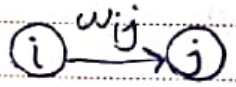
برای نوع داده‌ای وزن‌ها هیچ محدودیتی نداریم (حقیقی، مختلط و...)

- وزن مثبت ← تحریک $w>0$
- وزن منفی ← ممانعت $w<0$
- وزن صفر ← اتصالی وجود ندارد $w=0$

در مورد نوع داده‌ای وزن‌ها همواره باید توجه به مسئله باید تصمیم بگیریم

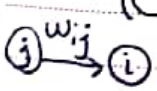
ارایه (بازنمایی) وزن‌ها:

در انتخاب درایه‌ها برای ماتریس وزن دو تا قرارداد وجود دارد:



① w_{ij} وزن اتصال نورون i به نورون j را نشان دهد

② w_{ij} یعنی وزنی که به مقصد j می‌آید از مبدأ i (جهتش برعکس قرارداد اول است)



روش بیشتر به کار رفته چون بعضی از روابط ریاضی آسان‌تر می‌شود

ماتریس وزن در این دو حالت Transpose همدیگر است.

نوع اتصالات:

① full connection (تام اتصال): خروجی هر نورون به تمام نورون‌های لایه بعد داده می‌شود

② uniformly Random Connection

تعدادی از خروجی‌های این لایه به صورت تصادفی یکنواخت انتخاب می‌شوند و به هر نورون در لایه بعد وصل می‌شود (برای هر نورون خروجی‌هایی که وارد آن می‌شود متفاوت خواهد بود و به صورت اتفاقی یکنواخت از خروجی لایه قبل انتخاب می‌شود، تعداد ورودی‌های نورون‌ها یکسان است)

Scanned with CamScanner

③ انتقال یک به یک ← خروجی هر نورون فقط به یک نورون در لایه بعد متصل می‌شود

ساختار (معماری) شبکه Network Architectures

① Feed forward (جلورو)

سیگنال همیشه رو به جلو می‌رود و برنمی‌گردد. پس گراف شبکه مسیر برگشت و مسیر بسته ندارد

این شبکه‌ها، شبکه‌های استاتیک هستند. وقتی یک ورودی به آن می‌دهیم یک خروجی برای

ما تولید می‌کند

خروجی تولید شده به ازای این ورودی وابسته به حالت قبلی نیست، به عبارت دیگر شبکه

حافظه ندارد

② Recurrent (بازخوردار)

برخلاف شبکه FF در گراف شبکه مسیر بسته وجود دارد. می‌گوییم شبکه دینامیک است.

وقتی یک ورودی اعمال می‌شود خروجی شبکه ممکن است مرتب عوض شود. علت این تغییرات

همان فیدبک‌هاست مثلا یک سیگنال تابک جای جلو می‌رود و بعد برمی‌گردد به ورودی نورون‌ها

وارد می‌شود و ممکن است خروجی این‌ها را عوض کند و قتی خروجی‌ها عوض شد اثر آن دوباره

در شبکه جلو می‌رود و خروجی شبکه را عوض می‌کند. بنابراین این شبکه‌ها حالتی دارند که خروجی آنقدر

تغییر می‌کند تا جایی که ثابت شود و این مقدار ثابت می‌شود پاسخ شبکه به ورودی مورد نظر.

این شبکه‌ها این خطر را هم دارند که خروجی هیچ‌گاه ثابت نشود و ناپایدار باشند، به طور کلی در

تئوری سیستم‌ها وجود فیدبک می‌تواند باعث ناپایداری شود

Scanned with CamScanner

جلسه پنجم - ۱۴۰۰/۷/۱۱

ساختار شبکه باید تعیین شود و پارامترهای آن نیز باید مشخص شود. پارامترها در یک فاز یادگیری در اثر تعامل با محیط (در اثر تعامل با داده‌هایی که از محیط گرفته شده) یادگیری می‌شوند

انواع یادگیری (Learning paradigms):

① supervised Learning (یادگیری با نظارت / با معلم)

در زمان آموزش، هر ورودی که به شبکه داده می‌شود، خروجی متناظرش هم داده می‌شود.

② unsupervised Learning (یادگیری بی نظارت)

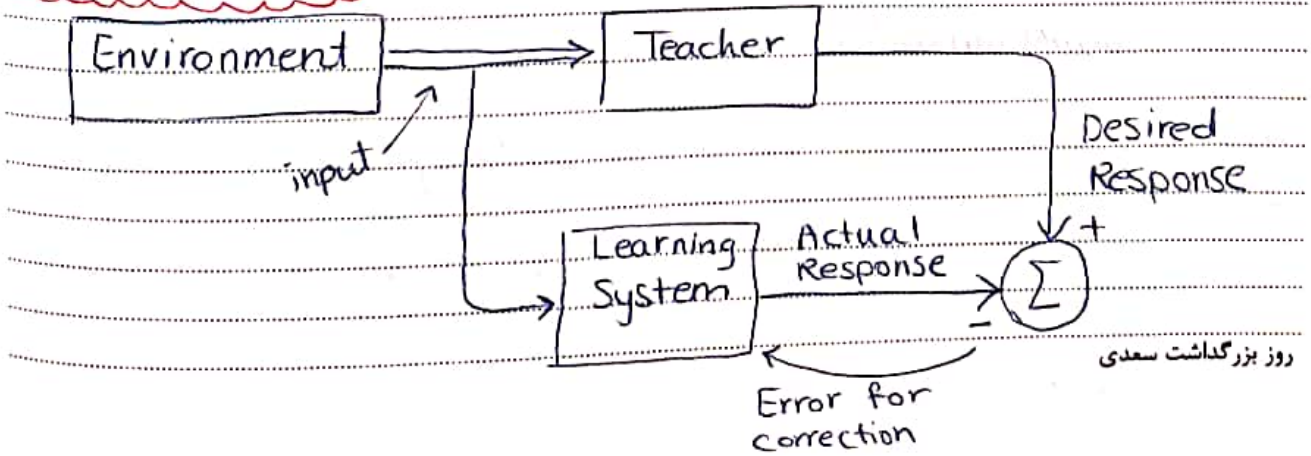
در فاز آموزش، داده‌های ورودی را به شبکه می‌دهیم اما خروجی را برای آن مشخص نمی‌کنیم

فقط داده‌های ورودی را می‌دهیم و شبکه باید تا خودش یاد بگیرد که چه خروجی‌ای به ما بدهد

③ Reinforcement Learning (یادگیری تقویتی) ← Graded Learning - Learning with a critic

بین دو حالت بالا است. به ازای ورودی‌ها، خروجی را مشخص نمی‌کنیم برای سیستم اما یک سیگنالی می‌دهیم که سیستم را راهنمایی می‌کند. در واقع یک فیدبکی از جانب محیط به سیستم داده می‌شود تا بفهمد خوب عمل کرده یا نه، که می‌تواند هزاره با تاخیر هم باشد.

Supervised Learning:



روز بزرگداشت سعدی

Scanned with CamScanner

اردیبهشت | ۲ | 22 Apr 2021 | ۹ رمضان ۱۴۴۲ | پنجشنبه

supervised Learning

یک ناظر داریم که اطلاعات از آن دریافت می‌شود و می‌شود input برای سیستم ما. این اطلاعات (ورودی‌ها) به معلم (Teacher) هم داده می‌شود. معلم مشخص می‌کند خروجی مطلوبی که این سیستم باید داشته باشد چه چیزی است. این خروجی مطلوب با پاسخی که سیستم در ازای این ورودی تولید کرده مقایسه می‌شود و یک خطایی به دست می‌آید.

این خطا به یادگیری سیستم ما کمک می‌کند. سیستم سعی می‌کند به گونه‌ای عمل کند که این خطا را به حداقل برساند.

معمولا در این نوع آموزش یک جفت ورودی و خروجی مطلوب داریم. یعنی هرگاه یک ورودی به سیستم می‌دهیم، به ازای آن ورودی، خروجی مطلوب را هم می‌دهیم.

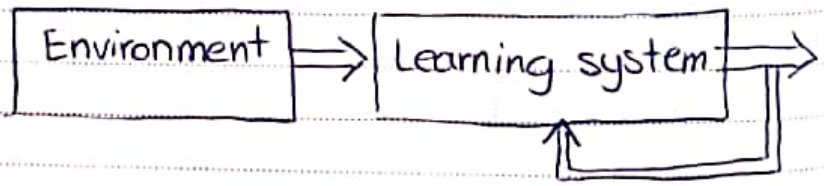
برای اینکه فرآیند آموزش صورت بگیرد، لازم است که سیستم کار آموزش را خیلی تکرار کند.

مثال‌ها: BP و LMS و perceptron

تاسیس سپاه پاسداران انقلاب اسلامی

unsupervised Learning

اردیبهشت | ۳ | 23 Apr | ۱۰ رمضان | جمعه



داده‌ها از محیط به سیستم داده می‌شود و سیستم باید خودش یاد بگیرد. مثلا سیستم ممکن است یاد بگیرد ویژگی‌هایی را از داده‌ها استخراج کند. یعنی یادگرفتن ویژگی‌های داده‌ها: ما داده‌هایی را به سیستم می‌دهیم و سیستم در خروجی ویژگی‌هایی از داده‌ها را به ما می‌دهد. یا مثلا در داده‌ها نظم پیدا می‌کند یا الگو پیدا می‌کند. وابستگی‌هایشان را پیدا می‌کند. خوشه‌هایی که در داده‌ها هست را پیدا کند و...

(گاهی اوقات برای استخراج ویژگی‌هایی بین از شبکه‌های بی‌نظارت استفاده می‌کنن)

وفات حضرت خدیجه (س) - روز بزرگداشت شیخ بهایی - روز معماری

Scanned with CamScanner

ممکنه سیستم یادبگیره که میزان شباهت ورودی به یک الگوی خاص رانه ارزش کند

یا ممکنه چندین الگو رو در نظر بگیره و میزان شباهت ورودی به هرکدام را در خروجی نشان دهد

ممکنه خوشه بندی کنه داده ها رو.

ممکنه به نقشه دوبعدی از داده هایی که ابعاد بالا دارند تولید کند و این نقشه معنی دار باشد. نقطای

مهمی را به فضای ۲ بعدی نگاشت کند

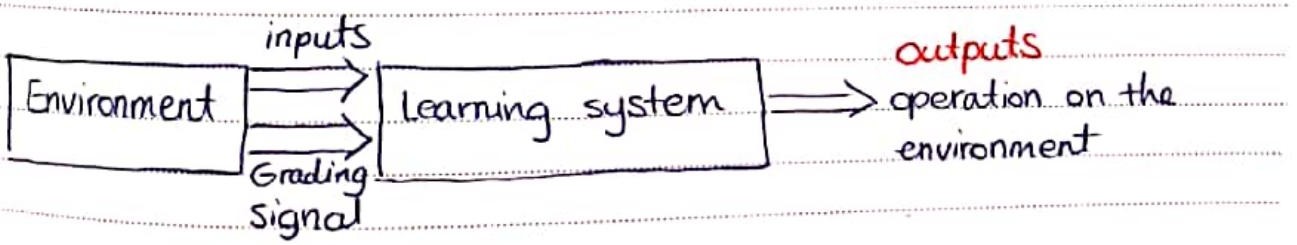
یادگیری تقویتی (Reinforcement Learning)

سیستم یادگیرنده از محیط اطلاعاتی را می گیرد هم چنین یک ورودی دیگر هم دارد که این ورودی

یک امتیاز یا ارزیابی است که راجع به کار خودش دریافت می کند.

این سیستم در ازای هر ورودی که دریافت می کند یک عمل انتخاب می کند که باید روی محیط انجام شود

درواقع سیستم یادگیری خواهد یادبگیرد که عملی روی این محیط انجام دهد.



عملی که سیستم روی محیط انجام می دهد، روی محیط اثر می گذارد و سپس به این می شود که یک سری

داده های جدید از محیط تولید شود و به سیستم یادگیرنده داده شود و در عین حال یک ارزیابی هم

از عمل تعیین شده توسط سیستم مشخص می شود و به سیستم کمک می کند که بتواند عمل های بهتری

را انتخاب کند

از آن جایی که برای سیستم مشخص می کنیم که به ازای هر ورودی، خروجی چه چیزی باید باشد، قاعدتا جزء

از supervised Learning باید طبقه بندی کند.

این نوع یادگیری معمولا برای ربات ها استفاده می شود.

Scanned with CamScanner

مسائل دسته‌بندی (classification problems)

← دسته‌بندی یک راه تشخیص یا شناسایی است. مثلا برای شناخت سنی x می‌توانیم بررسی کنیم به کدام دسته تعلق دارد. دراسان خیلی از کارهای شناختی با دسته‌بندی صورت می‌گیرد.

← یک دسته‌بندی کننده (classifier) یادمی‌گیرد که نمونه‌هایی را از دسته‌های مختلف ببیند و یاد بگیرد که نمونه‌های جدید به کدام دسته تعلق دارد.

داده‌هایی که به دسته‌بندی کننده داده می‌شود ممکن است داده‌های خام باشد یا ویژگی‌هایی باشد که از داده‌های خام استخراج شده است.

دسته‌بندی معادل بخش‌بندی کردن فضای ورودی است. مثل این است که فضای ورودی را مثل یک صفحه سفید در نظر بگیریم و در آن خطوطی را رسم کنیم و ناحیه‌هایی را مشخص کنیم و هر کدام از این ناحیه‌ها، فضایی را که متعلق به یک دسته خاص هستند را مشخص کند.

مسئله دسته‌بندی: فرض کنیم مجموعه‌ای از نمونه‌ها داریم که با بردارهای x نشان می‌دهیم و این‌ها از k تا دسته هستند. هدف این است که دسته‌بندی کننده‌ای طراحی کنیم که با استفاده از داده‌های x یاد بگیرد که چگونه دسته‌بندی نمونه‌ها را به این k تا کلاس انجام بدهد

فرض کنیم مسئله ۲ دسته‌ای باشد. تابعی به نام تابع تمایز خطی

$$y(X) = w^t X + w_0$$

در نظر می‌گیریم:

اگر $y(X) \geq 0$ باشد x به دسته‌ی C_1 و اگر $y(X) < 0$ باشد x به دسته‌ی C_2 تعلق دارد

(فرض کنیم فضای ورودی ۲ بُعدی باشد (برای اینکه در صفحه به راحتی نشان بدهیم)

در این صورت مرز تصمیم (decision boundry) می‌شود: $y(X) = 0$

در واقع w عمود بر خط مرز تصمیم است

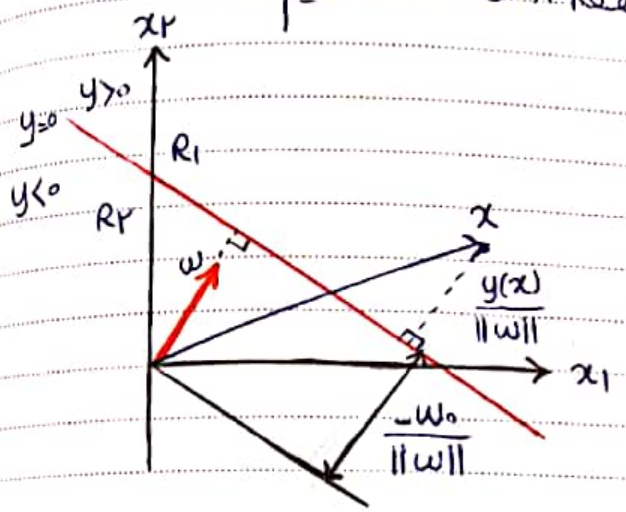
شکست حمله نظامی آمریکا به ایران در طبس (۱۳۵۹ هـ ش)

Scanned with CamScanner

۱۴۰۰

اینکه فاصله این خط تا مبدأ چقدر باشد توسط w_0 مشخص می‌شود ← $\frac{-w_0}{\|w\|}$

و یک بخش علامتدار از فاصله‌ای است که نقطه x از خط مرز تصمیم دارد



در حالت K دسته‌ای، K تا تابع داریم:

$$y_k(X) = w_k^T X + w_{k0} \qquad k = 0, 1, \ldots, K$$

x را متعلق به کلاس k می‌دانیم اگر: $y_k(X) > y_j(X) \quad \text{for all } j \neq k$

یعنی x ای که داریم را در K تا معادله می‌گذاریم و معادله‌ای که بزرگترین مقدار y را برای دهد، دسته را تعیین می‌کند

مرز تصمیم بین دو کلاس C_k و C_j دارای معادله روبه رو است: $y_k(X) = y_j(X)$

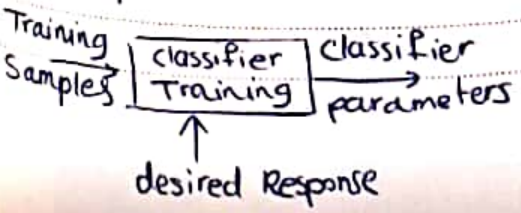
معادله مرز بین دسته k و j: $(w_k - w_j)^t X + (w_{k0} - w_{j0}) = 0$

۲ فاز دسته بندی:

① classifier Training (Learning) (آموزش دسته بندی کننده)

یعنی همان یادگیری پارامترها، اینکه خط‌ها چگونه رسم شوند در صفحه و تقسیم بندی فضا چگونه صورت بگیرد.

در این فاز نمونه‌های آموزشی را به همراه مقادیر یا پاسخ‌های مطلوب به classifier می‌دهیم و در این فاز یادگیری پارامترهای classifier مشخص می‌شود.

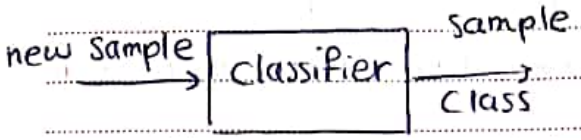


Scanned with CamScanner

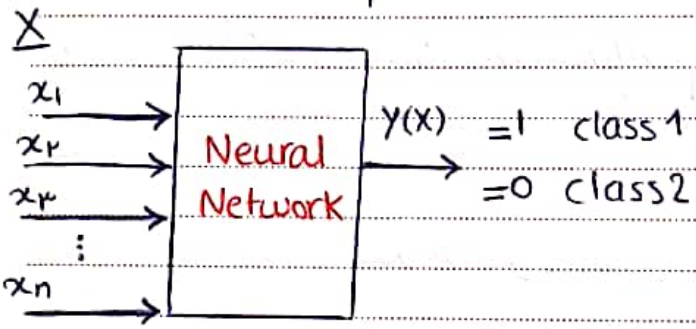
② classifier Testing (use)

② فاز تست (فاز استفاده از دسته‌بندی کننده)

یک نمونه به classifier می‌دهیم و در خروجی دسته‌ی نمونه را برای ما مشخص می‌کند



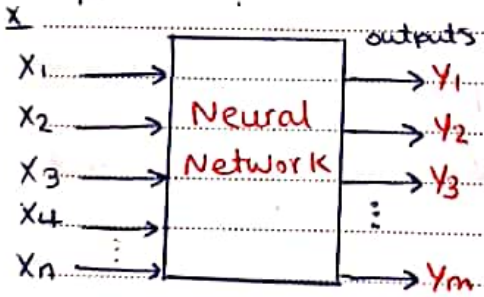
اگر خواهیم عمل دسته‌بندی را با استفاده از یک شبکه عصبی انجام بدهیم:



فرض کنید داده ورودی n بُعدی باشد و دسته‌بندی ۲ کلاسه می‌خواهیم انجام بدهیم

دو دسته را با یک خط خروجی می‌توان نشان داد. اگر بیش از دو کلاس و اگر به بیش از کلاس ۲

اگر m تا دسته داشته باشیم: ① یک راه این است که به ازای هر دسته ۱ خروجی قرار بدهیم و



در این صورت m تا خط خروجی خواهیم داشت

در این صورت اگر ورودی به کلاس i ام تعلق داشته باشد

خط خروجی مربوط به آن فعال می‌شود

② یک راه دیگر این است که از coding استفاده کنیم. برای نشان دادن m تا خروجی،

$\log_2^m$ خط کافی است. مثلا اگر ۸ تا دسته داشته باشیم یک که ۳ بیتی کافی است و با ۳

خط می‌توانیم نشان بدهیم.

معمولا از حالت اول استفاده می‌کنیم و به ازای هر خروجی یک خطی گذاریم چون ساده‌تر است.

Scanned with CamScanner

۱۴۰۰

مثال: می خواهیم سیستمی بسازیم که هر فردی به آن عرضه می کنیم مشخص کند که آیا یک بسکتبالیست است یا وزنه بردار. فرض هم بر این است که هر فردی که به این سیستم می دهیم حتما یا وزنه بردار است یا بسکتبالیست. (دسته بندی دو کلاسه)

راه حل:

چه ویژگی هایی هست که این دو دسته را از هم متمایز می کند؟ انتخاب ویژگی ها یک مسئله مهم است.

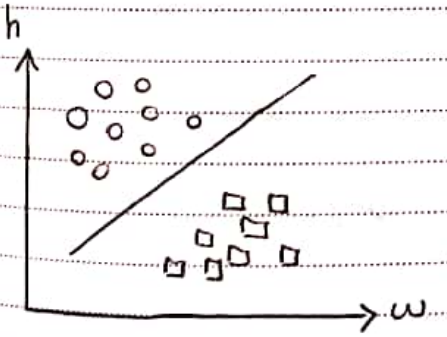
در این مسئله دو ویژگی قد و وزن کمر را در نظر می گیریم: $X = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$

چون این مسئله دو کلاسه است فضای ورودی باید به ۲ قسمت تقسیم شود.

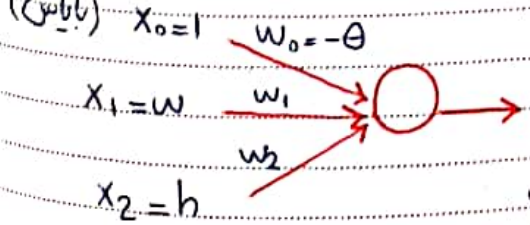
فاز آموزش شبکه به این صورت است: تعداد زیادی نمونه از وزنه بردارها و بسکتبالیست ها نیاز داریم داده

و نمودار مورد نظرمان را در مورد هر کدام اندازه گیری کنیم.

اگر داده ها را رسم کنیم چنین شکلی داریم:



اگر یک نورون داشته باشیم بخواهیم مسئله را حل کنیم:



$$y = f(I) : \begin{cases} 1 & I \geq 0 \quad \text{class 1} \\ -1 & I < 0 \quad \text{class 2} \end{cases}$$

$$y = f(I) = f\left(\sum_{i=0}^{2} w_i x_i\right)$$

$$X = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} \quad W = \begin{bmatrix} -\theta \\ w_1 \\ w_2 \end{bmatrix} \Rightarrow I = -\theta + w_1 x_1 + w_2 x_2$$

Net Input

Scanned with CamScanner

اردیبهشت | ۹ | 29 Apr 2021
پنجشنبه | ۱۶ رمضان ۱۴۴۲

برای یافتن مرز تصمیم $I = 0$ قرار می‌دهیم:

$$w_1 x_1 + w_2 x_2 - \theta = 0 \Rightarrow x_2 = -\frac{w_1}{w_2} x_1 + \frac{\theta}{w_2}$$

← معادله خط مرز تصمیم

اگر این خط را بکشیم انتظار داریم که بسکتبالیست‌ها یک طرف و وزنه‌بردارها طرف دیگر آن قرار گیرند

داده‌های آموزشی

اگر مسئله به گونه‌ای باشد که پس از یافتن خط مرز تصمیم، همه‌ی داده‌های دسته اول یک طرف خط و همه‌ی نمونه‌های دسته‌ی دوم طرف دیگر خط قرار بگیرند، می‌گوییم این مسئله جدایی‌پذیر خطی است

در غیر این صورت جدایی‌پذیر خطی نیست

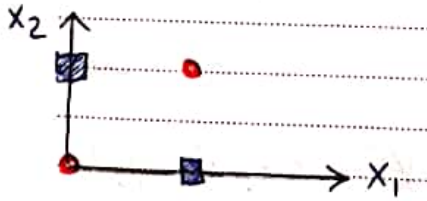
Minsky نشان داد که پرسپترون فقط می‌تواند یک خط بکشد و فقط می‌تواند مسائل جدایی‌پذیر خطی را حل کند. بنابراین پرسپترون بسیاری از مسائل را نمی‌تواند حل کند.

مسئله XOR

روز شوراها

اردیبهشت | ۱۰ | 30 Apr
جمعه | ۱۷ رمضان

مثلا مسئله‌ی ساده‌ای مثل XOR جدایی‌پذیر خطی نیست



$x_1 x_2$	output	class
00	0	1
11	0	1
01	1	2
10	1	2

این مسئله ماهیتاً linearly separable نیست یعنی نمی‌توان هیچ خطی در صفحه رسم کرد که داده‌های ۲ کلاس را از هم جدا کند.

مسئله XOR با یک perceptron (پرسپترون) قابل حل نیست.

روز ملی خلیج فارس

Scanned with CamScanner

۱۴۰۰

Single Layer FeedForward Neural Networks (شبکه‌های عصبی رو به جلو تک لایه)

(Hebb processing Unit) ① واحد پردازش هبب

یادگیری هم وقوعی coincidence learning:
- اگر دو نورون هم‌زمان فعال باشند ارتباط بین آن‌ها باید تقویت شود
- اگر دو نورون هم‌زمان خاموش باشند ارتباط بین‌شان باید تقویت شود

الگوریتم Hebb:

۱) مقداردهی اولیه وزن‌ها $w_i = 0 \quad (i=1 \text{ to } n)$

۲) فرض کنیم یک نمونه آموزشی (s,t) داریم، s ورودی و t خروجی مطلوب متناظر با آن است.

برای هر نمونه ورودی آموزشی مثل (s,t) این دو مرحله را انجام بده:

۳) $x_i = s_i \quad i=1,2,\dots,n$

$y = t$

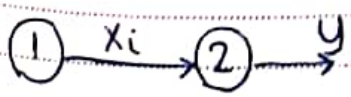
۴) مقدار وزن‌ها را اصلاح کن با این قاعده:

$$w_i^{new} = w_i^{old} + x_i y \qquad i=0,1,2,\dots,n$$

۵) پایان

سوال: قاعده‌ی به هنگام شدن وزن‌ها چه ارتباطی با قانون Hebb دارد؟

با آمدن هر نمونه، جمله‌ی $x_i y$ به وزن قبلی اضافه می‌شود، اگر $x_i y$ یک عدد مثبت باشد وزن مورد نظر افزایش پیدا می‌کند و تقویت می‌شود. $x_i y$ در صورتی مثبت است که نه x_i و نه y صفر نباشند. x_i و y یا ۱ اند یا ۱-. اگر هر کدام‌شان ۰ باشد وزن تقویت نمی‌شود اما اگر هر دو یک یا منفی یک باشند تقویت می‌شود. چون مانند شکل روبه رو،



ورودی و خروجی نورون دوم ۱ است. خروجی نورون اول هم ۱ است پس وزن بینشان باید تقویت شود

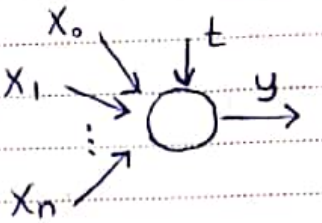
نکته: در تابع فعال‌سازی Hebb نورون دو حالته است. خروجی این تابع ۰ و ۱ است.

Scanned with CamScanner

جلسه ششم - ۱۴۰۰/۷/۱۸

قانون آموزش perceptron:

$$w^{new} = w^{old} + (t-y)x$$



مقدار مطلوب در فاز آموزش از طریق ورودی t به نورون گفته می‌شود.

Roseblatt ثابت کرد که این الگوریتم از هر مقدار اولیه‌ی w^{old} که شروع بکنیم در یک تعداد دوره iteration همگرا می‌شود و به جواب می‌رسد. در نتیجه این قانون شد قانون یادگیری پرسپترون و این اولین مدل نورونی بود که به طور جدی معرفی شد.

در این مدل نورون n تا ورودی می‌گیرد ← $x_1, x_2, \dots, x_n$ و یک ورودی bias ← x_0

و یک ورودی t هم دارد.

در ادامه Minsky پیشنهاد کرد که یک ضریب α در جمله $(t-y)x$ ضرب شود که $\alpha < 1$ و در واقع هر بار اصلاح با مقدار کمتری صورت بگیرد و به همگرایی سریعتر کمک کند

$$w^{new} = w^{old} + \alpha(t-y)x$$

و پیشنهاد داد که مقدار α این باشد:

$$\alpha = \frac{1}{\|x\|} \qquad \alpha = \frac{x \cdot w}{\|x\|^2}$$

در ادامه Rosenblatt در دانشگاه Cornell یک سیستم درست کرد که اسمش رو Mark I perceptron گذاشت.

می‌خواست شبکیه چشم انسان را شبیه‌سازی کند با یک نورون. می‌خواست تصویر حروف را روی آرایه‌ای از سنسورها بیندازد و می‌خواست از روی این تصویر نورون تشخیص بدهد که چه حرفی بوده است.

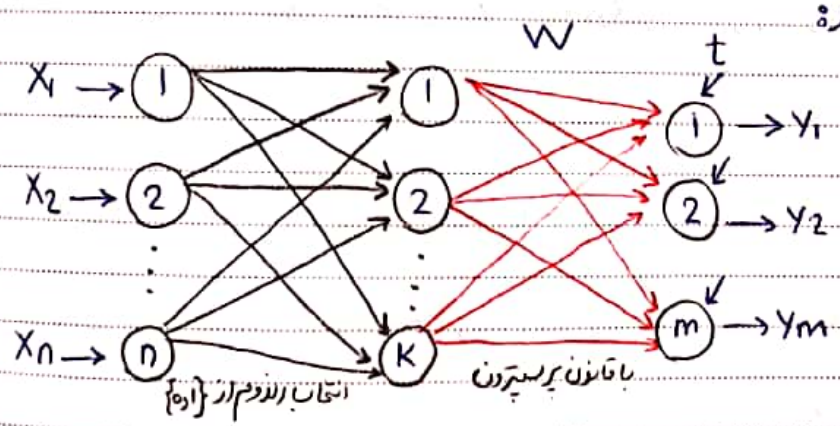
در آن زمان همه چیز هنوز الکتریکی بود، الکترونیک و دیجیتال اصلا پیشرفتی نکرده بود. خوش بسنسوری درست کرد که بتونه تصویر حرف رو Sense بکنه.

Scanned with CamScanner

۱۴۰۰

آرایه‌ای از سلول‌های حساس به نور را ایجاد کرد که 20*20 بود یعنی ۴۰۰ تا سلول.
۴۰۰ تا سنسور را در کنار هم قرار داد و ۴۰۰ تا سیگنالی که این‌ها دریافت می‌کردند را به سیستمش می‌داد تا سیستم بتواند حرف مورد نظر را تشخیص بدهد. برای این تشخیص لازم بود تا شبکه‌ای از پرسپترون‌ها ایجاد شود و با یک پرسپترون این عمل ممکن نبود. یک شبکه ۳ لایه از پرسپترون‌ها ایجاد کرد.
مقدار هر وزن را با یک مقاومت نشان می‌داد و این وزن‌ها را تغییر می‌داد. به این صورت که پتانسیومتر قرار داده بود و شفت آن را به شفت یک موتور متصل کرده بود. وقتی موتور تحریک می‌شد خود آن می‌چرخید و این چرخش شفت پتانسیومتر را می‌چرخاند و نتیجه این می‌شد که مقاومت تغییر میکرده. بدین ترتیب این سیستم می‌توانست به صورت سخت افزاری یاد بگیرد که این مقدار مقاومت چقدر باید باشد.

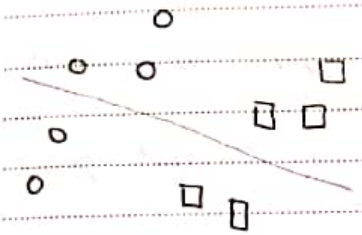
ساختار شبکه‌ای که ایجاد کرده بود:



در لایه دوم K تا نورون و در لایه سوم m تا نورون قرار دارد. m مقدار خروجی است که می‌خواهد شناسایی کند. در نتیجه m تا خروجی داریم. نورون‌های لایه اول فقط وظیفه‌شان این است که ۴۰۰ تا ورودی را به لایه بعد توزیع کنند. نورون‌های لایه دوم روی ورودی عمل می‌کنند و نتیجه‌شان به لایه بعد داده می‌شود. لایه آخر هم باز روشن به عملی انجام میده و خروجی ایجاد می‌شه. برای پرسپترون‌های لایه آخر مقادیر t را داریم چون می‌دانیم وقتی یک ورودی داده می‌شود خروجی کدام است چیزی باید باشد. پس لایه آخر را می‌توان با قانون پرسپترون آموزش داد. اما در مورد لایه وسط مقدار t را نداریم پس نمی‌توان با قانون پرسپترون آموزش داد. به همین دلیل گفت که من وزن‌های این لایه را به صورت رندوم از ۰ و ۱ انتخاب می‌کنم. پس در لایه وسط یادگیری انجام نمی‌شود.

Scanned with CamScanner

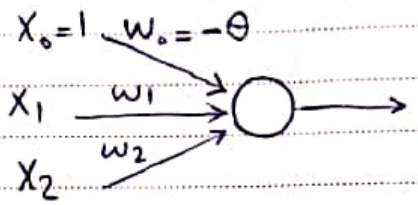
دسته‌بندی با یک پرسپترون



داده‌ها ۲ دسته‌اند ـ دایره و مربع

فضای ورودی ۲ بُعدی است، هر نقطه با ۲ عدد قابل نمایش است

پس نورون به این صورت است:



$$\text{Net Input}: \quad I = -\theta + w_1 x_1 + w_2 x_2$$

می‌خواهیم این نورون را آموزش بدهیم پس از قانون یادگیری پرسپترون استفاده می‌کنیم:

$$w^{new} = w^{old} + (t-y)x$$

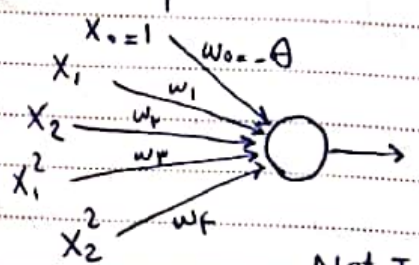
بار اول w^{old} را نداریم به همین دلیل خودمان یک مقداری برایش در نظر می‌گیریم. در نتیجه w_0 و w_1 و w_2 را داریم در شروع کار. پس معادله مرز ($I=0$) را داریم که یک خط است می‌توان آن را رسم کرد.

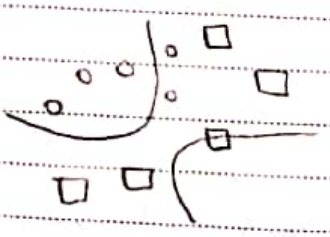
در ادامه یکی یکی ورودی‌های مختلف را اعمال می‌کنیم و چک می‌کنیم آیا مرز تغییر کرده یا نه. وقتی که یک بار همه‌ی ورودی‌ها را اعمال کنیم و مرز تغییری نکند یعنی همگرا شده است. (مرز تغییر نکند یعنی وزن‌ها تغییری نکرده)

دسته‌بندی با پرسپترون مرتبه دوم

داده‌های این مثال به صورت خطی جدایی پذیر نیستند و این مسئله با یک نورون خطی قابل حل نیست

می‌توانیم یک نورون درجه ۲ در نظر بگیریم. به این معنا که علاوه بر ورودی‌ها، درجه دوم آن‌ها را هم به عنوان ورودی بدهیم.





$$\text{Net Input}: \quad I = -\theta + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2$$

Scanned with CamScanner

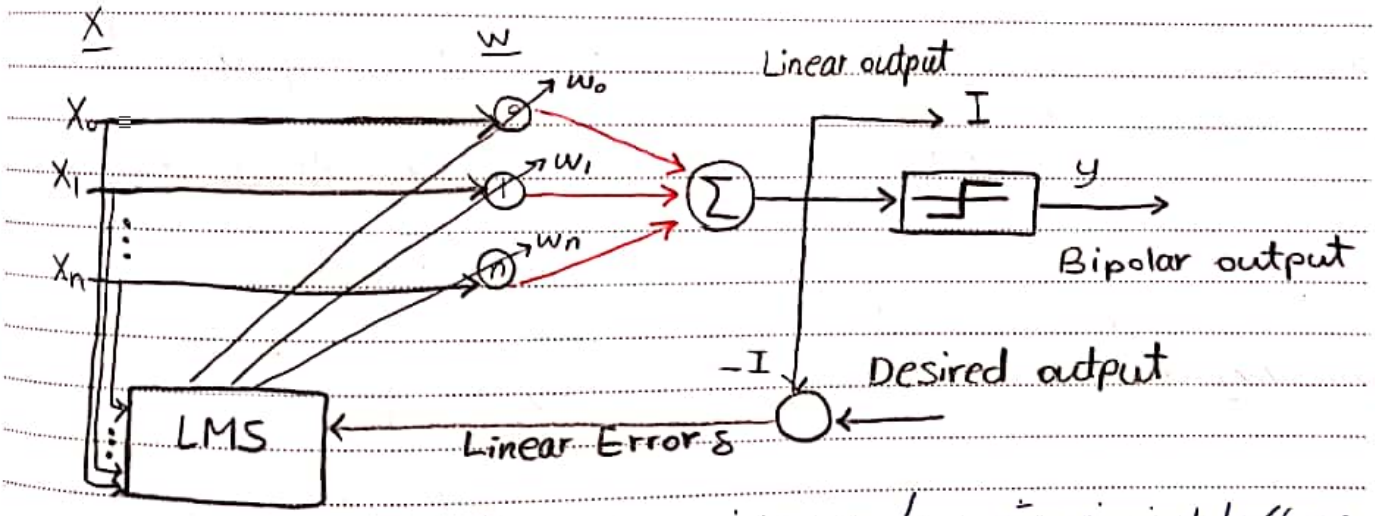
۱۴۰۰

فرد با یک مقدار اولیه برای w^{old} در نظر می‌گیریم و در ادامه ورودی‌های مختلف را اعمال می‌کنیم و تغییرات مدرز را بررسی می‌کنیم تا زمانی که همگرا شود.

نورون غیرخطی قدرت بیشتری دارد و می‌تواند مسائلی که جدایی‌پذیر خطی نیستند را حل کند اما در عوض هزینه‌ای دارد. در نورون خطی ۳ تا پارامتر بود که باید یادگرفته می‌شد اما در این نورون مرتبه‌ی مقدار پارامترها ۵ تاست بنابراین آموزش هزینه بیشتری دارد. اگر مسئله‌ای از این نورون‌ها داشته باشیم مقاومت زیادی با مسئله‌ای از نورون‌های خطی دارد. (مقدار وزن‌ها یکباره خیلی بزرگ بالا و پایین نرود و سنگین‌تر می‌شود)

The Adaline processing Unit

Adaptive Linear Element → Adaline



ورودی‌ها وارد نورون می‌شود، در یک سری وزن‌های ضرب می‌شود، نتایج با هم جمع می‌شوند و اسم خروجی را I می‌گذاریم که Linear output است. I را به یک تابع علامت می‌دهد و خروجی را Bipolar output می‌نامیم با y نشان می‌دهیم که یا ۱ است یا ۱-. خروجی خطی را با مقدار مطلوب خروجی مقایسه می‌کنیم و تفاضل آن‌ها می‌شود خطای خطی (δ). این خطا را به همراه x_iها به الگوریتم LMS می‌دهیم و تعیین می‌کند که وزن‌ها چگونه باید تغییر کنند

Scanned with CamScanner

رابطه ریاضی برای AdaLine:

$$I = w_0 x_0 + w_1 x_1 + \dots + w_n x_n = X.W = X^T W = W^T X$$

$$y = sgn(I)$$

در دسته‌ی performance learning از قوانین یادگیری قراردادی گیرد. براساس عملکرد کاری کند خطا را حداقل میکند

در این مدل، وزن‌ها توسط قانون LMS تعیین می‌شوند. گاهی از این قانون با عنوان

LMSE (Least Mean Square Error) نیز یاد می‌شود.

قاعده LMS ← Least Mean Square ← وزن‌ها به مقدار بهینه همگرا می‌شوند، فارغ از مقدار اولیه

وزن‌ها را به گونه‌ای تغییر می‌دهد که متوسط مربع خطا، حداقل شود

قاعده یادگیری AdaLine: (N تعداد نمونه‌ها)

تابع خطا: $F(w) = \lim_{N\to\infty} \frac{1}{N} \sum_{k=1}^{N} (t_k - I_k)^2 = E[(t_k - I_k)^2]$

$$\Rightarrow F(w) = E[(t_k - I_k)^2] = E[(t_k - w^T x_k)^2]$$

$$= E[t_k^2 - 2 t_k w^T x_k + w^T x_k x_k^T w]$$

$$= \underbrace{E[t_k^2]}_{P} - 2w^T \underbrace{E[t_k x_k]}_{q} + w^T \underbrace{E[x_k x_k^T]}_{R} w = P - 2w^T q + w^T R w$$

P: constant scalur — t_k هر یک عدد است

q: vector — بردار

R: matrix — ماتریس

quadratic form — معادله درجه ۲ نسبت به w

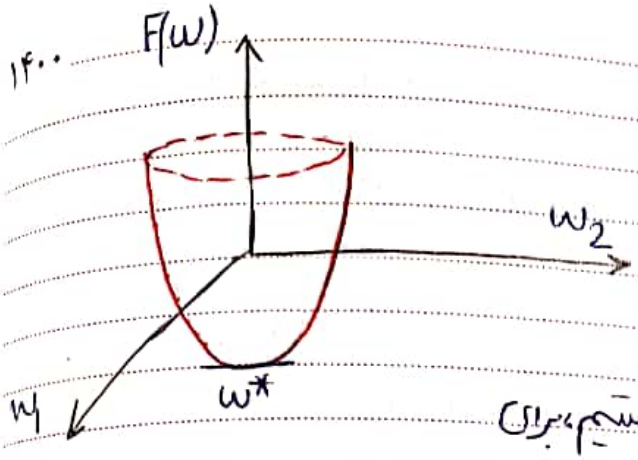
ماتریس R یک ماتریس متقارن است و مثبت معین است و این رابطه در مورد عناصر آن

$R : (n+1) \times (n+1)$

برقرار است: $\forall X : \sum\sum r_{ij} x_i x_j \geq 0$

روز جهانی قدس (آخرین جمعه ماه رمضان)

Scanned with CamScanner



همانطور که گفتیم F نسبت به W ها درجه ۲ است

در فضای وزن ها، F چنین شکلی دارد:

شکل سهمی گون (paraboloidal) دارد

این سطح یک minimum دارد که ما به دنبال آن هستیم، برای

یافتن min، از F(W) نسبت به W مشتق می گیریم:

$$\nabla_W F(W) = -2q + 2RW = 0$$

$$\Rightarrow RW^* = q \quad \Rightarrow W^* = R^+ q$$

R^+ ← شبه معکوس یا معکوس تعمیم یافته R (pseudo-inverse or generalized inverse)

اگر R معکوس پذیر باشد: $R^+ = R^{-1}$ و یک minimum یکتا دارد شبیه شکل بالا.

این سطح را می گویند non-degenerate ← هموار

$$W^* = R^{-1} q$$

اگر R معکوس پذیر نباشد سطح متفاوت می شود و هموار نیست، ممکنه minimum های مختلف

داشته باشه و اگر چند min باشه تریکترین min پیدا را به ما می دهد: $W^* = R^+ q$

در این حالت سطح را degenerate می گویند ← ناهموار است

ما به R و q نیاز داریم اما پیدا کردن آن ها کار راحتی نیست. به همین دلیل راه حل دیگری پیشنهاد شد

اینکه بیایم روی سطح حرکت کنیم و آنقدر پایین برویم که به min برسیم. اگر در جهت عکس گرادیان F نسبت

به W حرکت کنیم بعد از iteration به min نزدیکتر می شویم و بهترین جهت برای حرکت است.

گرادیان F جهت بیشترین نرخ افزایش را نشان می دهد و جهت عکس آن، بیشترین نرخ کاهش

را نشان می دهد

Scanned with CamScanner

$$\nabla_W F(W) = \nabla_W \left[\lim_{N\to\infty} \frac{1}{N} \sum (t_k - I_k)^2 \right] = \lim_{N\to\infty} \frac{1}{N} \nabla_W \left[\sum (t_k - I_k)^2 \right]$$

$$= \lim_{N\to\infty} \frac{1}{N} \left[\sum 2\underbrace{(t_k - I_k)}_{\delta_k} \underbrace{\nabla_W (-I_k)}_{-X_k} \right] \qquad I_k = W^T X_k$$

$$\boxed{\nabla_W F(W) = -\lim_{N\to\infty} \frac{1}{N} \sum_{k=1}^{N} 2\delta_k X_k = -2E[\delta_k X_k]}$$

$$\boxed{W_{k+1} = W_k + \alpha \delta_k X_k} \qquad 0 < \alpha < \frac{2}{\lambda_{max} R}$$

$E[\delta_k X_k]$ یعنی $\delta_k X_k$ های مختلف را برای بی نهایت نمونه X_k میانگین بگیریم. widrow گفت بجای اینکه این ریاضی $\delta_k X_k$ را در نظر بگیریم، خود $\delta_k X_k$ را در نظر می‌گیریم در هر iteration، بنابراین فرمول به شکل $W_{k+1} = W_k + \alpha \delta_k X_k$ درآمد. بنابراین این شد قاعده‌ای که x را اعمال می‌کنیم وزن‌ها را به صورت تکراری به هنگام می‌آوریم تا به min برسیم.

- widrow, Hoff Learning law
- LMS learning law
- Delta rule

$\lambda_{max} R$ ← بزرگترین مقدار ویژه‌ی ماتریس R، اما ماتریس R را نداریم

خود widrow پیشنهاد داد که مقدار α در این بازه باشد: $0 < \alpha < \frac{2}{\lambda_{max} R}$

گفتند برای α یک مقدار اولیه‌ای بین $0.01 < \alpha < 10$ در نظر بگیریم مثلا $\alpha = 0.1$

و با سعی و خطا سعی کنید α ای بیابید که مناسب باشد

اگر α خیلی بزرگ باشد باعث می‌شود الگوریتم همگرا نشود و اگر خیلی کوچک باشد مشکلش این است که الگوریتم خیلی کند می‌شود.

یه پیشنهاد دیگه‌ای که داد این بود که: $\frac{1}{10n} < \alpha < \frac{1}{n}$ که n تعداد ابعاد ورودی (X) است

روز بزرگداشت شیخ کلینی

Scanned with CamScanner

دو نسخه دیگر هم از قانون آموزش Widrow وجود دارد:

The batching Version ①

در این نسخه به جای اینکه به ازای هر x اعمال شده w ها را به هنگام در آورد، این کار را نمی کند. میاد x را اعمال می کند، δ را پیدا می کند ولی w را به هنگام در نمی آورد، دوباره x بعدی را اعمال می کند δ را می یابد ولی w را به هنگام در نمی آورد و ... تا به تعداد ورودی. وقتی همه ورودی ها اعمال شدند (یک batch از ورودی)، حالا حاصل ضرب $\delta_k x_k$ را برای همه حساب می کند و میانگین آن را به دست می آورد و w را اصلاح می کند.

در واقع با میانگین از $\delta_k x_k$ ها w را اصلاح می کند و مزیتش این است که سریعتر است.

The Momentum Version ②

در این نسخه یک جمله با ضریب μ به قانون اضافه می کند:

$$w_{k+1} = w_k + \alpha(1-\mu)\delta_k x_k + \mu(w_k - w_{k-1}) \qquad \mu \gg \alpha$$

(eg. $\mu = 0.9, \alpha = 0.1$)

$w_k - w_{k-1}$ تغییرات w در iteration قبلی است. برای اعمال تغییر در w به ضریبی از تغییر قبل را هم در نظر می گیرد علاوه بر ورودی های این دفعه.

یک ضریب $(1-\mu)$ هم گذاشته برای کنترل نقش این دو جمله.

مزیت: کمک می کند از نوسانات زیاد برای وزن ها در یک استاد جلوگیری بشود.

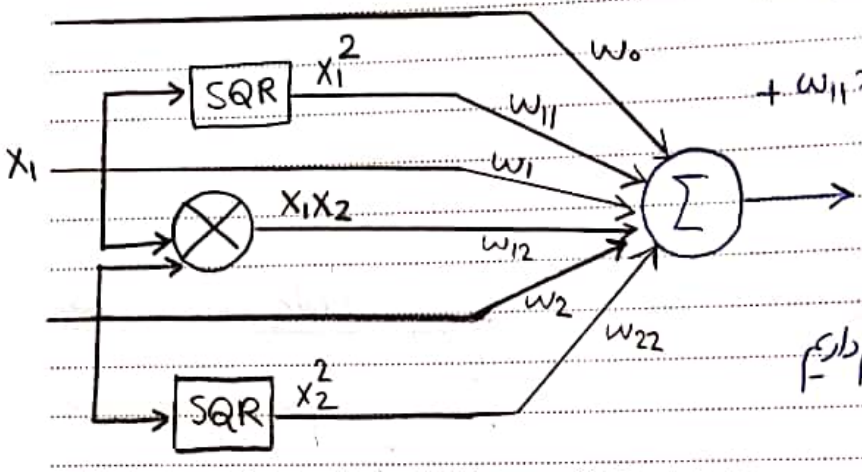
این دو نسخه هر کدام در یک جایی بهتر عمل می کنند و به طور کلی نمی توان گفت کدام یک بهتر است.

Scanned with CamScanner

اردیبهشت | ۲۱ | 11 May 2021
سه‌شنبه | ۲۸ رمضان ۱۴۴۲

Second order AdaLine

$$I = w_0 + w_1 x_1 + w_2 x_2 + w_{11} x_1^2 + w_{22} x_2^2 + w_{12} x_1 x_2$$



مثل پرسپترون درجه ۱، Adaline درجه ۲ هم داریم

جلسه هفتم ـ ۱۴۰۰/۷/۲۰

شبکه‌های جلورو چندلایه — MultiLayer Feedforward Neural Networks

این شبکه‌ها می‌توانند تعداد لایه‌های زیادی داشته باشند و عمیق شوند. هم‌چنین به آن‌ها MLP (MultiLayer Perceptrons هم گفته می‌شود. (چون از گذشته با پرسپترون آغاز شد)

می‌تواند برای افزایش توانایی شبکه‌های تک لایه در جایی که داده‌های ما به صورت خطی جدایی پذیر نیستند، مورد استفاده قرار بگیرد.

در این شبکه‌ها تابع فعال شدن نورون‌ها حتما باید پیوسته باشد. تابع پله‌ای یا علامت نمی‌تواند باشد. علتش هم این است که لازم داریم از این تابع مشتق بگیریم.

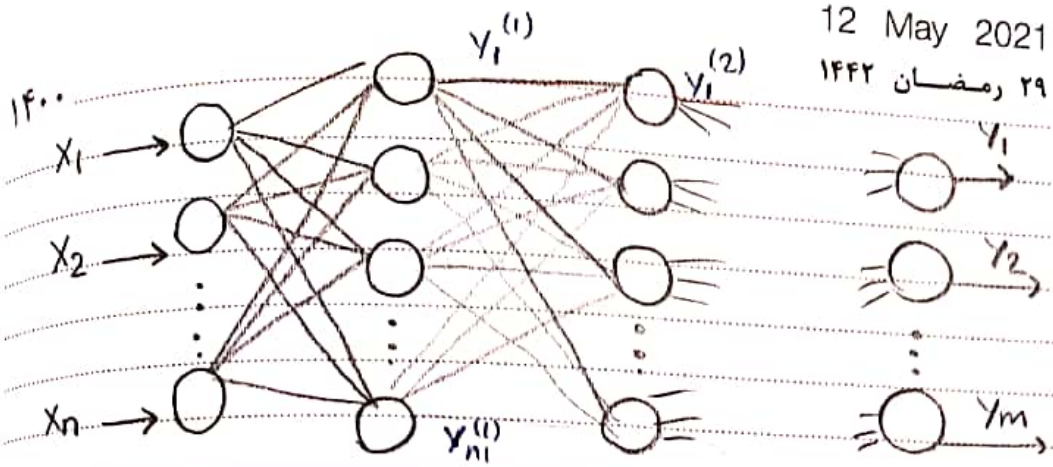
این شبکه‌ها حداقل دارای ۳ لایه هستند: ورودی، یک لایه مخفی، خروجی

تعداد نورون‌ها در لایه‌ی مخفی K را با K نشان می‌دهیم

اتصالات از یک لایه به لایه بعد کامل است. یعنی هر خروجی از نورون در یک لایه به تمام نورون‌های لایه‌ی بعدی متصل می‌شود.

نورون‌های لایه ورودی هیچ تغییری روی x نمی‌دهند و فقط ورودی را به لایه بعد توزیع می‌کنند

Scanned with CamScanner



خروجی لایه مخفی اول: $Y_1^{(1)}$ $Y_2^{(1)}$ ... $Y_{n_1}^{(1)}$

شماره لایه ← $Y_1^{(1)}$ ← شماره نورون

خروجی لایه مخفی دوم: $Y_1^{(2)}$ و $Y_2^{(2)}$ و ... و $Y_{n_2}^{(2)}$

قاعده ای که ابتدا برای آموزش این شبکه پیشنهاد شد:

قاعده پس انتشار خطا — Error backpropagation rule (BP)

قاعده دلتای تعمیم یافته — Generalized Delta Rule

در آموزش این شبکه ۳ مرحله باید طی شود:

① یک ورودی به شبکه اعمال می شود، این ورودی باید تا انتهای شبکه پیش برود و تمام نورون های شبکه، خروجی شان حساب شود. با استفاده از وزن ها و توابع فعال سازی، خروجی ها حساب می شود.

② خروجی هایی که در لایه های خروجی تولید شده با خروجی نظیر مطلوبش مقایسه شود و مقدار خطا محاسبه شود. این خطا می شود، خطا در خروجی شبکه.

از این خطا می توانیم میزان خطا در یک لایه قبل را محاسبه کنیم و مجددا با استفاده از این خطا می توانیم خطاها لایه قبل را محاسبه کنیم و ... تا برسیم به اولین لایه مخفی. اسم این روش از این جا آمده. خطا را از خروجی سمت عقب انتشار می دهد.

③ با استفاده از خطا، تغییر وزن ها را در جهت عکس انتقالات محاسبه می کنیم

Scanned with CamScanner

این ۳ مرحله آنقدر تکرار می‌شود روی ورودی‌های مختلف تا اینکه الگوریتم همگرا شود و خطا به حد قابل قبولی در خروجی برسد.

استخراج قانون BP:

فرض ← در فاز آموزش نمونه‌های ورودی را با S نشان می‌دهیم

$$S_p = \begin{bmatrix} S_{p_1} \\ S_{p_2} \\ \vdots \\ S_{p_n} \end{bmatrix}$$ ← p امین نمونه در مجموعه آموزشی

$$T_p = \begin{bmatrix} t_{p_1} \\ t_{p_2} \\ \vdots \\ t_{p_m} \end{bmatrix}$$ ← خروجی مطلوب برای S_p

وقتی S_p اعمال می‌شود در خروجی لایه‌های مختلف این را داریم ←

$$Y_p^{(l)} = \begin{bmatrix} Y_{p_1}^{(l)} \\ y_{p_2}^{(l)} \\ \vdots \\ y_{p_{n_l}}^{(l)} \end{bmatrix}$$

خطا در خروجی ← E_p (وقتی ورودی S_p اعمال شده باشد)

$w_{ij}^{(l)}$ ← وزن اتصال از واحد i از لایه‌ی $l-1$ به واحد j در لایه‌ی l

عید سعید فطر (تعطیل)

تابع خطا:

$$E_p = \frac{1}{2} \sum_j (t_{pj} - y_{pj})^2$$

خطای تک تک نورون‌های لایه خروجی را با هم جمع کرده‌ایم و به توان ۲ رسانده‌ایم

۲۴ | اردیبهشت | جمعه
14 May ■ ۲ شوال

فعالیت یا ورودی خالص نورون j در لایه‌ی l به واسطه‌ی ورودی p:

$$I_{pj}^{(l)} = \sum_k w_{kj}^{(l)} \, y_{pk}^{(l-1)}$$

خروجی نورون j برای سمپل p در لایه l:

$$y_{pj}^{(l)} = f_j\left(I_{pj}^{(l)}\right)$$

می‌خواهیم مشتق خطا را نسبت به وزن w_{ij} در لایه‌ی l پیدا کنیم.

تغییرات خطا در خروجی با یک وزنی که از نورون i در لایه‌ی $l-1$ رفته به نورون j در لایه l، برای خواهیم بیابیم.

تعطیل به مناسبت عید سعید فطر

Scanned with CamScanner

ازقاعده زنجیره‌ای مشتق استفاده می‌کنیم:

$$\frac{\partial E}{\partial w_{ij}^{(l)}} = \frac{\partial E_p}{\partial I_{pj}^{(l)}} \cdot \frac{\partial I_{pj}^{(l)}}{\partial w_{ij}^{(l)}}$$

$$\frac{\partial I_{pj}^{(l)}}{\partial w_{ij}^{(l)}} = \frac{\partial}{\partial w_{ij}^{(l)}}\left[\sum_k w_{kj}^{(l)} y_{pk}^{(l-1)}\right] = y_{pi}^{(l-1)}$$

$$-\frac{\partial E_p}{\partial I_{pj}^{(l)}} = \delta_{pj}^{(l)}$$

فقط یکی از جمله‌ها که $k=i$ است مشتق آن غیر صفری شود

$$\Rightarrow -\frac{\partial E_p}{\partial w_{ij}^{(l)}} = -\nabla_w E_p = \delta_{pj}^{(l)} y_{pi}^{(l-1)}$$

این رابطه‌شان می‌دهد که تغییرات خطا در خروجی چگونه وابسته است به تغییرات وزن‌ها در شبکه.

رابطه‌ی update شدن وزن‌ها در BP:

$$w_{ij}^{new(l)} = w_{ij}^{old(l)} + \eta\, \delta_{pj}^{(l)} y_{pi}^{(l-1)}$$

η با سعی و خطا تعیین می‌شود

روش نزول در امتداد گرادیان

مشخص کردن $\delta_{pj}^{(l)}$:

$$\delta_{pj}^{(l)} = -\frac{\partial E_p}{\partial I_{pj}^{(l)}} \Rightarrow \delta_{pj}^{(l)} = -\frac{\partial E_p}{\partial y_{pj}^{(l)}} \cdot \frac{\partial y_{pj}^{(l)}}{\partial I_{pj}^{(l)}}$$

$$E_p = \frac{1}{2}\sum_j (t_{pj} - y_{pj})^2$$

$$\frac{\partial E_p}{\partial y_{pj}} = -(t_{pj} - y_{pj})$$

$$y_{pj}^{(l)} = f_j(I_{pj}^{(l)})$$

$$f_j'(I_{pj}^{(l)})$$

در این جا lها را نوشته‌ایم چون E_p را فقط برای لایه خروجی داریم

برای واحدهای خروجی داریم ←

$$\delta_{pj} = (t_{pj} - y_{pj}) \cdot f_j'(I_{pj})$$

Scanned with CamScanner

برای لایه‌های مخفی:

$$\delta_{pj}^{(l)} = -\frac{\partial E_p}{\partial I_{pj}^{(l)}} \Rightarrow \delta_{pj}^{(l)} = -\frac{\partial E_p}{\partial y_{pj}^{(l)}} \cdot \frac{\partial y_{pj}^{(l)}}{\partial I_{pj}^{(l)}}$$

(E_p را بیابیم)

$$\frac{\partial E_p}{\partial y_{pj}^{(l)}} = \sum_k \frac{\partial E_p}{\partial I_{pk}^{(l+1)}} \cdot \frac{\partial I_{pk}^{(l+1)}}{\partial y_{pj}^{(l)}}$$

$$f_j'(I_{pj}^{(l)})$$

$$-\delta_{pk}^{(l+1)} \qquad \frac{\partial}{\partial y_{pj}^{(l)}} \sum w_{ik}^{(l+1)} y_{pi}^{(l)} = w_{jk}^{(l+1)}$$

$$\Rightarrow \frac{\partial E_p}{\partial y_{pj}^{(l)}} = -\sum_k \delta_{pk}^{(l+1)} \cdot w_{jk}^{(l+1)}$$

$$\boxed{\delta_{pj}^{(l)} = f_j'(I_{pj}^{(l)}) \cdot \sum_k \delta_{pk}^{(l+1)} \cdot w_{jk}^{(l+1)}}$$

← برای لایه‌های مخفی

الان برای خطا در لایه l یک فرمول پیدا شده که می‌شه با استفاده از خطا در لایه $l+1$ آن را حساب کرد

در نتیجه با استفاده از این رابطه می‌توانیم خطا را از لایه بعدی به لایه قبلی منتشر کنیم. (قاعده BP)

همانطور که گفتیم تابع f باید مشتق پذیر باشد چون در رابطه‌مان f' داشتیم.

تابع سیگموئید باینری (Binary Sigmoid) مشتق دارد و می‌توان برحسب خودش نوشت:

$$f(z) = \frac{1}{1+e^{-kz}} \rightarrow f'(z) = k f(z)[1 - f(z)]$$

تابع Bipolar Sigmoid هم می‌تواند استفاده شود:

$$f(z) = \frac{2}{1+e^{-kz}} - 1 \rightarrow f'(z) = \frac{k}{2}[1 + f(z)][1 - f(z)]$$

Scanned with CamScanner

الگوریتم آموزش BP :

① ابتدا مقادیر تصادفی کوچکی به وزن‌ها اختصاص دهید

① تا زمانی که شرط توقف غلط است قدم‌های ۲ تا ۷ را انجام بده

② برای هر زوج آموزشی (S_p, T_p) قدم‌های ۳ تا ۶ را تکرار کن

③ خروجی همه‌ی واحدها را از اول شبکه تا آخر شبکه حساب کن:

$$y_{pj}^{(1)} = f_j\left[\sum_{i=0}^{n} w_{ij} S_{pi}\right]$$ ← برای لایه مخفی اول که ورودی‌اش S است

$$y_{pj}^{(l)} = f_j\left[\sum w_{ij}^{(l)} y_{pi}^{(l-1)}\right] \quad l>1$$ ← برای لایه‌های بعدی به‌جای S، y داریم

④ دلتا را برای واحدهای خروجی حساب کن:

$$\delta_{pj} = (t_{pj} - y_{pj})\, f_j'(I_{pj})$$

⑤ دلتا را برای همه‌ی واحدهای مخفی پیدا کن:

$$\delta_{pj}^{(l)} = \left(\sum_k \delta_{pk}^{(l+1)} w_{jk}^{(l+1)}\right) f_j'(I_{pj}^{(l)})$$

به این صورت محاسبه می‌شود:

δ را در لایه خروجی داریم، با استفاده از آن δ را برای یک لایه قبل حساب کن و مجددا با استفاده از δ بدست آمده برای یک لایه قبل حساب کن و همینطور رو به عقب.

⑥ همه‌ی وزن‌ها را update کن:

$$w_{ij}^{new(l)} = w_{ij}^{old(l)} + \eta\, \delta_{pj}^{(l)} y_{pi}^{(l-1)}$$

⑦ چک کردن شرط خاتمه

انقدر برای ورودی‌های مختلف این مراحل را تکرار می‌کنیم تا شرط خاتمه صحیح شود

Scanned with CamScanner

مجموعه آموزشی فقط یکبار به شبکه اعمال نمی‌شود بلکه باید چندین بار کل داده‌های آموزشی به شبکه اعمال شود تا به همگرایی برسیم. وقتی یک دور همه‌ی داده‌های آموزشی را اعمال کنیم به آن یک دوره اعمال ورودی یا epoch گفته می‌شود.

الگوریتم BP یک الگوریتم کند است چون باید مکرر ورودی‌ها را به شبکه اعمال کنیم و معمولا داده‌های آموزشی زیاد هستند، هم‌چنین محاسبات زیاد است و باید تکرار شود. در نتیجه آموزش شبکه‌های feed forward یک آموزش زمان‌بر و کند است و مقدار پارامترها زیاد است.

فرض کنید آموزش انجام شده، چگونه از شبکه آموزش دیده استفاده کنیم؟

① وزن‌ها را برابر با وزن‌های فاز آموزش قرار دهید

② ورودی را بدهید و از این رابطه خروجی نورون‌ها را در لایه اول حساب کنید:

$$y_i^{(1)} = f\left(\sum_{j=0}^{n} w_{ji}^{(1)} x_j\right)$$

③ از روی خروجی نورون‌های لایه اول، لایه‌های بعدی را حساب کنید:

$$y_i^{(e)} = f\left(\sum_{j} w_{ji}^{(e)} y_j^{(e-1)}\right)$$

④ خروجی لایه آخر بدست می‌آید

شبکه feedforward آموزشش سنگین است، استفاده از آن راحت است. وقتی وزن‌ها مشخص شد کافیه بیاد مسیر از ورودی تا خروجی پیمایش بشه تا خروجی مشخص شود.

Network operation:

تابع انرژی شبکه (تابع خطای شبکه) ← E_{rms}

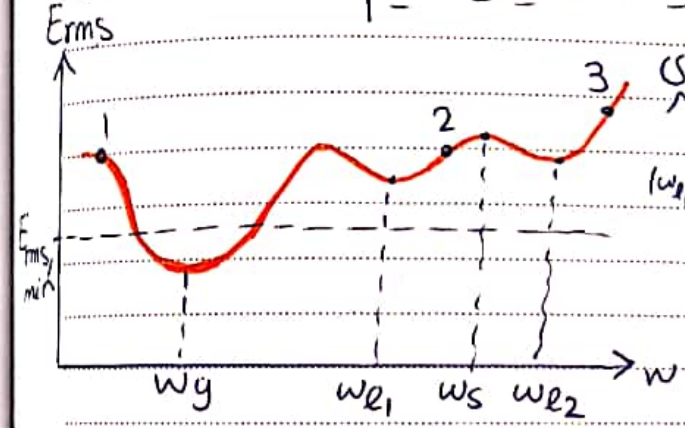
در این شبکه هدف ما این است وزن‌ها به گونه‌ای پیدا شوند که این خطا حداقل شود و این عمل را به این صورت انجام می‌دهیم که در جهت خلاف گرادیان روی سطح انرژی (خطا) می‌آییم پایین.

در این شبکه تابع خطا یک سطح بسیار پیچیده‌ای در فضای وزن‌ها دارد بنابراین ممکن است min های محلی

Scanned with CamScanner

۱۴۰۰

زیادی داشته باشد و احتمالش زیاد است که در این min های محلی گیر بیفتیم و وقتی که در این min محلی افتاد دیگر نمی تواند خارج شود و الگوریتم ما متوقف می شود در یک min محلی همگرا می شود. اینکه این اتفاق بیفتد خیلی به این بستگی دارد که نقطه شروع ما روی سطح خطا کجا باشد مثلا اگر از نقطه ای در نزدیکی min سراسری آغاز کنیم ممکنه به راحتی min سراسری را پیدا کنیم اما اگر از یک نقطه در نزدیکی یک min محلی شروع کنیم به احتمال زیاد در آن گیر می افتیم.



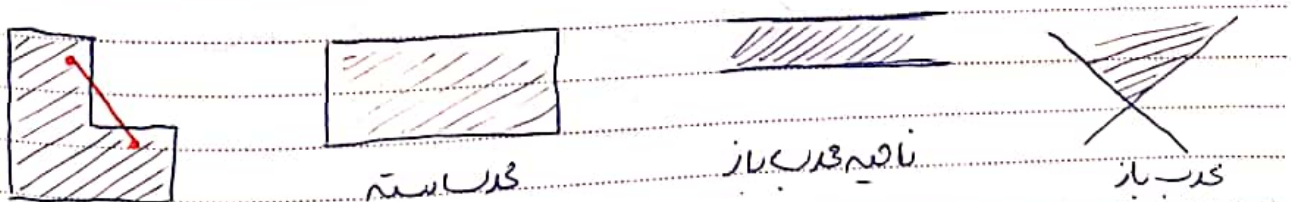
مثال: در شکل روبرو اگر از نقطه 1 شروع کنیم به min سراسری می رسیم. اگر از نقطه 2 شروع کنیم در min محلی گیر میفتیم (w_{e_1}) در مورد نقطه 3 هم در min محلی گیر می افتیم (w_s)

← بنابراین انتخاب مقادیر اولیه وزن ها مهم است

جلسه هشتم - ۱۴۰۰/۷/۲۵

عملکرد شبکه به عنوان دسته بند (HLP)

یک ناحیه را محدب (convex) می گوییم اگر هر دو نقطه از ناحیه را که با یک خط هم متصل می کنیم، تمام آن خط، درون ناحیه بیفتد



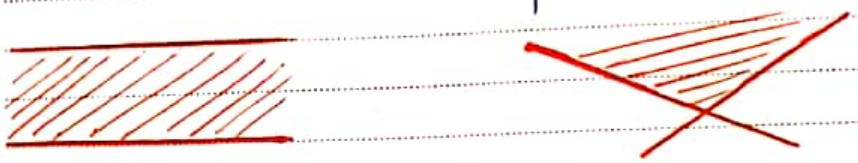
ناحیه های محدب { بسته / باز

وقتی یک نورون داشته باشیم میاد در فضای ورودی یک خطی کشیده و ناحیه را به ۲ بخش تقسیم می کند

Scanned with CamScanner

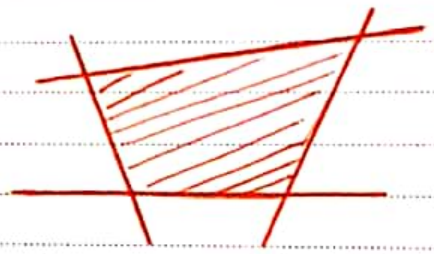
وقتی در لایه‌ی اول چندتا نورون داریم، هرکدام از نورون‌ها یک خط در فضا ترسیم می‌کند.
در لایه‌ی دوم، این خطوط رسم شده به گونه‌ای که مطلوب است ترکیب می‌شوند تا ناحیه‌ی مورد نظر ما را تشکیل دهند.

فرض کنید یک شبکه با معماری 1-2-n داشته باشیم:



معماری 1-3-n:

معماری 1-4-n:



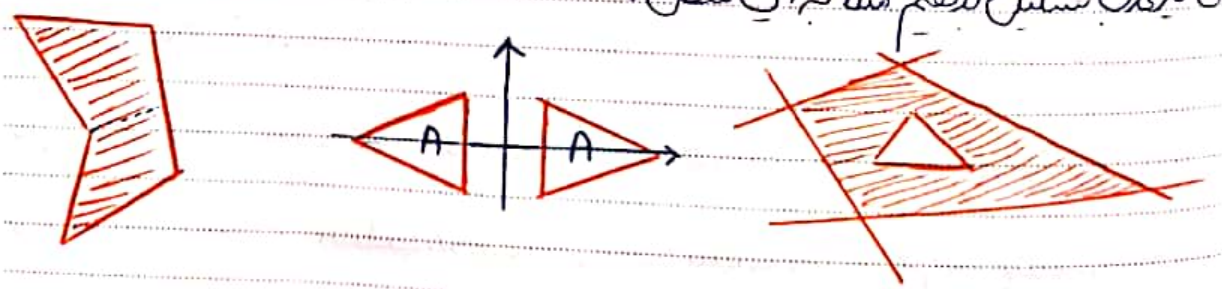
⇐ نتیجه: تعداد اضلاع ناحیه‌ای که تشکیل می‌شود برابر تعداد نورون‌های لایه‌ی اول است.

روز ملی جمعیت

اردیبهشت | ۳۱ | جمعه
21 May ■ ۹ شوال

اینکه خط رسم شده یا منحنی، تابع فعال‌سازی نورون هست که این را مشخص می‌کند. ربطی به تعداد ورودی ندارد.

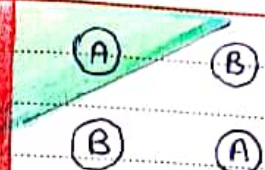
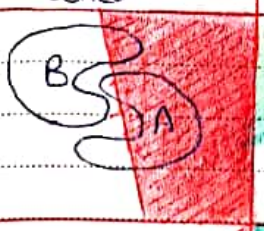
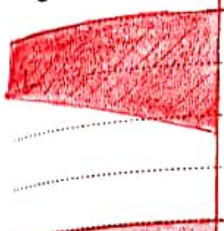
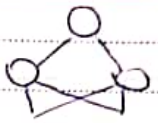
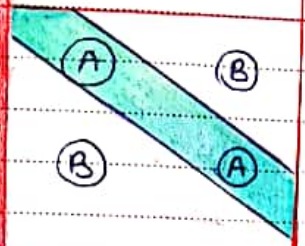
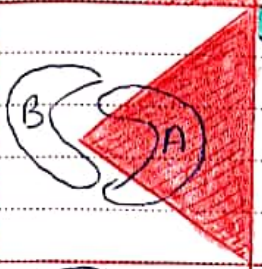
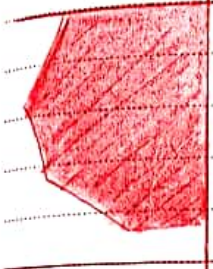
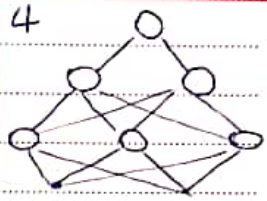
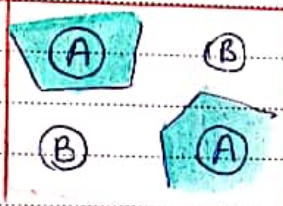
همانطور که دیدیم با معماری‌های ۳ لایه، نواحی به دست آمده محدب هستند. خیلی اوقات نیاز داریم ناحیه‌ی غیرمحدب تشکیل بدهیم. مثلا به این شکل:



راه‌حل: یک لایه اضافه کنیم. مثلا در همین شکل سمت راست، مثلث و ۴ ضلعی در لایه‌ی سوم ایجاد می‌شود.
در لایه‌ی چهارم با هم ترکیب می‌شود. یا مثلا سمت چپ: از دو شکل کلاب تشکیل شده، هرکدام در لایه‌ی ۳ تولید می‌شود در لایه‌ی ۴ ترکیب می‌شوند. پس با شبکه ۴ لایه می‌توان هر شکلی با هر پیچیدگی را ایجاد کرد.

روز اهدای عضو، اهدای زندگی

Scanned with CamScanner

تعداد لایه‌ها	نوع نواحی	مسئله XOR	کلاس‌های ناحیه‌ای Meshed	نواحی با اشکال کلی‌تر ۱۴۰۰
2	half plane bounded by Hyperplane			
3  ۲ تا خط در فضا مستوی نباشد	Convex Open or closed Regions			
4 	Arbitrary (Complexity Limited by No. of Nods)			

Memorization and Generalization

Memorization : وقتی شبکه را آموزش دادیم، مشخص می‌کند شبکه چقدر این‌ها را خوب یاد گرفته

یعنی میزان صحت پاسخ‌ها به داده‌هایی که برای آموزش به کار رفته

یعنی این شبکه با آموزشی که دیده قدرت حفظ کردنش چقدر بوده

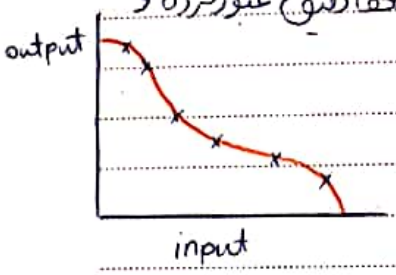
Generalization : عبارت است از قدرت شبکه برای پاسخ صحیح دادن به ورودی‌های جدیدی که با آن‌ها آموزش ندیده است.

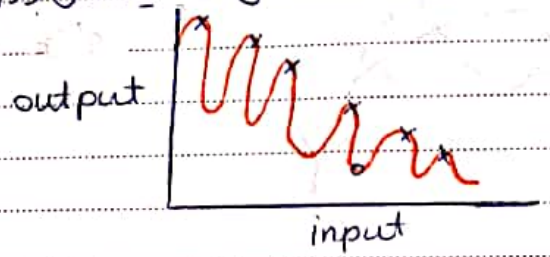
← برای ما مطلوب است که Memorization و Generalization هر دو بالا باشند.

اما مشاهده می‌شود که Memorization و Generalization عنصر کار می‌کنند. یعنی در آموزش شبکه اگر سعی کنیم Memorization را بالا ببریم Generalization پایین می‌آید و برعکس. معمولا در طراحی شبکه باید یک تعادلی میان این‌ها برسیم.

Scanned with CamScanner

یک سری راه‌حل‌هایی پیدا شده که هم Memorization و هم Generalization بالا باشد. که این راه‌حل‌ها با تغییر اون شبکه همراه است مثلا تغییر اون تابعی که minimize می‌کنیم برای آموزش شبکه، ایجاد کنیم آموزش این شبکه خیلی شبیه برازش منحنی است. داده‌های ما نقاط هستند که در برازش منحنی داریم و تابعی که شبکه باید یاد بگیرد همان منحنی است که برازش می‌کنیم. اگر بخواهیم شبیه آموزش شبکه را با مسئله‌ی برازش منحنی در نظر بگیریم مسئله‌ی Memorization می‌شود اینکه منحنی برازش شده چقدر از داده‌ها دقیق عبور کرده و





مسئله Generalization می‌شود اینکه شبکه چقدر از داده‌ها دقیق عبور کرده و چقدر خوب می‌تواند به آموزش‌هایی که دیده interpolation انجام دهد.

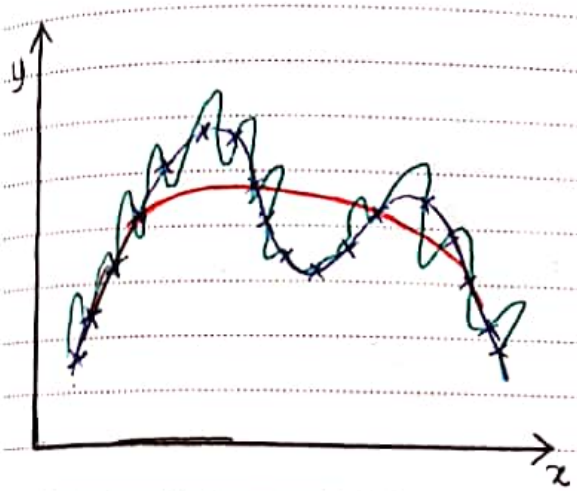
با توجه به دو منحنی رسم شده در بالا، در حالت سمت چپ Generalization مناسبی داریم چون هر نقطه‌ای به عنوان تست به مدل بدهیم یک جواب معقول در محدوده‌ی داده‌هایی که داریم به ما می‌دهد. اما در شکل سمت راست Generalization پایین است. ممکنه برای یک داده تست که می‌دهیم جواب دارای خطای بالایی باشد. در شکل سمت راست Memorization خوب است یعنی برای داده‌های آموزشی خوب جواب می‌دهد. این دقیقاً مسئله‌ای است که در curve fitting هم داریم. در سمت چپ interpolation خوب است در سمت راست نه.

اگر شبکه را با تعداد زیادی نمونه آموزشی و تعداد epoch های زیاد آموزش بدهیم، شبکه به سمت حفظ کردن داده‌های آموزشی (Memorization) می‌رود و Generalization کم می‌شود.

در این حالت معمولا ممکنه لازم بشه مجبور بشویم نورون‌ها را زیاد کنیم. وقتی نورون‌ها زیادی می‌شود در واقع تعداد پارامترها زیاد می‌شود. مشابه آن در curve_fitting هم داریم یعنی اگر تعداد پارامترهای منحنی که می‌خواهیم فیت کنیم زیاد باشد overFitting پیش می‌آید.

Scanned with CamScanner

در این حالت شبکه به داده های نویز هم حساس است. چون کوچکترین مقادیر نویز را هم یاد خواهد گرفت و این حالت در مسئله (مدل سعی دارد منحنی ای بیابد که از همه نقاط عبور می کند) در این حالت در برازش منحنی interpolation و extrapolation ضعیف است.



در شکل زیر اگر دو نورون در لایه مخفی بگذاریم، منحنی قرمز به دست می آید. اگر ۵ تا نورون بگذاریم منحنی آبی به دست می آید

اگر ۱۴ تا بگذاریم، منحنی سبز به دست می آید. از نقاط عبور می کند اما چون تعداد پارامترها زیاد است، منحنی حاصل یک منحنی درجه بالا است در نتیجه نوسان می شود

در برازش منحنی با منحنی های ساده شروع می کردیم. این جا هم دوست داریم تا جایی که برای شبکه یاد گرفته می شود ساده ترین حالت ممکن باشد. چون در این صورت محاسبات و تعداد نورون ها کمتر است.

می خواهیم بی جهت تعداد نورون ها و پارامترها را زیاد نکنیم. ترجیحمان این است که تعداد نورون ها کم باشد

می خواهیم بررسی کنیم انتظاری که از شبکه feed forward داریم که پاسخ خوب بدهد برای نمونه های جدید تا چه حد معقول و منطقی است؟

فرض کنید مجموعه همه داده های ورودی و خروجی یک مسئله در مجموعه U باشد (مستطیل) یک سری از نقاط این مجموعه با تابع نورون R ها هنگ هستند و ما می خواهیم شبکه ای را آموزش بدهیم که این را تشخیص بدهد. شبکه را با نمونه های آموزشی که انتخاب می کنیم آموزش می دهیم. فرض کنیم نقاط داخل ناحیه T، مجموعه آموزشی باشد. نقاط مجموعه X هم به عنوان تست به کار می بریم. آموزش شبکه باید آن چه را با مجموعه T یاد گرفته تعمیم بدهد تا برای نقاط جدید جواب بدهد.

Scanned with CamScanner

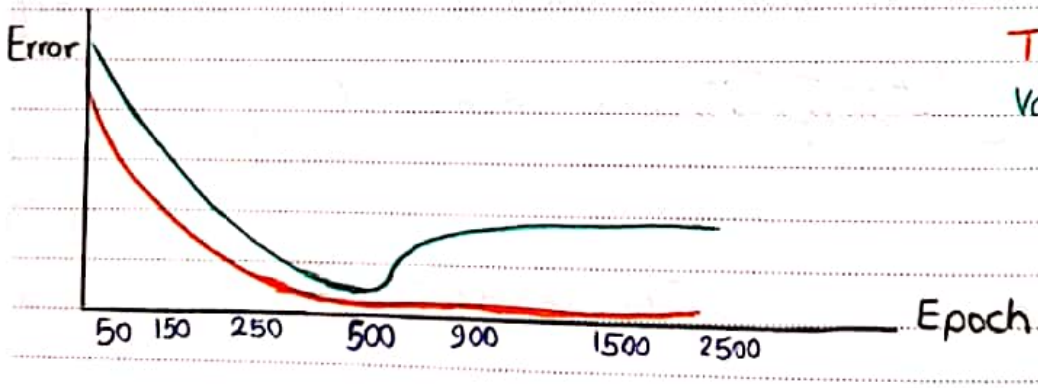
به شکل‌های مختلفی می‌تواند آموزشش از روی مجموعه T را تعمیم بدهد. مثلا شکل، نقاط خط‌چین نشان می‌دهد آموزش داده شده به شبکه این ناحیه را یاد گرفته و خارج از آن را نه. در خیلی از حالات تعمیم یادگرفته برای T اصلا شامل X نمی‌شود.

آن‌جه که شبکه دید نقاط مجموعه T بوده، چطور انتظار داریم برای عکس نقاط جواب صحیح بدهد؟!

باید انتظارات خود را کم کنیم و بگیم اینکه چقدر آموزش شبکه هم است

نکته دیگه: نقاطی که در مجموعه آموزشی می‌گذاریم خیلی مهم است مثلا در این جا اگر نقاط آموزشی که انتخاب می‌کنیم در بیرون سه R پخش بودند، نتیجه بهتری حاصل می‌شد. چون ناحیه X را هم پوشش می‌داد.

در یک سیستم واقعی منظور این است که برای همه حالاتی که سیستم می‌تواند داشته باشد و روی اعمال کنیم و به خروجی حاصل، به سیستم بدهیم. شامل همه حالات است حتی وقتی سیستم خراب است یا مشکل دارد...



در این نمودار برای یک مسئله، خطای مجموعه آموزشی و Validation به ازای تعداد epoch های مختلف رسم شده است. تا حدود epoch=۵۰۰ آموزش به خوبی یادگرفته که هم برای Validation Set و هم برای Training Set خطا را حداقل کند اما از ۵۰۰ به بعد خطای Validation Set در حال افزایش است یعنی شبکه برای نمونه‌های جدید تصمیم‌گیری غلطی کند. یعنی تا ۵۰۰=epoch، Generalization و Memorization خوب هستند اما از ۵۰۰ به بعد Generalization شروع به خراب شدن می‌کند، Memorization همچنان خوب است. پس نتیجه می‌گیریم که نباید آموزش را بیشتر از ۵۰۰ تا epoch ادامه بدهیم.

Scanned with CamScanner

نکته: همانطور که گفتیم با لایه در شبکه ff می‌توانیم ناحیه‌بندی فضا را به هر شکلی که می‌خواهیم انجام بدهیم این حرف به این معنی است که می‌توانیم هر مسئله‌ی دسته‌بندی را با شبکه ۲ لایه حل بکنیم. اما ممکن آموزش شبکه خیلی سخت باشد و با اضافه کردن لایه، آسان‌تر حل شود و آموزش صورت بگیرد.

جلسه پنجم - ۱۴۰۰/۷/۲۷

مسئله دوم: تخمین تابع (Regression or Function Approximation)

در بسیاری از کاربردها با این وضعیت روبه‌رو هستیم که مقدار زیادی داده از یک پدیده جمع‌آوری کرده‌ایم و می‌خواهیم مدلی برای این پدیده بسازیم و یک تابع به این داده‌ها فیت کنیم

فرض کنید مجموعه‌ای از مشاهدات را از پدیده‌ای داریم که شامل بردار X و y است. بردار X خروجی است که به آن پدیده وارد می‌شود و y پاسخی است که آن پدیده می‌دهد. هدف این است که برای این پدیده مدلی را توسعه بدهیم که به کمک آن بتوانیم اگر X جدیدی داریم y ای که پاسخ آن است را پیش‌بینی کنیم.

این مسئله همیشه بوده و رویکردی که از آن استفاده می‌شده، برازش منحنی بوده است.

شبکه‌های عصبی این عمل را به خوبی انجام می‌دهند

تخمین تابع با شبکه‌های FF:

فرض کنید نگاشت f را داریم که: $f: A \subset R^n \rightarrow R^m$ که با مجموعه‌ای از سمپل‌ها

(S_i, T_i) $i = 1, 2, \dots, L$ نشان داده می‌شود.

این رابطه را هم داریم: $T_k = f(S_k)$ $k = 1, 2, \dots, L$ یعنی مقدار مطلوبی که ما داریم، با تابع f مرتبط می‌شود با مقدار تحریک. این تفاوتی است که با مسئله‌ی دسته‌بندی دارد.

Scanned with CamScanner

فرض کنید X_k یکی از نمونه حهای آموزشی است. مربع خطا برای آن اینگونه بدست می آید:

$$f_k(X_k, W) = | f(X_k) - y_k(X_k, W) |^2$$

یا خروجی پیش بینی شده توسط شبکه در صورت اعمال ورودی X_k است

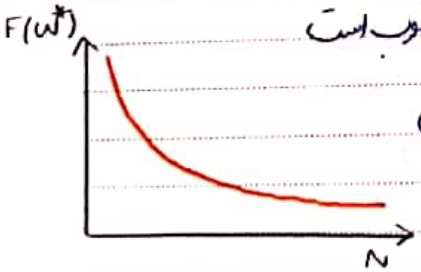
مقدار مطلوب یا T_k برای $f(X_k)$ ← فرقی که با مسائل قبلی دارد این است که به جای T_k، $f(X_k)$ میگذاریم

فرض کنید تعداد زیادی از این نمونه ها را داریم و آن ها را اعمال می کنیم. خطای کل:

$$F(W) = \lim_{N\to\infty} \frac{1}{N} \sum_{k=1}^{N} F_k(X_k, W) \geq 0$$

این میانگین یا مثبت است یا صفر است. مطلوب با اینه که خطا به عدد خیلی کوچکی برسد.

لازم است شبکه را ارزیابی کنیم تا بفهمیم تخمینی که این تابع به ما می دهد چقدر خوب است



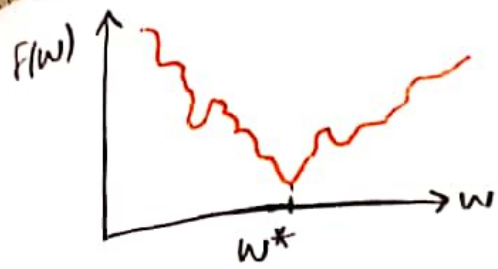
برای اشیای فرض کنید یک نمونه به شبکه می دهیم، اگر درست جواب بدهد دقت ۱۰۰٪ و اگر غلط باشد ۰٪ است. برای ۲ نمونه، اگر هر ۲ درست باشد ۱۰۰٪، هردو غلط ۰٪، یکی غلط یکی درست ۵۰٪ و ...

نتیجه اینکه با داشتن تعداد نمونه های مختلف، متفاوت می شود. من توان تعداد نمونه های مختلف داده درست، باید یک اساسی باشد.

با توجه به نمودار، هرچه تعداد نمونه ها را افزایش می دهیم خطای کل کاهش پیدا می کند.

کاهش پیدا می کند تا اینکه در جایی ثابت می شود، در این جا نشان می دهد که تعداد نمونه های تست کافی است که در این حد باشد. بنابراین برای اینکه بفهم مجموعه تستم چقدر خوب ارزیابی می کند و آیا کافی است یا نه، همیام یه مجموعه اختیاری کنم و بعد به مرور اندازه اش را افزایش می دهم. در هر بار افزایش سوال می کنم آیا مقدار خطا کمتر شد؟ اگر کمتر شد یعنی مقدار قبلی کافی نبوده و باید تعداد داده های تست را افزایش بدهیم. هرگاه دیگر تغییر آنچنانی در خطای ایجاد نشد می گوییم مقدار داده های تست کافی است.

Scanned with CamScanner



۱۴۰۰
در این مسئله هم مثل دسته بندی می توانیم برای شبکه در حین آموزش یک سطح در نظر بگیریم، سطح خطا یا loss که در فضای وزن ها قرار دارد و دنبال این هستیم که با حرکت روی این سطح به min خطا برسیم

قضیه: برای هر $\varepsilon > 0$ و هر تابع f که $[0,1]^n \to R^m$ باشد (یعنی به صورت مربع روی مکعب قابل انتگرال گیری است) آن گاه یک شبکه‌ی سه لایه‌ی feedforward وجود دارد که می تواند f را با متوسط خطای (mse accuracy) ε تخمین بزند.

نکته: ε هر مقدار دلخواهی می تواند باشد یعنی تا حد دلخواهی می توان دقت را به ۱۰۰ درصد نزدیک کرد

← اینکه بتوان با شبکه ۳ لایه پیاده سازی کرد می تواند سخت باشد (آموزش سخت است یعنی همگرا شدنش و رسیدن آن به خطای مطلوب، به راحتی صورت نمی گیرد)

برای اینکه راحت تر باشیم می توانیم شبکه ۴ لایه یا ۵ لایه استفاده کنیم

Scanned with CamScanner

عوامل تاثیرگذار روی عملکرد شبکه FF

① تعیین وزن های اولیه

اگر به حوزی انتخاب شود هم سریع همگرا می شویم و هم به min سراسری می رسیم. اما متاسفانه قاعده ای به دست نیامده، فقط یک سری راهنما داریم. مثلا اینکه وزن های اولیه نباید خیلی بزرگ باشند. اگر خیلی بزرگ باشند، نورون ها، به خصوص نورون هایی که در انتهای شبکه و نزدیک به خروجی هستند، عملشان در نواحی که تابع فعال سازی تقریباً ثابت است قرار می گیرد در نتیجه مشتق بسیار کوچک است و یادگیری کم است اصلاح وزن ها توسط ضریب δ صورت می گیرد که در روابط دیدیم δ با f' مربوط است. f' به صورت یک عامل ضرب شونده در یک تابع دیگری اثرگذار روی δ است.

$$w_{ij}^{new(l)} = w_{ij}^{old(l)} + \eta \delta_{pj}^{(l)} y_{pi}^{(l-1)}$$

توصیه دیگر اینکه وزن های اولیه خیلی کوچک هم نباشند چون در این صورت net Input به نورون ها کوچک می شود و خروجی ها به صفر نزدیک می شود و باز هم موجب کاهش سرعت همگرایی می شود.

$$\delta_{pj} = (t_{pj} - y_{pj}) f_j'(I_{pj}) \qquad \delta_{pj}^{(l)} = f_j'(I_{pj}^{(l)}) \cdot \sum_k \delta_{pk}^{(l+1)} \cdot w_{jk}^{(l+1)}$$

معمولا وزن ها را در فاصله $[-0.5, 0.5]$ یا $[1, -1]$ به صورت اندوم انتخاب می کنند.

اگر هنگام آموزش شبکه خطای ما به جایی رسید که دیگر کاهش پیدا نکرد (که به آن یک plateau گفته می شود) توصیه این است که آموزش را ادامه ندهید و بهتر است آموزش را از نو شروع کنید با مقادیر اولیه جدید برای وزن ها چون تغییرات نامطلوبی در وزن ها پیش می آید

② تابع فعال سازی

بهتر هست توابع Antisymmetric (فرد) داشته باشیم. یعنی خروجی هم مقادیر مثبت و هم مقادیر منفی داشته باشد و نسبت به محور قائم متقارن باشد. وقتی اینطور باشد یادگیری سریعتر صورت می گیرد. مثل این است که در

Scanned with CamScanner

۱۴۰۰

یک حسن جهم داریم تشویق می کنیم و هم تنبیه. وقتی مقدار منفی نداشته باشد (nonsymmetric) مثل

این است که فقط داریم تشویق می کنیم و تنبیه نمی کنیم.

← شیب تابع f برای تابع Sigmoid ← عاملی است که روی میزان اصلاح وزن ها تاثیرگذار است

$$f(z) = \frac{1}{1+e^{-\alpha z}} \qquad f'(z) = \alpha f(z)\left[1 - f(z)\right]$$

α شیب تابع فعال سازی است:

α در f تاثیرگذار است پس α در δ تاثیرگذار است، بنابراین در میزان اصلاح تاثیرگذار است.

α نباید خیلی بزرگ یا خیلی کوچک باشد α در واقع سرعت بالا رفتن منحنی Sigmoid را کنترل می کند

اینکه نزدیکتر می شود به تابع پله یا دورتر (تدریجی تر) می شود.

تصحیح وزن ها با عبارت $\eta \delta_{pj}^{(l)} y_{pi}^{(l-1)}$ صورت می گیرد که δ خودش یک عامل ضرب شونده ی f دارد

f هم عامل ضرب شونده ی α دارد. پس در نهایت α و η که هر دو پارامترهای کنترلی اند هم ضرب

می شوند. پیشنهاد شد که بیایم $\alpha = 1$ بگیریم و بعد کنترل را با η اعمال کنیم.

③ ارائه ی داده ها (Data Representation)

خیلی اتفاقات به خصوص در مسائل پیچیده این سوال مطرح است که ورودی ها را چه چیزی در نظر بگیریم.

یعنی چه داده هایی ببینند بدهیم؟ نکته دیگر این است که چگونه این داده ها را به شبکه بدهیم؟

این انتخاب تا حدی هم است که می تواند تعیین کند مسئله قابل حل باشد یا خیر.

مثلا یک انتخاب این است که داده ها را پیوسته بدهیم یا گسسته سازی انجام بشود (مثال دما و هوا)

وقتی توابع فعال سازی را دوقطبی در نظر می گیریم داده ها هم باید دوقطبی (Bipolar) باشند داده ها

باید 1- و 1 باشند و تابع فعال سازی هم بین 1- و 1 تغییر می کند

Scanned with CamScanner

۴) مجموعه‌های Training و Validation

دو حالت را در نظر بگیرید: ① مسئله ما به گونه‌ای است که به‌دنبال برای آن داده داریم و هیچ مشکلی در تهیه داده زیاد نداریم. در این حالت ۳ مجموعه Train، Validation و Test درست می‌کنیم و سعی می‌کنیم انواع داده‌ها را در هر ۳ مجموعه قرار بدهیم.

② نمی‌توانیم نمونه‌های زیادی داشته باشیم. در مسائل عملی معمولا این حالت پیش می‌آید و ما هم از نظر تعداد داده دهم تنوع داده، محدودیت داریم. مثلا نمی‌توانیم یک سیستم را خراب کنیم برای اینکه داده‌های مربوط به خراب شدن را داشته باشیم و در دنیای واقعی یک سری از داده‌ها را اصلا نمی‌توان تهیه کرد.

در این حالت تمام داده‌ها را برای آموزش به‌کار می‌بریم. در حین آموزش به یک جایی می‌رسیم که کاهش خطا جزئی می‌شود، در این جا آموزش را قطع می‌کنیم. یعنی به‌جای استفاده از Validation set، از افت شدن مقدار تابع خطای مجموعه آموزش استفاده می‌کنیم

پیشنهاد دیگر این است که یک نمونه را کنار بگذاریم و با n-1 نمونه‌ی دیگر شبکه را آموزش بدهیم و با آن 1 نمونه تست کنیم. بعد دوباره این کار را برای همه نمونه‌ها انجام بده (n بار). هر بار خطا را حساب کن بعد میانگین مربع این خطا را حساب کن و به عنوان خطای generalization شبکه اعلام کن. به این روش leave one out گفته می‌شود

سومین پیشنهاد: از روی داده‌هایی که داریم یک سری داده به‌صورت دستی و مصنوعی ایجاد کنیم. مثلا به شکل‌های مختلف به داده‌هایی که داری نویز اضافه کن تا داده جدید به دست آید. یا تغییراتی در داده ایجاد کن مثلا در مورد داده تصویری می‌توان با چرخاندن آن یک تصویر جدید ایجاد کرد. یا مثلا scale کردن (کوچک و بزرگ کردن شیء در تصویر) یا با تغییر شدت روشنایی تصویر. به این روش‌ها data augmentation گفته می‌شود.

Scanned with CamScanner

⑤ تعداد نمونه‌های آموزشی

Baum و Haussler نشان دادند که در یک شبکه FF تعداد وزن‌ها (یا اتصالاتی) که در آن است اگر N_W باشد و خطای قابل قبول برای تست E باشد آن گاه باید شبکه را با تعداد $L = \frac{N_W}{E}$ نمونه آن قدر آموزش بدهید که خطا به کمتر از $\frac{E}{2}$ برسد. در این صورت می‌توانیم مطمئن باشیم که خطا برای داده‌های تست از E بیشتر نمی‌شود.

مثال:

$N_W = 80$ ، $E = 10\%$ $\Rightarrow$ $L = 80/0.1 = 800$

پس ۸۰۰ تا داده باید انتخاب شود و آن قدر آموزش داده شود که خطا کمتر از ۵ درصد شود

⑥ چقدر آموزش بدهیم (تعداد epoch)

همانطور که گفتیم در طراحی مان باید بین Memorization و Generalization تعادل برقرار کنیم. (قدرت حفظ کردن و قدرت تعمیم دادن شبکه).

معنی اش این است که نباید زیاد آموزش بدهیم چون به سمت حفظ کردن می رود.

نباید دنبال این باشیم که همان خطا را min کنیم. ترجیح می دهیم خطای جزئی را بپذیریم در ازای اینکه Generalization بهتری داشته باشیم. ۳ معیار برای توقف ذکر شده:

1. اندازه گرادیان را در هر مرحله به دست آوردیم. در min واقعی مشتق صفر است. وقتی گرادیان نزدیک به صفر شود ما به پاسخ نزدیکیم. یک مقدار آستانه در نظر بگیرید، اگر گرادیان از آن کوچکتر شد متوقف شوید

2. نرخ تغییرات متوسط مربع خطا برای epoch اگر از یک حدی کمتر شد متوقف شویم. پس در این جا داریم از معیار خطا برای توقف آموزش استفاده می‌کنیم

Scanned with CamScanner

۳. Early Stopping ← وقتی خطا برای مجموعه آموزشی در حال ثابت شدن است می‌توانید آموزش را متوقف کنید

جلسه دهم - ۱۴۰۰/۸/۴

⑦ نرخ یادگیری (Learning Rate)

تنظیم نرخ یادگیری وابسته به مسئله‌ای است که می‌خواهیم حل کنیم. یعنی این ضریب η (eta) که روی سرعت همگرایی تاثیرگذار است و الگوریتم نسبت به مقدار آن حساس است، به مسئله وابسته است.

غلاکاری می‌توانیم بکنیم جز اینکه از طریق تجربی آن را پیدا کنیم. نکته مهمی که در مورد آن می‌دانیم این است که مقادیر کوچک η تضمین می‌کنند که پایین رفتن در امتداد گرادیان صورت بگیرد. در کتاب‌ها اغلب شده مقدار مناسب برای η در بازه ۰/۰۰۱ تا ۱۰ گفته شده است.

⑧ معماری شبکه (Network Architecture)

۱۴ | خرداد | جمعه
4 Jun ■ ۲۳ شوال

قدم اول و مهم و اصلی است.

تعداد نورون‌های لایه ورودی باید معلوم باشد (سایز ورودی را داریم در نتیجه تعداد نورون‌های لایه اول را داریم)

چون میدانیم چه بسا سخن از مسئله انتظار داریم تعداد نورون‌های لایه خروجی را هم می‌دانیم که البته در مورد لایه خروجی قبلا گفتیم که m یا $\log m$ تا خروجی می‌توانیم استفاده کنیم

در مورد لایه‌های مخفی: باید از مقادیر نه چندان بزرگ برای لایه‌های مخفی شروع کنیم و جلو برویم. یعنی مثلا یک لایه مخفی با یک تعداد نورون که فکر می‌کنید برای این مسئله کفایت می‌کند قرار دهید، بعد بررسی می‌کنید. تغییری دهید و این کار تکرار می‌شود

در واقع طراحی شبکه‌های پیش رو، یک بار نیست. یک فرآیندی است که به دفعات با مشخصات مختلف برای لایه‌های مخفی تکرار می‌شود که بتوانید به یک معماری بهتر دست پیدا کنید.

Scanned with CamScanner

مثلا میگه شبکه‌ای طراحی کنید که ۸۸ درصد دقت دارد و شبکه‌ی دیگری را بررسی نکنید. سوالی که این جا ۱۴۰۰
جواب داده نشده این است که آیا معماری بهتر از این وجود نداشت؟ این مورد باید بررسی شود.
هنگام طراحی باید معماری های مختلف را امتحان کنید و از بین آن ها بهترین شان را انتخاب کنید.

شاخص های مختلف شبکه های FF

مشکلات این شبکه: ① گیر بودن آموزش (گیر بودن هنگام این در فاز آموزش)
② هنگ کردن به min محلی

برای حل این دو مشکل کارهای زیادی صورت گرفته است

① یک کار این بود که قانون به هنگام درآوردن وزن ها را تغییر دهند (یعنی قانون یادگیری را عوض کنند)

② راه دیگر: با تغییر تابع فعال شدن نورون ها، بهبود ایجاد کنیم

③ راه دیگر: روش های ترکیبی هوش محاسباتی

دسته اول: تغییر قانون اصلاح وزن ها (BP)

۱. روش های درجه ۱

ویژگی متدهای درجه ۱ این است که همه براساس گرادیان هستند. یعنی براساس شیب حرکت روی سطح خطا هستند. چندتا از روش هایی که پیشنهاد شده در این دسته:

۱) یک جمله به قانون یادگیری اضافه می شود که به آن جمله momentum گفته می شود در اصلاح وزن ها momentum ایادی کند

۲) روش دیگر: به هنگام درآوردن دسته ای

۳) روش دیگر: روش هایی هستند که نرخ یادگیری را تطبیقی می کنند. یعنی نرخ یادگیری η که اندازه قدم حرکت به سمت پاسخ را برای ما مشخص می کند با شرایطی که داریم تطبیقی می کند.

قیام خونین ۱۵ خرداد (۱۳۴۲ هـ ش) (تعطیل) - روز جهانی محیط زیست

Scanned with CamScanner

۴) روش آخر، روش تندترین شیب (steepest descent)

A.1 اضافه کردن جمله‌ی Momentum

خیلی اتفاقات ممکنه در بین داده‌ها، داده‌های پرت داشته باشیم که مقدارشان خیلی با داده‌های دیگر تفاوت دارند. وجود این داده‌ها به آموزش صدمه می‌زند و توصیه می‌شود قبل از آموزش این داده‌ها حذف شوند.
جمله‌ی Momentum سعی دارد اگر به چنین داده پرت در داده‌های آموزشی برخورد کرد، صدمه آن را کم کند

$$W_{ij}^{(l)}(t+1) = W_{ij}^{(l)}(t) + \alpha \delta_{pj}^{(l)} y_{pj}^{(l-1)} + \mu \left[W_{ij}^{(l)}(t) - W_{ij}^{(l)}(t-1) \right] \qquad 0 < \mu < 1$$

تغییر وزن این دفعه با ضریبی از تغییر وزن دفعه قبل جمع می‌شود. یعنی اگر مثلا در استاد یک بردار در جهت گرادیان در حال حرکت به سمت پایین هستیم و یک دفعه یک داده پرت یک جهت اغراق به این بردار می‌دهد. حالا اگر این بردار منحرف شده با ضریبی از تغییر وزن دفعه‌ی قبلی جمع شود، حاصلش می‌شود برداری که کمتر اغراق دارد و بیشتر متمایل هست به سمت جهت گرادیان تا جهتی که این داده پیشنهاد پرت داده است.

بنابراین حرکت در این حالت در جهت گرادیان نقطه‌ی x خواهد بود بلکه در جهت گرادیان قبلی به اضافه جهت این نقطه خواهد بود.

ضریب μ را معمولا نزدیک ۱ می‌گیرند و α هم مقدار کوچکی است

وقتی از Momentum استفاده می‌کنیم این احتمال وجود دارد که اصلاح وزن‌ها جاهایی که داده پرت نداریم بزرگ می‌شود (در نتیجه با قدم‌های بزرگ‌تری حرکت می‌کنیم و سرعت همگرایی بالا می‌رود). هم‌چنین وقتی قدم‌ها بزرگ است این احتمال وجود دارد که از برخی min های محلی عبور کنیم. پس اثرات مثبتی از جهت‌های دیگر هم دارد

Scanned with CamScanner

مثال از انتخاب مقادیر η و μ

بهتر است η را کوچک و μ را نزدیک ۱ انتخاب کنیم. با کوچک شدن η بهبود ایجاد می‌شود ولی از یک جایی وقتی کوچکتر شد شروع به بدتر شدن می‌کند. چون یادگیری ضعیف می‌شود

A.2 به روز آوردن دسته‌ای وزن‌ها

۱) pattern mode : هر تک نمونه‌ای که اعمال می‌شود، همه‌ی وزن‌ها را اصلاح کنیم

۲) یک راه دیگر این است که نمونه اعمال شود، خطا را حساب کنیم اما وزن‌ها را آپدیت نکنیم. این کار را به تعداد نمونه‌های درون batch تکرار می‌کنیم. بعد از آن از خطاها میانگین می‌گیریم و اصلاح وزن‌ها بر حسب میانگین خطا صورت گیرد

یعنی برای هر sample ای که داریم خطا : $e_j(p) = t_j(p) - y_j(p)$ را ذخیره می‌کنیم

و بعد میانگین می‌گیریم :

$$E_{avg} = \frac{1}{2L} \sum_{p=1}^{L} \sum_{j \in C} e_j^2(p)$$

هدف این است که سرعت را بالا ببریم.

با انجام این کار، تعداد زیادی به هنگام در آوردن وزن انجام نمی‌شود در نتیجه محاسبات کاهش پیدا می‌کند

یک batch می‌تواند شامل همه‌ی نمونه‌ها باشد یعنی مساوی یک epoch باشد. یعنی یک دور تمام نمونه‌ها را اعمال کنیم، خطا را بیابیم و آخر سر وزن‌ها را آپدیت کنیم

A.3 تطبیق کردن نرخ یادگیری

دو رویکرد وجود دارد: ① با یک η بزرگ شروع کنیم و آن را تدریجا کاهش بدهیم به طوری که وقتی به ناحیه‌ی همگرایی می‌رسیم η کوچک شده باشد.

در ابتدا اگر قدم‌ها بزرگ باشد خوب است چون معمولا از کجا شروع کنیم سراسری دور هستیم، با قدم‌های بزرگ به سمت نقطه min می‌رویم و ممکن است از روی min های محلی هم بپریم که سرعت همگرایی بالا می‌رود. اما وقتی به ناحیه پاسخ نزدیک می‌شویم η بزرگ نامناسب است به همین دلیل می‌گوییم η را کاهش بدهیم به صورت تدریجی.

Scanned with CamScanner

② Delta_bar_Delta : برای هر وزن یک نرخ یادگیری مختص خودش دارد و این نرخ یادگیری در حین آموزش یادگیرنده می شود. بنابراین شبکه در حین آموزش این مورد را هم یاد می گیرد که نرخ یادگیری برای هر وزن چقدر باشد.

A.4 تندترین شیب (steepest Descent)

روش ترکیبی است: در روش delta_bar_delta نرخ یادگیری برای هر وزن مختص خودش تعیین می شود در BP استاندارد گفتیم η یک عدد ثابت است که برای تمام وزن ها در کل آموزش ثابت است. در steepest descent کاری می کنیم بین این دو راه است. نرخ یادگیری را ع می نامیم و برای تمام وزن ها مثل BP استاندارد، یکسان است اما در حین آموزش مقدارش تعیین می شود

روش آموزش: یک مقدار اولیه برای ع نسبت می دهید

در هر مرحله نرخ یادگیری را ضرب در 2 کن، وزن ها را محاسبه کن و خطا را با این وزن های جدید محاسبه کن. اگر خطا نسبت به دفعه قبل کاهش پیدا کرده این تغییر وزن ها خوب است

اگر کاهش پیدا نکرده این اصلاح وزن ها و تغییر نرخ یادگیری خوب نیست، مقادیر وزن ها را به حالت قبلی برگردان و نرخ یادگیری را نصف کن. مجددا MSE را محاسبه کن اگر MSE کاهش پیدا نکند، دوباره ع را نصف کن. به صورت مکرر این کار را انجام می شود. اینقدر ع نصف می شود تا به حالتی برسیم که MSE کاهش پیدا کند نسبت به قبل. وقتی این اتفاق افتاد نتیجه می گیریم این ع و وزن هایی که با آن اصلاح شده اند خوب هستند، آن ها را نگه می داریم و به قدم بعدی می رویم.

بنابراین به این ترتیب مقدار ع مکررا کم و زیاد می شود تا به یک مقدار مناسب در این قدم برای آن برسیم و در این قدم خطا در حال کاهش است و وزن ها را اصلاح می کنیم و به دور بعدی آموزش می رویم و این مراحل تکرار می شود. سوالی که مطرح می شود این است که آموزش را چه زمانی متوقف کنیم؟

Scanned with CamScanner

۱۴۰۰

شروط توقف: ① تغییر خطای این بار نسبت به دفعه قبل اگر از مقدار مشخصی کمتر شد متوقف شویم:

$$\frac{E(w_m) - E(w_{m+1})}{E(w_m)} \leqslant E_{min}$$

② ε اگر از مقدار آستانه کمتر شد:

$$\varepsilon \leqslant \varepsilon_{min}$$

مقایسه روش steepest Desent با delta-bar-delta و BP نشان داده که steepest descent خیلی کند همگرا می‌شود

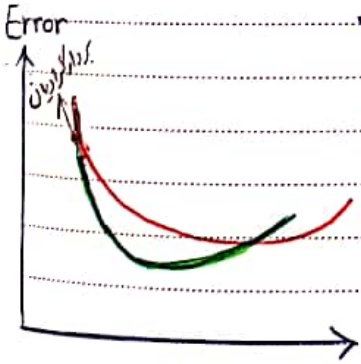
B. روش‌های درجه 2

به جای اینکه پایین رفتن روی سطح را براساس شیب انجام بدهیم، براساس انحنا پیش برویم که مشتق درجه ۲

انحنای سطح خطا در نقطه را به ما می‌دهد

۳ روش وجود دارد:
- QuickProp
- Gauss_Newton Method
- The Levenberg_Marquardt Method

دیدیم که اگر براساس انحنا عمل کنیم بهتر از گرادیان جواب می‌دهد.



درشکل روبه رو، منحنی‌ها تابع خطا را نشان می‌دهند منحنی قرمز رنگ برای حالتی است که از گرادیان استفاده می‌کنیم و منحنی سبز رنگ حالتی است که از انحنا استفاده می‌کنیم.

دو منحنی در ابتدا روی هم هستند و بردار گرادیان برای هر دوی آن‌ها یک جهت را نشان می‌دهد اما اگر از انحنا استفاده کنیم منحنی خط سبز رنگ انحنای بیشتری در این نقطه دارد. بنابراین انحنایی که به دست می‌آوریم ما را به سمت منحنی سبز هدایت می‌کند و زودتر به پایین می‌رسیم و خطا حداقل می‌شود.

بنابراین یک جا ممکن است گرادیان جهت را نشان دهد که احتمالا بهترین جهت برای آن نقطه نباشد و انحنا جهت بهتری را مشخص می‌کند.

Scanned with CamScanner

به طور کلی در روش های درجه ۲ فاصله ی تا نقطه ی min یا نقطه ی کمینه، در فضای وزن ها، با رابطه ی زیر تعیین می شود: مشتق اول خطا تقسیم بر مشتق دوم خطا ←

$$\Delta w = -\frac{\partial E/\partial w}{\partial^2 E/\partial w^2}$$

خطا برای شبکه MLP یا NN با نمونه آموزشی:

$$E(w) = \frac{1}{N}\sum_{i=1}^{N}[t_i - z_i]^2 = \frac{1}{N}\sum_{i=1}^{N}[t_i - f(w, x_i)]^2$$

در روش های درجه ۲ با توجه به نکته ای که در مورد فاصله Δw گفتیم، یک قاعده کلی می توانیم برای تغییر وزن ها به کار ببریم که به این شکل است:

$$w_m = w_{m-1} - \varepsilon R d_m$$

که در آن ε نرخ یادگیری، R یک تابع از مشتق دوم است و d_m مجموع مشتقات خطا برای یک batch یا epoch است یعنی d_m مرتبط با مشتق اول و R مرتبط با مشتق دوم است و اصلاح بایاس مزبور از Δw دار صورت می گیرد.

روش B.2 Gauss_Newton

$$d^s = \frac{\partial^2 E}{\partial w^2}$$ ← مشتق دوم E نسبت به w

روز صنایع دستی

در این روش R ای که در معادله کلی داشتیم برابر $1/d^s$ است. $R = 1/d^s$

بنابراین قاعده آموزش اینگونه است:

$$w_m = w_{m-1} - \varepsilon R d_m \Rightarrow \Delta w_m = -\varepsilon \frac{d_m}{d^s_m}$$

در هر Epoch ابتدا ε را برابر ۱ قرار می دهید و اگر MSE با این ε کوچکتر از آخرین MSE که داشتیم باشد، این ε را می پذیریم اگر کوچکتر نیست ε را تقسیم بر ۲ کنید و وزن ها را به مقدار قبلی بازگردانید و دوباره بررسی کنید که MSE کوچکتر شد یا نه.

یعنی همان فرآیندی که در steepest Descent داشتیم در این جا در این جهت تعیین ε، مشابهش به کار گرفته شده است.

Scanned with CamScanner

وقتی به حالتی برسیم که خطا کاهش پیدا کرده باشد، وزن‌ها را به عنوان وزن‌های جدید قبول می‌کنیم همان طور که گفتیم ε را ابتدا ۱ می‌گذاریم و آن را آپدیت می‌کنیم که باعث می‌شود نرخ یادگیری در حین آموزش خودش تعیین شود و نیاز نداریم نرخ آموزش را تعیین کنیم.

شرط توقف مشابه قبل:

$$① \quad \frac{E(w_m) - E(w_{m+1})}{E(w_m)} \leq E_{min}$$

$$② \quad \varepsilon \leq \varepsilon_{min}$$

تا این جا گفتیم که برای تک وزن کار را به این شکل می‌توانیم انجام بدهیم اما برای حالتی که وزن‌های مختلف را می‌خواهیم داشته باشیم یک راه این است که همه‌ی مشتقات درجه ۲ را در یک ماتریس قرار بدهیم و سعی کنیم به صورت عملیات ماتریسی یکباره برای همه‌ی وزن‌ها این کار را انجام بدهیم.

ماتریسی که مشتقات درجه ۲ را در خودش دارد ماتریس Hessian نام دارد.

چون مشتق درجه ۲ را در فرج به کار داشتیم باید از معکوس ماتریس H باید استفاده کنیم: $R = H^{-1}$

برای یک شبکه با تابع خطای MSE به این صورت است:

$$E = \frac{1}{N} \sum_{i=1}^{N} (t_i - z_i(x_i, w_i))^2$$

درایه‌های ماتریس H اینگونه به دست می‌آیند:

$$H = \frac{\partial^2 E}{\partial w_i \partial w_j} = \frac{2}{N} \sum_{i=1}^{N} \left\{ (t_i - z_i) \left(\frac{-\partial^2 z_i}{\partial w_i \partial w_j} \right) + \left(-\frac{\partial z_i}{\partial w_i} \right) \left(-\frac{\partial z_i}{\partial w_j} \right) \right\}$$

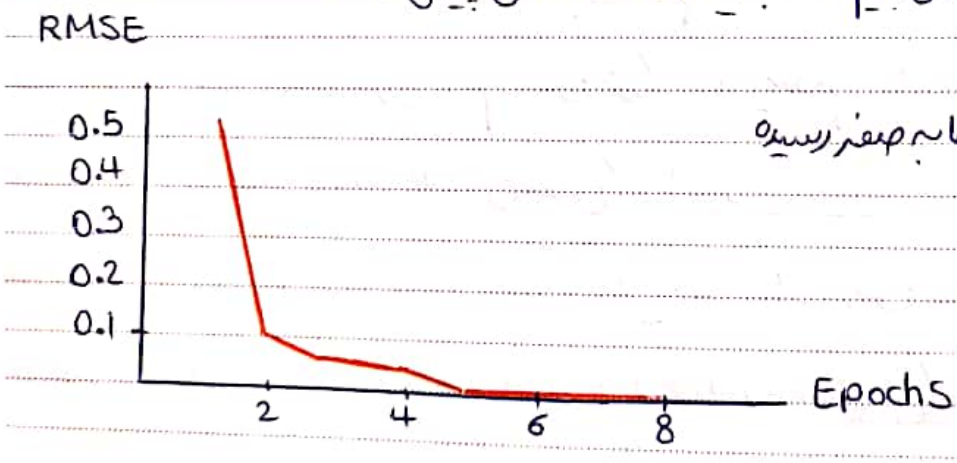
Scanned with CamScanner

رابطه‌ها از مشتق گرفتن از رابطه‌ی E حاصل می‌شوند.

نکته‌ای که مشاهده می‌شود این است که جمله‌ی اول به‌گونه‌ای حساس است که می‌تواند از نظر محاسباتی ناپایداری به وجود بیاورد و در محاسبه‌ی H^{-1} و معکوس کردن H دچار مشکل شویم. به همین دلیل از جمله‌ی اول که مشتق درجه ۲ است صرف نظر می‌شود و H را به این صورت تقریب می‌زنیم:

$$H = \frac{\partial^2 E}{\partial w_i \partial w_j} \approx \frac{2}{N} \sum_{i=1}^{N} \left\{ \left(-\frac{\partial z_i}{\partial w_i} \right) \left(-\frac{\partial z_i}{\partial w_i} \right) \right\}$$

مثال: وقتی این روش را به کاری بریم خطا به این صورت کاهش پیدا می‌کند:



حدودا در 5 تا epoch خطا به صفر رسیده

B.3 روش Levenberg_Marquardt (LM)

به‌جای اینکه E را در نظر بگیریم و هر بار آن را تغییر بدهیم یک جمله‌ی دیگر اضافه می‌کنیم. E را ثابت در نظر می‌گیرد. جمله‌ی e^{λ} را به مشتق اضافه می‌کنیم:

$$R = \frac{1}{d_m^s + e^{\lambda}}$$

قاعده‌ی Δw_m برای یک وزن به این صورت درمی‌آید ←

$$\Delta w_m = -\frac{d_m}{d_m^s + e^{\lambda}}$$

Scanned with CamScanner

ماتریس Hessian به این صورت درمی آید:

$$H' = H + e^{\lambda} I$$

R در حالت ماتریسی:

$$R = \frac{1}{(H + e^{\lambda} I)}$$

Δw_m در حالت ماتریسی ↓

$$\Delta w_m = -\frac{d_m}{(H_m + e^{\lambda} I)}$$

در این حالت باید λ را در فرآیند آموزش تعیین کنیم.

یک مقدار اولیه برای λ در نظر می گیریم و همین طور که جلو می رویم ابتدا به اندازه $\Delta\lambda$ آن را کاهش بدهید و بررسی کنید که MSE کاهش پیدا می کند یا خیر.

اگر کاهش پیدا کرد این حرکت خوب بوده و اصلاح وزن ها را به این شکل قبول می کنیم. اگر MSE کاهش پیدا نکرده، کاهش λ خوب نبوده و $\Delta\lambda$ را به λ اضافه می کنیم. دوباره MSE را محاسبه می کنیم و بررسی می کنیم که قابل قبول هست یا نه.

هرچقدر λ بزرگتر شود چون در مخرج است اصلاح کمتری می شود.

در روش LM نیز همانند Gauss_Newton و steepest Descent باهم ترکیب شده اند.

شرط خاتمه به این صورت است:

$$\lambda > 10^{\Delta\lambda} + Max[H]$$

Max[ ] ← بزرگترین مقدار ویژه ی ماتریس H

Scanned with CamScanner

شرط خاتمه‌ی دیگر:

$$\frac{E(w_m) - E(w_{m+1})}{E(w_m)} < E_{min}$$

شکل از مقایسه روش BP و ورژن های مختلف آن در اسلاید آمده، به ترتیب عملکرد:

BP < delta_bar_delta < steepest Descent < Gauss_Newton < LM

مشاهده می شود که این دو روش درجه ۲ بهتر از همه‌ی روش های درجه ۱ عمل می‌کنند و LM از همه بهتر.

سوالی که مطرح می شود این است که روش های درجه ۲ که اینقدر خوب عمل می‌کنند چه هزینه‌ای دارند؟

هزینه‌شان از لحاظ محاسبات سنگین است که دارند: ماتریس Hessian و معکوس کردن آن و ...

II تغییر تابع فعال سازی نورون ها

پیشنهاد دیگری که دارند عوض کردن تابع فعال سازی نورون ها بود. اینکه سعی کنیم تابع anti_symmetric بکاربریم. یعنی این خاصیت را داشته باشند: $f(-x) = -f(x)$

مثال: Binary and Bipolar Sigmoid، تانژانت هایپربولیک (Tanh(x))، arctan(x)، تابع لگاریتمی Log(1+x)، تابع گاوسی $\exp(-x^2)$

کمتر از مشکلاتی که شبکه‌های FF دارند بر می‌گردد به تابع فعال سازی و اگر آن را عوض کنیم امکان کلیدر هست

III تکنیک های ترکیبی

۱) مثلا استفاده از تکنیک های پردازش تکاملی: الگوریتم ژنتیک می‌تونه برای بهینه کردن معماری و پارامترهای شبکه بکار بره، یا Genetic programming به کار ببریم، روش هایی مثل PSO (هوش جمعی).

نگاه این روش ها این است که طراحی به سمت طراحی بهینه برود و حتی باید هزینه‌ی آن را هم بدهیم، مثلا کندتر بودن

۲) استفاده از مفاهیم فازی در شبکه‌ها (neuro fazy) ← علیه پدیده‌ی ۴، ورودی ۴ یا خروجی آن‌ها اعداد فازی باشند. نتیجه خوب داده

۳) هم فازی و هم الگوریتم ژنتیک را با هم بیاوریم. حسن فازی این است که جزئیات آزار دهنده را با رفتن به یک فضای دیگر حذف می‌کند.

Scanned with CamScanner

جلسه یازدهم - ۱۴۰۰/۸/۹

pattern clustering Neural Network

مسئله‌ی ۳: مسئله‌ی pattern clustering (خوشه‌بندی)

خوشه‌بندی ساختارهایی را که در داده‌های ورودی وجود دارد کشف می‌کند و گروه‌ها (خوشه‌هایی) تشکیل می‌دهد که سازمان دهی آن‌ها معنی‌دار است.

داده‌های ورودی می‌توانند داده‌های خام باشند و یا ویژگی‌هایی باشند که از داده‌های خام استخراج شده‌اند و به هرکدام از آن‌ها یک pattern گفته می‌شود. بنابراین کاری که خوشه‌بندی انجام می‌دهد این است که پترن‌ها یا الگوها را برای تا گروه‌بندی می‌کند به گروه‌هایی (خوشه‌هایی) که معنی‌دار هستند.

هدف این کار این است که بتوانیم در بین الگوهای شبیه‌ها را پیدا کنیم و تفاوت‌ها را پیدا کنیم و بتوانیم براساس آن‌ها نتیجه‌گیری‌های مفیدی برای مسئله‌مان کنیم. (هدف خوشه‌بندی)

برای خوشه‌بندی به ۳ چیز نیاز داریم:

① سنجشگر نزدیکی (proximity Measure)

② معیار خوشه‌بندی (clustering Criterion)

③ الگوریتم خوشه‌بندی (clustering Algorithm)

معیار خوشه‌بندی ← مبنایی است که خوشه‌بندی براساس آن انجام می‌شود

مثال: فرض کنید می‌خواهیم تعدادی حیوان را خوشه‌بندی کنیم. سوالی که مطرح می‌شود این است که برچه اساسی می‌خواهیم خوشه‌بندی را انجام بدهیم؟ مثلا: نحوه تولید مثل، بینی دارند یا نه؟،

معیار خوشه‌بندی روی تعداد دسته‌ها هم تاثیرگذار است

این دسته‌بندی در واقع براساس سنجشگری که خواص این پترن‌ها را اندازه‌گیری می‌کند و نزدیک بودنشان به یکدیگر را بررسی می‌کند، انجام می‌گیرد. (proximity Measure)

Scanned with CamScanner

تعریف رسمی: فرض کنید $X = \{x_1, x_2, \dots, x_N\}$ این x_iها الگوهای ما هستند.

m-خوشه‌بندی برای X اینگونه تعریف می‌شود: X را به m تا زیرمجموعه $C_1, C_2, \dots, C_m$ تقسیم کنیم

به طوری که:

1) $C_i \neq \emptyset \quad i = 1, 2, \dots, m$ هر C_i حداقل یک عضو داشته باشد

2) $\bigcup_{i=1}^{m} C_i = X$ اجتماع C_iها بشود X

3) $C_i \cap C_j = \emptyset \quad i \neq j, \; i, j = 1, 2, \dots, m$ هیچ‌کدام از این خوشه‌ها اشتراک نقطه‌ای با هم نداشته باشند

در هر کدام از این خوشه‌ها الگوهایی که داریم به هم شبیه‌ترهستند نسبت به الگوهایی که در خوشه‌های دیگرند.
یعنی الگوهای خوشه‌های مختلف باهم متفاوت اند و الگوهای درون یک خوشه کم‌شبیه هستند.

موارد استفاده از خوشه‌بندی:

① داده‌هایی که حجم داده‌ها زیاد است، می‌توان با خوشه‌بندی به جای هر الگو (پترن)، هر خوشه را در نظر بگیریم. به آن data Reduction (کاهش حجم داده) می‌گویند. ← مثلا به جای همه اعضای یک خوشه میانگین آن‌ها را در نظر بگیریم

② در مواردی که هیچ‌گونه آشنایی نداریم می‌توانیم یک آشنایی پیدا کنیم و می‌توانیم یک سری فرضیه‌های پیشنهادی بدهیم و آن‌ها را verify کنیم با داده‌های بیشتر (تولید و تست فرضیه)

28 | 18 Jun

④ می‌توانیم برای داده‌های جدید پیش‌بینی کنیم که به کدام خوشه تعلق دارد.

پیش‌پردازش داده‌ها

اگر داده‌ها شرایط مطلوب برای کاری که ما می‌خواهیم انجام بدهیم را نداشته باشند، رسمی می‌کنیم شرایطشان را تغییر دهیم به گونه‌ای که مناسب‌تر بشوند برای هدف ما، به این کار Data preprocessing می‌گویند.

وقتی می‌خواهیم مسئله را حل کنیم یک سوالی که در ابتدای کار مطرح می‌شود این است که کدام متغیرها را وردی بدهیم؟ وقتی تعداد وردی‌ها زیاد باشد تعداد وزن‌ها زیاد می‌شود و آموزش شبکه سخت‌تر می‌شود بنابراین تلاش می‌کنیم که ممکن است کمتر است وردی‌ها را کم کنیم.

Scanned with CamScanner

۱۴۰۰

۲) مسئله دیگر این است که ورودی ها ممکن است با هم correlation داشته باشند که موجب می شود جواب داده ای برای مسئله وجود نداشته باشد و آموزش سخت می شود. و شبکه در واقع در یک فضای با بعد کمتر کار می کند زیرا دو تا از داده ها با هم correlation دارند ما بدون اینکه نیاز باشد رفته ایم به بُعد بالاتر. در این حالت می توانیم از بین ورودی هایی که با هم correlation دارند یکی را انتخاب کنیم و مترادف هم. پس مطلوب است که ورودی ها از هم مستقل باشند.

۳) بسیاری از اوقات جمع آوری داده برای مسئله غیر ممکن است یا هزینه زیادی دارد به طوری که مانع انجام کار می شود. در این حالت تعداد الگوهایی که داریم کمتر از تعداد مطلوب است.

باید سعی کنیم ورودی ها تعدادشان کم باشد تا تعداد وزن ها کمتر باشد و آموزش آسان تر شود. یعنی اگر تعداد متغیرها (تعداد ورودی) زیاد و داده ها کم باشد تعیین صحیح وزن ها مشکل می شود.

۴) ممکن است اندازه اعدادی که داریم خیلی با هم متفاوت باشند. مثلا یکی از متغیرها در بازه [۱۰۰۰, ۱۰۰۰۰] و متغیر دیگر در بازه [۱ و ۰,۰۰۱] باشد. در وزن ها برای فعال شدن نیاز بود که Net Input که x_i ها را با هم جمع می کند. اگر یکی از x_i ها خیلی بزرگ و یکی خیلی کوچک باشد، مقدار بزرگ غلبه می کند و کوچکتر بی اثر می شود. بنابراین اثر متغیرهای کوچکتر وارد نمی شود و یک کار مهمی که در پیش پردازش باید انجام شود این است که همه متغیرها را به یک بازه خاص بیاوریم. مثلا بازه [۱ و ۰] که نرمال سازی (هنجارسازی) نامیده می شود.

نرمال کردن داده ها (Normalization of Data)

① Standardization

هرکدام از اندازه گیری ها را با یک تبدیل خطی به یک فضای جدید ببریم که میانگین متغیرها در آن فضا برابر صفر و انحراف معیارشان ۱ است.

تبدیلی که این کار را انجام می دهد:

$$x_{T_i}^n = \frac{x_i^n - \bar{x}_i}{\sigma_i}$$

$$\bar{x}_i = \frac{1}{N}\sum_{n=1}^{N} x_i^n$$

$$\sigma_i^2 = \frac{1}{N-1}\sum_{n=1}^{N} (x_i^n - \bar{x}_i)^2$$

Scanned with CamScanner

با انجام این تبدیل اکثر داده‌ها در فاصله $26 \pm$ قرار می‌گیرند. چون میانگین آن‌ها را صفر کردیم عددی داده‌ها در اطراف مبدا می‌افتند و در فاصله‌ی $[-26, +26]$

اگر خوشه‌سازی را برای داده‌های انجام بدهیم که مقادیر مطلوب خروجی را هم داریم مثلا عمل دسته‌بندی، باید بردار داده‌های خروجی هم همان عمل (خوشه‌سازی) انجام شود. (چون وزن‌های مناطق نرمالیزه شده‌اند)

اگر این عمل را انجام بدهیم باعث می‌شود بعضی از وزن‌ها بزرگ نشوند که مورد مطلوبی است. چون به همگرایی سریع آموزش صدمه می‌زنند.

② Simple Range Scaling

با این رابطه همه داده‌ها در بازه $[1\ 0]$ قرار می‌گیرند ←

$$x_{Ti} = \frac{x_i - x_{imin}}{x_{imax} - x_{imin}}$$

③ Normalization of correlated Multivariate Data

می‌توانیم عمل نرمالیزه کردن را به‌جای اینکه برای تک‌تک درایه‌ها انجام دهیم، برای یک بردار انجام بدهیم.

داده‌های چند متغیره‌ای داریم که ممکنه بین‌شان همبستگی هم وجود داشته باشد.

اگر K تا وروری داشته باشیم. میانگین انگوهای مختلف را حساب می‌کنیم. N تا انگو داریم و بردار میانگین داریم

$$X = \{x_1, x_2, \dots, x_K\} \quad \bar{x} = \{\bar{x}_1, \bar{x}_2, \dots, \bar{x}_K\} \quad \bar{x} = \frac{1}{N}\sum_{n=1}^{N} x^n$$

هرکدام از مولفه‌های بردار $\bar{x}$، میانگین یکی از متغیرهای ماست.

ماتریس کوواریانس را این‌گونه تشکیل بدهیم:

$$C_X = \frac{1}{N-1}\sum_{n=1}^{N}(x^n - \bar{x})(x^n - \bar{x})^T$$

C_X یک ماتریس متقارن است، ابعادش $k \times k$ است که k بُعد وروری است

می‌خواهیم داده‌ها را به فضای جدیدی ببریم که واریانس واحد باشد و متغیرها از هم مستقل باشند. (روانتر)

می‌خواهیم همبستگی میان متغیرها را از بین ببریم. این کار را می‌توان با eigen value decomposition (تجزیه مقدار ویژه) یا PCA (تحلیل مولفه اصلی) انجام داد.

Scanned with CamScanner

۱۴۰۰

یعنی با اعمال تبدیل PCA روی داده ها می توان از فضای X به یک فضای Y رفت که فضای Y این ویژگی ها را دارد: ① میانگین المنت های آن ۰ است ② ماتریس cov قطری است (بین متغیرها هم بستگی وجود ندارد)

$$Y = A(X - \bar{X}) \qquad m_y = 0$$

تبدیل را ماتریس A انجام می دهد که از روی بردارهای ویژه ی ماتریس C_x به دست می آید.

$$C_y = \frac{1}{N-1}\sum_{i=1}^{N}(Y_i - \bar{Y})(Y_i - \bar{Y})^T = A C_x A^T$$

• تشخیص داده پرت (outlier Detection)

معمولا در داده هایی که از دنیای واقعی جمع آوری می کنیم داده هایی وجود دارند که با بقیه خیلی تفاوت دارند. به آن ها outlier یا داده ی پرت گفته می شود که به آموزش ضربه می زنند بنابراین باید داده های پرت را حذف کنیم. در مقابل داده های پرت داده های inlier یا غیر پرت هستند.

قبل از تشخیص و حذف outlier ها باید روی داده ها نرمال سازی انجام شود.

چگونه داده های پرت را تشخیص بدهیم؟

باید داده ها را بر حسب اندازه مرتب کنیم و آن هایی که در ابتدا و انتهای بازه قرار دارند را بررسی کنیم زیرا خیلی کوچک یا خیلی بزرگ هستند و می تواند جزو داده های پرت باشند

دو رویکرد وجود دارد: یکی اینکه داده های پرت را حذف کنیم (Trimming)

یکی اینکه به جای آن ها داده های جدیدی قرار دهیم ← تغییر داده ها (winsorizing)

← در این جا مقادیر انتهای بازه ها را که تشخیص داده ایم داده های پرت هستند به جای آن ها مقداری قرار می دهیم که نزدیک تر است به میانگین داده ها مثلا به جای همه ی داده هایی که از میانگین + 3σ بزرگ تر هستند همین مقدار mean + 3σ را قرار می دهیم. یا بزرگ ترین مقداری که داده پرت نیست و inlier است را به جای داده های پرت خیلی بزرگ قرار دهیم (برای مقادیر کوچک هم همین طور : mean - 3σ و کوچک ترین غیر پرت)

Scanned with CamScanner

به راه دیگر: تمام اعدادی که بزرگتر از ۹۹ امین صدک یا کوچکتر از اولین صدک هستند را به جای آن ها مقادیر بزرگ را قرار بدهیم. (همان مقدار ۹۹ امین صدک را بگذاریم)

تشخیص داده های پرت در داده های چند متغیره

باید به سراغ تبدیل PCA برویم. PCA یک دستگاه مختصات متعامد به ما می دهد. مولفه های اصلی که در این فضای جدید خواهیم داشت به این صورت هستند که بیشترین واریانس در اولین مولفه ها هست و همین طور که به جلوی رویم واریانس کم می شود.

داده های پرت در تغییرات تاثیرگذار هستند و در واریانس تاثیر می گذارند بنابراین می توان گفت مولفه های اول بیشترین میزان تغییرات را دارند و این تغییرات می توانند تاثیر داده های پرت باشند یعنی اگر داده پرت داشته باشیم تأثیرش بیشتر در مولفه های اول دیده می شود. به این ترتیب می توان از مولفه های اصلی استفاده کنیم و داده های پرت را تشخیص بدهیم.

kامین مولفه اصلی X را با $\alpha_k' X$ نشان می دهیم و $var(\alpha_k' X) = \lambda_k$

مولفه های اول بیشترین واریانس را دارند

λ_k مقدار ویژه ی kام

α_k بردار ویژه ی kام

Scanned with CamScanner

self_organizing (Feature) Maps (SOM)(SOFM)

در این جا وارد مدل آموزش بدون نظارت می شویم و تفاوت این مسئله با مسئله های قبل در این است.

یکی از کارهایی که این شبکه ها می توانند انجام دهند این است که داده ها را از فضاهای با ابعاد بالا Map می کنند به فضای ۱ یا ۲ یا ۳ بعدی.

شبکه هایی که برای این مسئله داریم:

معماری این شبکه های بدون نظارت یک معماری ۲ لایه است: لایه ورودی و لایه خروجی

معماری ساده ای دارند. لایه ورودی متغیرهای ورودی را از خارج شبکه می گیرد و به شبکه اعمال می کند

در این شبکه نورون های لایه خروجی می توانند به صورت خط (آرایه ۱ بعدی)، آرایه ۲ بعدی و یا آرایه ۳ بعدی

چیده شوند. وزن ها به تمام این نورون های خروجی متصل می شوند.

آموزش این شبکه ها اینگونه است که براساس یک قانون یادگیری وزن ها را یکی یکی می گیرد و وزن ها

را آپدیت می کند. تا به همگرایی برسد.

Scanned with CamScanner

انتظار داریم که آموزش به گونه‌ای انجام شود که هرکدام از نورون های لایه خروجی نماینده (مرکز) یک خوشه باشند

وقتی آموزش شبکه تمام می‌شود، مجموعه بردارهای وزن نورون ها CodeBook را تشکیل می‌دهند. هر بردار یک مرکز خوشه را تعیین می‌کند.

یادگیری رقابتی ← در این جا یادگیری بی نظارت ما یادگیری رقابتی است

هر دوری که به شبکه اعمال می‌شود نورون ها با هم رقابت می‌کنند برای اینکه در پاسخ به این ورودی فعال شوند.

در انتهای یک دوره رقابت یک نورون فعال می‌شود و به عنوان بازنده می‌شود

در پایان دوره، وزن نورون برنده را براساس یک قاعده و وزن نورون های بازنده براساس یک قاعده دیگر صورت اصلاح می‌گیرد (یا اینکه اصلا اصلاحی صورت نمی‌گیرد برای نورون های بازنده)

وقتی اصلاح وزن انجام شد دوباره ورودی بعدی می‌آید و دوباره رقابت داریم و برنده که برنده اصلاح می‌شود و بازنده همانند... همین طور این فرآیند ادامه پیدا می‌کند تا شرط خاتمه ارضا شود. به این نوع یادگیری، یادگیری رقابتی گفته می‌شود.

جلسه دوازدهم ـ ۱۴۰۰/۸/۱۱

تیر | ۴
جمعه | ۱۴ ذی القعده
25 Jun

در یادگیری رقابتی فرق نورون برنده و بازنده در آموزشی است که می‌بینند.

اصلاح وزن ها در نورون برنده با نورون های بازنده متفاوت است.

ایده‌ی یادگیری رقابتی از مغزانسان گرفته شده است. اطلاعات مختلفی که دارد مغزانسان می‌شوند هرکدام به یک منطقه از مغزانسان نگاشت می‌شوند. اولین بار Rosenblatt مطرح کرد که یادگیری رقابتی در مغزانسان اتفاق می‌افتد و موجب می‌شود نگاشت اطلاعات به این شکل انجام شود.

Scanned with CamScanner

۱۴۰۰

رقابت به این صورت است که هر نورون تلاش می کند Net Input خودش یا Activation اش max شود. وقتی اینطور شود خروجی مناسب و قوی ای خواهد داشت.

هر نورون سعی می کند خودش برنده شود و بقیه برنده نشوند. تلاش می کند Net Input بقیه نورون ها را پایین بیاورد به همین دلیل یک سیگنال منفی و عادت کننده به بقیه نورون ها می دهد. در نهایت یک نورون برنده می شود.

وقتی نورون برنده مشخص شد، وزن های نورون برنده به این صورت اصلاح می شود: این بردار x که الان به عنوان ورودی آمده یک نقطه در فضاست، وزن نورون برنده هم یک بردار که به مقداری جذب می شه به سمت x. (تولیدی) وزن بقیه ی نورون ها (غیر برنده ها) هیچ تغییری نمی کند. سپس یک ورودی دیگر اعمال می شود، یک نورون برنده می شود، نورون برنده ی جدید به سمت ورودی جدید جذب می شود و ... این فرآیند ادامه پیدا می کند و وزن ها تغییر می کند

لایه ورودی ← به تعداد ابعاد ورودی، نورون دارد

لایه خروجی ← در انتهای آموزش هر نورون در لایه خروجی نشان دهنده ی یک خوشه است و بخشی از داده ها را به خودش اختصاص می دهد. بنابراین تعداد نورون ها در لایه خروجی برابر با تعداد خوشه ها در داده ها است. اگر تعداد خوشه ها را بدانیم به همان تعداد نورون می گذاریم. اگر ندانیم، از مقدار حدسی که می زنیم، نورون بیشتری می گذاریم. در این صورت تعدادی خوشه ممکن است دو تکه شوند که در نهایت باید ترکیبشان کنیم.

برای قانون یادگیری وزن ها باید به مقدار یکوچک اتفاقی مقدار دهی اولیه شوند یا اینکه از بردارهای ورودی m تا را به صورت تصادفی برداریم و به عنوان وزن اولیه بدهیم. مزیتش این که می تونه آموزش را سریع کند.

انتخاب برنده (Winner Selection)

۱- استفاده از فعالیت نورون ها

اون نورون برنده است که حداکثر فعالیت را از خودش نشان بدهد یا به عبارت دیگر، ورودی خالصش بزرگترین ورودی خالص در بین نورون ها باشد. ورودی خالص به این صورت است:

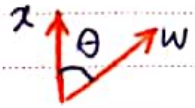
$$I_j = \sum_{i=1}^{n} w_{ij} x_i$$

Scanned with CamScanner

همه‌ی نورون‌ها، وقتی یک ورودی وارد می‌شود، مقدار I_j را محاسبه می‌کنند. هر نورون یک سیگنال عالمت گذاشته برای بقیه نورون‌ها ارسال می‌کند. هر نورون مجددا I_j را با وجود این سیگنال‌های عالمت گذاشته محاسبه می‌کند. یک حد آستانه داریم که اگر activation برای نورون کمتر از آن شد آن نورون را حذف می‌کنیم چون شرایط رقابت با بقیه نورون‌ها را ندارد و از رقابت خارج می‌شود.

۲) استفاده از فاصله تا بردار ورودی ← وقتی یک ورودی وارد می‌شود یک نقطه‌ای از فضای وزن‌ها را نشان می‌دهد که با یک بردار هم می‌توانیم نشان داده شود. Net Input یک نورون را می‌توان به صورت ضرب داخلی x و w_j نوشت:

$$I_j = \sum_{i=1}^{n} w_{ij} x_i = X \cdot W_j = \|X\| \, \|W_j\| \cos\theta$$



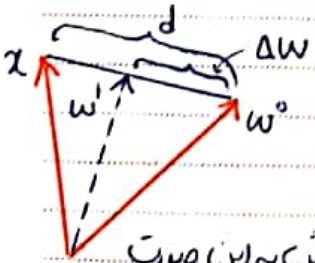
اگر اندازه وزن‌ها را نرمالیزه کنیم به ۱، حداکثر شدن I بستگی به $\cos\theta$ دارد. یعنی θ حداقل باشد. که حداقل بودن θ یعنی نقطه w نزدیکتر باشد به نقطه x. بنابراین:

وقتی یک x وارد می‌آید از فاصله‌ی بردارهای وزن مختلف تا این بردار x استفاده کنیم. اونی که نزدیکتر به x باشد برنده است و اصلاح وزنش هم با جذب شدن w به سمت x انجام می‌شود یعنی θ کوچکتر می‌شود.

فاصله‌ای که حساب می‌کنیم می‌تواند اقلیدسی باشد:

$$d_j = \|x - w_j\| = \sqrt{\sum_{i=1}^{n} (x_i - w_{ij})^2}$$

ممکن است نخواهیم این عملیات را انجام دهیم چون نیاز به توان رساندن و جذر گرفتن به تعداد دفعات زیاد دارد. می‌توان از فاصله‌های دیگر هم استفاده کرد.



بشکل روبه‌رو فرض کنید w^0 نورون برنده باشد وقتی که x وارد شده است. این بدین معناست که w^0 از تمام بردارهای نورون‌های دیگر به x نزدیکتر بوده است. می‌خواهیم w^0 جذب شود به سمت x. بردار $x - w^0$ را در نظر می‌گیریم. قاعده آموزش به این صورت است که بردار وزن نورون برنده برابر بشود با مقداری که قبلا داشت به اضافه‌ی کسری از بردار تفاضل (Δw). در این صورت مقداری که w^0 تغییر می‌کند برابر بردار Δw است و نتیجه‌اش بردار w' است. که به سمت x کشانده شده.

Scanned with CamScanner

۱۴۰۰

$$\Delta w_j = \beta(x - w_j) = \beta d_j$$

$$w_{j+1} = w_j + \beta(X - w_j)$$

قاعده آموزش:

$(0 < \beta < 1)$ ← نشان می دهد چقدر جذب شود

β نرخ یادگیری است و اندازه ای که w نزدیک می شود به بردار ورودی را کنترل می کند. بقیه وزن ها تغییری نمی کنند. در نهایت هر کدام از وزن ها محصول ورود یک سری بردار x است. که طبعا آن ها این بردار وزن، باهم یک خوشه را تشکیل می دهند. فاصله دیگری که می توانیم استفاده کنیم:

$$d_j = \sum_{i=1}^{n} |x_i - w_{ij}|$$

آموزش در این شبکه به ۲ شکل می تواند باشد:

1) pattern or recursive mode ← یعنی وزن ها برای هر ورودی اصلاح می شوند.

2) Batch Mode به ازای هر ورودی وزن ها اصلاح نمی شوند بلکه اصلاح وزن ها بعد از تعدادی ورودی (که به این تعداد یک Batch گفته می شود) انجام می شود.

Batch می تواند تعداد بزرگ باشد که به اندازه همه ورودی ها باشد. در نتیجه اعمال یک Batch می شود یک epoch. یعنی یک epoch ورودی را اعمال می کنم و اصلاح وزن ها را پیدا می کنم ولی اعمال نمی کنم تا epoch کامل شود. در انتهای epoch با میانگین گیری اصلاح را انجام می دهیم.

اعمال ورودی ها epoch به epoch انجام می شود تا جایی که تغییر مهم دیگری مشاهده نشود. یعنی بردار وزن های نورون ها به نقطه ای که باید منتقل می شدند رسیدند.

یک معیار توقف دیگر: حتی متوسط فاصله میان نورون برنده و همه ورودی هایی که این نورون نماینده آن هاست، یک مقدار minimum برسد به این معناست که این بردار وزن به اندازه کافی به همه ورودی ها مربوط به خودش نزدیک است. این فاصله حتی min شود می تواند معیاری برای توقف باشد. این فاصله به این صورت محاسبه می شود: (مجموع)

$$D = \sum_{i=1}^{k} \sum_{n \in C_i} (x^n - w_i)^2$$

برای همه نورون ها (1 تا k)

و برای همه ورودی هایی که در هر خوشه است (C_i)

Scanned with CamScanner

نکته: نورون هایی که در شبکه در کنار هم در یک همسایگی نزدیک قرار دارند خوشه هایی را نشان دهند که در فضای ورودی نزدیک هم هستند.

اگر مراکز خوشه ها را هم وصل کنیم و عمود منصف خط حاصل را رسم کنیم، این عمود منصف ها در نهایی یک شکل را تشکیل می دهند که به یک ناحیه بندی در فضا ایجاد می کند. این اتفاق در دسته بندی هم می افتاد.

می توانیم برای هرکدام از نورون ها یک ناحیه (دوره) جذب تعریف کنیم. (ناحیه تاثیر یا جذب)

ناحیه ای که نقاط موجود در آن به این وزن نورون یا مرکز خوشه نزدیکترند تا سایر مرکز خوشه ها، ناحیه جذب این نورون است و اگر یک ورودی جدید در این ناحیه وارد شود متعلق به این خوشه خواهد بود.

با این ناحیه بندی می توانیم مشخص کنیم بعد از این هر ورودی ای باید متعلق به کدام خوشه خواهد بود.

به این سلول ها Voronoi cells و به این ناحیه بندی Voronoi tessellation می گویند.

self_organizing Feature Maps

یک کار دیگری که کوهونن انجام داد این بود که گفت می توانیم نقشه هایی از داده ها تشکیل بدهیم که داده های با ویژگی های نزدیک به هم در نقشه هم نزدیک باشند. این ایده هم از اتفاقی که در مغز انسان می افتد گرفته شد. هر بخشی در مغز مربوط به کار مشخصی است و در یک ناحیه نزدیک کارهای شبیه به هم در نواحی نزدیک هم انجام می شود. به حفظ سازماندهی مکانی topology preservation گفته می شود. اگر در فضای n بعدی داده هم نزدیک باشند، این داده ها وقتی در مغز نگاشت می شوند، نزدیک هم نگاشت می شوند یعنی سازمان داده ها هم می ریزد.

نقشه ویژگی کوهونن (نقشه ویژگی خودسازمانده) نیز این ویژگی را دارد یعنی داده ها به فضای با ابعاد کمتر به صورت نگاشت می شوند اما این نگاشت به گونه ای است که توپولوژی حفظ شود.

Scanned with CamScanner

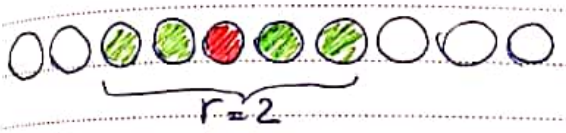
برای اینکه خاصیت حفظ توپولوژی ایجاد شود اینطور عمل می‌کنیم:

وقتی یک نورون برنده می‌شود، نه تنها وزن خودش بلکه وزن همسایه‌هایش را هم اصلاح کنیم.

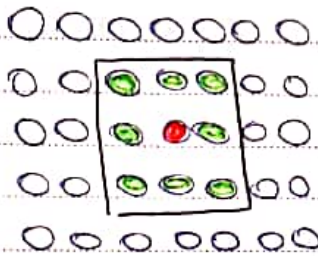
اصلاح وزن همسایه‌ها متناسب با میزان فاصله‌ای که از نورون برنده دارند صورت می‌گیرد. یعنی نورون‌هایی که دورتر هستند اصلاح وزنشان کم‌تر است.

در این جا نیاز داریم که همسایگی را هم تعریف کنیم:

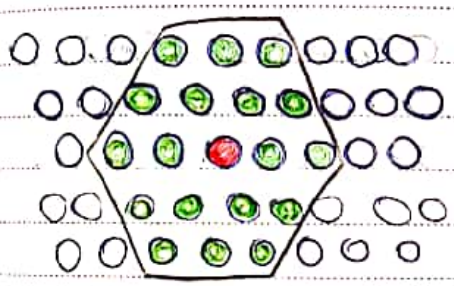
اگر آرایه 1 بعدی باشد:



اگر آرایه 2 بعدی باشد:



همسایگی مربعی با $r=1$



همسایگی ۶ ضلعی با $r=2$

این یک راه است برای تعیین همسایگی‌ها، راه دیگری هم وجود دارد.

فرض کنید ورودی x را داریم: $x = \{x_1, x_2, \dots, x_n\}$

این ورودی در نقطه‌ای از نقشه مانند $\{i_{win}, j_{win}\}$ وارد می‌شود ← محل نورون برنده است

وزن‌های نورون برنده و همسایه‌هایش با این قاعده به روز می‌شود:

$$w_j' = w_j + \beta\, NS[x - w_j]$$

قاعده قبلی که گفتیم برای خوشه‌بندی بود این قاعده برای تشکیل نقشه است.

یک عامل NS داریم که در $x - w_j$ ضرب شده است. NS → Neighbor Strength

NS برای اعمال تاثیر فاصله‌ی نورون از نورون برنده بر روی میزان اصلاح وزن اضافه شده است.

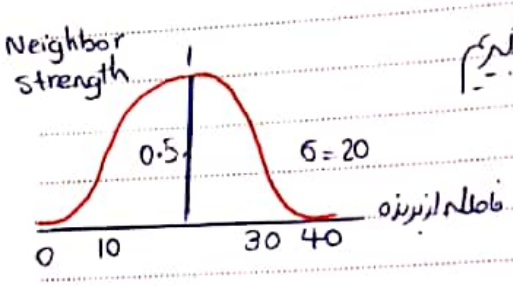
β هم مثل گفتیم ضریب است که میزان اصلاح را مشخص می‌کند.

Scanned with CamScanner

انواع NS ها:

۱) NS می تواند یک تابع خطی که با فاصله کاهش است، باشد. مقدار این تابع در همسایگی به صورت خطی کاهش پیدا می کند (ساده ترین نوع NS)

۲) Guassian decay function ← NS را تابع گاوسی در نظر بگیریم



$$NS = Exp\left[\frac{-d_{i,j}^2}{2\sigma^2}\right]$$

نقطه روی نورون برنده قرار می گیرد (مقدار ۱)

همسایه ها به نسبت فاصله ای که از نورون برنده دارند این تابع مشخص می کند که در چه ضریبی ضرب شوند (به عدد کوچکتر از ۱) هر چه فاصله بیشتر شود این ضریب کوچکتر می شود.

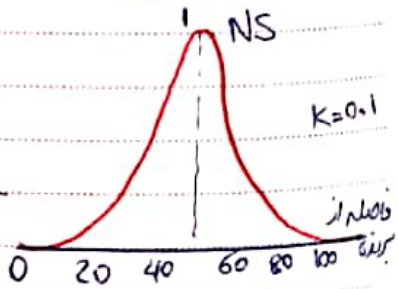
σ در این تابع پهنی و باریکی این تابع را کنترل می کند و اندازه ی همسایگی را هم مشخص می کند. بین σ و اندازه ی همسایگی که در نظر داریم باید رابطه ای برقرار باشد.

روز صنعت و معدن

تیر | ۱۱ | جمعه
2 Jul ■ ۲۱ ذی القعده

۳) Exponential Decay Function ← NS تابع نمایی باشد

$$NS = Exp[-k d_{i,j}]$$



← با سرعت بیشتری NS کاهش پیدا کرده اثر کمتری برای نورون های همسایه دارد. نقطه مثل نورون برنده است.

اصلاح وزن همسایه ها به مرور زمان باید کم و متوقف شود و فقط نورون برنده را اصلاح کنیم. به همین دلیل اندازه ی همسایگی همین طور که آموزش جلو می رود باید با یک قاعده ای کوچک شود. ابتدا همسایگی را خیلی بزرگ می گیریم، حتی گفته شد در ابتدا بیشترین تعداد نورونی که می تواند در همسایگی قرار بگیرد را قرار بدهیم. زیرا در ابتدا بردارهای وزن نورون ها به صورت تصادفی داده شده اند و می توانند در جاهای خیلی پرتی باشند

Scanned with CamScanner

۱۴۰۰

و با آمدن ورودی ها ممکن است یک نورون هیچ گاه برنده نشود. در بین این وزن های تصادفی اولیه ی نورون ها ممکن است یک نورون اقتدر دور باشد از ورودی ها که هیچ گاه نتواند برنده شود. همیشه نورون های دیگری هستند که فاصله شان از ورودی کمتر است نسبت به این نورون. و ما تا آخر بیش می رویم و وزن این نورون همان وزن اولیه ی تصادفی باقی می ماند و این نامطلوب است. به این نورون، نورون مُرده می گویند که وزن هایش اصلا اصلاح نمی شوند.

برای اینکه جلوی به وجود آمدن نورون های مرده گرفته شود یکی از کارها این گفت در ابتدا همسایگی را باید بزرگترین همسایگی ممکن بگیرید و بعد به تدریج اندازه همسایگی را کاهش بدهید. در این صورت در ابتدا همه ی نورون ها اصلاح می شوند و از شکل گیری نورون مرده جلوگیری می شود.

برای کوچکتر کردن اندازه ی همسایگی یک رابطه خطی معرفی کرد:

$$\sigma_t = \sigma_0 \left(1 - \frac{t}{T}\right)$$

σ_0 ← اندازه ی اولیه ی همسایگی

σ_t ← اندازه همسایگی در قدم t-ام

T ← تعداد کل iteration هایی که قرار است σ_0 را به مقدار ۰ برساند، یعنی فقط نورون برنده بماند

می توان رابطه ی نمایی تعریف کرد:

$$\sigma_t = \sigma_0 \, Exp\left[-\frac{t}{T}\right]$$

کاهش اندازه همسایگی را می توان NS (تغییر دامنه) ترکیب کرد:

$$NS(d,t) = Exp\left[\frac{-d_{ij}^2}{2\sigma_t^2}\right] = Exp\left[-d_{ij}^2 / 2\{\sigma_0 \, Exp(-t/T)\}^2\right]$$

Scanned with CamScanner

Learning Rate Decay

دیدیم در حالت کلی کوهونن گفت خود نرخ یادگیری هم نباید ثابت باشد. در شبکه FF هم همینطور بود. ولی ما هیچ وقت در عمل به همگرایی نمی‌رسیدیم، میزان تغییرات جزئی باشد.

خطی

$$\beta_t = \beta_0 \left(1 - \frac{t}{T}\right) \quad \text{یا} \quad \beta_t = \beta_0 \exp\left(-\frac{t}{T}\right)$$

نمایی

$$\beta_t = \frac{2}{3+t}$$

باید β را کاهش بدهیم اما جوری که از اون کمتر نشود.

قاعده اصلاح وزن‌ها در شبکه کوهونن:

$$W_j(t) = W_j(t-1) + \beta(t)\, NS(d,t)\, [x(t) - W_j(t-1)]$$

این‌ها هم آموزش می‌تواند Batch Mode یا Recursive Mode باشد.

آموزش شبکه SOM را می‌توان در ۲ فاز در نظر گرفت:

① ordering (مرتب شدن) ← برای بردارهای وزن topological ordering پیش می‌آید

در ابتدا بردارهای وزن چون تصادفی انتخاب می‌شن ممکنه خیلی پرت و پلا باشند جابجایی‌های بزرگی باید صورت بگیره تا در حدود ناحیه‌ای که باید باشند قرار بگیرند.

در این فاز نرخ یادگیری باید بالا باشد، تندتر هم کاهش پیدا کند به صفر برسد.

اندازه همسایگی باید ابتدا بزرگ باشه (حداکثر نورون‌های لایه پوشش بده) و بعد کاهش پیدا کنه.

برای این فاز چیزی حدود ۱۰۰۰ iteration لازمه

② فاز همگرایی ← در این فاز، دانه‌بندی دقیق است یعنی وزن هر نورون دقیق مشخص می‌شه

تغییرات خیلی کمه جابجایی‌های خیلی کوچیک

چند صد یا چند هزار iteration طول می‌کشه

نرخ یادگیری در کل این فاز باید از صفر باشه که تغییرات جزئی باشه

Fine-tunning

Scanned with CamScanner

مقداردهی اولیه نقشه ← D مقادیر رقابتی

Map

(۲) انتخاب: بردارهای ورودی

(۳) ابتدا از وزن‌های رندوم شروع کردیم سپس مقدار iteration را که جلو رفتیم به سری وزن‌ها بایدیت میدیم

اینارو بعنوان وزن‌های اولیه درنظر گرفتیم و آپدیتش را از سر گرفتیم

جلسه سیزدهم - ۱۴۰۰/۸/۱۴

گفتیم نقشه‌های کوهونن توپولوژی حفظ شده یعنی این نقشه معنی دار است و وقتی ورودی آن حرکت کند به سری وزن‌ها هم کم و زیاد می‌شود.

خاصیت این نقشه این است که اگر ۲ تا ورودی در فضای ورودی کم تر یک باشند در نقشه هم به هم نزدیک هستند و آن‌هایی که دور باشند در نقشه هم دور از هم.

مثال مجموعه داده iris که هدف اینه که یک نقشه ۲ بعدی تشکیل بدهیم و داده‌های ۴ بعدی را به کمک map کنیم

اندازه map ← 8×8

$\beta_0 = 0.1$ $T = 20 \times N$ $N = 150$ همسایگی را گاوسی و ضعیف در نظر گرفته

اندازه همسایگی به تدریج کاهش پیدا می‌کند تا اینکه به ۱ ورودی می‌رسد

در نهایت هر کدام از ورودی‌ها یک بردار مرجع شده‌اند هر کدام یک کد هستند در Codebook

یک سری از ورودی‌ها به آن نورون map شده‌اند و مرکز آن ورودی‌ها را وزن‌های این نورون ارائه میدهد

مثال: Input distribution estimation

گفتیم که در قانون یادگیری، وقتی ورودی می‌آید نزدیک‌ترین بردار وزن نورون‌ها، به سمت آن جذب می‌شود. یعنی بردارهای وزن نورون‌های مانکشانده می‌شوند به سمت جاهایی که داده وجود دارد و جاهایی که چگالی داده‌ها بالاست.

روز قلم

Scanned with CamScanner

این بردارها می‌توانند برای تخمین توزیع ورودی به کار گرفته شوند. یعنی بردارهای وزن نورون‌های شبکه می‌توانند تقریب توزیع ورودی را به ما بدهند.

مثلا ورودی را اگر از یک توزیع یکنواخت در بازه [۱،۰] بگیریم هرچقدر آموزش طول زود سعی می‌کند بردارهای وزن نورون‌ها به صورت متعلم پوشش یکنواخت به صفحه بدهد.

۱. SOM می‌توانیم مدل‌های برای تولید شکل‌های مختلف ایجاد کنیم

Time_adaptive SOM ← فاکتورهای را در SOM آورده اند. اگر شکل با زمان تغییر کند SOM هم به تغییر کند اصلاش. کاربرش. دنبال کردن اشیاء متحرک

Evolving self-organizing Maps

گفتیم SOM خیلی مفیده در حوزه‌هایی دسته‌بندی و خوشه‌سازی داده‌های با ابعاد بالا.

اما این شبکه محدودیت‌هایی دارد ① در مورد اندازه‌ی شبکه باید از اول تصمیم بگیریم و ثابتش کنیم.

برای یک مسئله هیچ ایده‌ای نداریم که چند تا نورون باید بدهیم.

② گفتیم در شبکه خودسازمان‌ده نقشه‌ی حاصل از آن، هر نورون یک خوشه را ارائه میدهد و آن‌هایی که نزدیک به هم واقع می‌شوند چیزهای شبیه به هم را ارائه می‌دهند. اگر بخواهیم به سری داده‌ها را دقیقاً خوشه‌بندی کنیم به سری داده‌های شبیه به هم که روی نقشه به چند تا نورون map شده‌اند، در حقیقت باید به یک نورون map بشوند.

معماری، خوشه‌هایی که روی نقشه تشکیل شده یک مقدار کار اضافی لازم دارد برای اینکه خوشه‌های مربوط با هم ترکیب شوند.

③ وقتی یک نقشه داریم و آن را آموزش داده‌ایم، ورودی جدیدی که داریم نمی‌توانیم به هر جای نقشه که

Scanned with CamScanner

کمترین جایی این هست map نیست بلکه فهوده به یکی از نورون‌ها map نیست. یعنی بدین
مکانیزم عملیات گسسته دارد و پیوسته نیست

برای حل این مشکلات کارهایی صورت گرفته است. به یکی از شبکه‌ها به نام SOM رشد یابنده به
وجود آمده. چند رشته هستند. به صورت پویا نقشه را شکل می‌دهد. یک نقشه اولیه از شکل کار
و رشد پیدا می‌کند.

جلسه چهاردهم - ۱۸/۸/۱۴۰۰

(GSOM) Dynamic self-organizing maps with Controlled Growth

در این روش همین طور که جلوی می‌رویم نقشه رشد می‌کند. این رشد توسط یک عامل گسترش یا
عامل توزیع (Spread Factor) کنترل می‌شود.

اگر در این سیستم یک نقشه‌ی کوچکی داشته باشیم یک تصویر کلی به ما می‌دهد از خوشه‌ها.
این ویژگی به خصوص وقتی مفید است که دنیاست های بزرگ داشته باشیم (broader) (macro)
و خواهیم یک دید کلی از دیده‌ها به دست بیاوریم. در این حالت یک نقشه‌ی با یک SF پایین ایجاد می‌کنیم.
اگر خواهیم نقشه‌ای با جزئیات بیشتر داشته باشیم باید شبکه را بیشتر رشد بدهیم.
یکی دیگه‌ای که دیشه کرد: به تیکه‌ی کوچکش را گرفت و بسطش داد. این متنی که علاقه‌مندیم.
و بزرگش کنیم.

رشد این شبکه دارای خوشه‌هایی است که در داده‌ها وجود دارد. در واقع شکلی که این رشته به خودش
می‌گیرد منتج از خوشه‌هایی است که در داده‌ها است.
مثلا فرض می‌کنیم شبکه دیگ قسمت‌های خاصی بیشتر رشد کرده و همین قسمت‌ها به ما خوشه‌ها را
نشان می‌دهند.

Scanned with CamScanner

دیدگاه دیگر این شبکه این است که شبکه‌ای که رشد آن به خودش می‌گیرد در بخش‌های خوشه‌های آن ۱۴۰۰
است. مثلا در شکل می‌بینیم

شروع کار از یک نقشه‌ی اولیه‌ی ۲×۲ (چهار نورون) است. در ۴ راس یک مربع نورون‌ها قرار می‌گیرند.
افزایش نورون‌ها در حین رشد شبکه، در نورون‌های مرزی انجام می‌گیرد که این نورون‌ها بالاترین خطای
تجمع (highest accumulated error) را دارند. اگر جای نورون باشد که خطای بالایی را نشان می‌دهد،
در واقع نشانه‌ی این است که این جا به نورون بیشتری نیاز داریم و بنابراست خوشه‌ها بیشتر تشکیل بشه که خطا بیاد پایین.
خوشه‌های این جا برابل‌گش دارند. پس در مرزها خطا بالاست آن‌طور می‌شه
اضافه شدن نورون با یک قاعده‌ای است که ساختار شبکه کم کم رشد. همان ساختار نوری مانندی که
از ابتدا با آن شروع کرده بسطش می‌دهد.

پارامتر spread factor برای کنترل رشد به کار می‌رود. اگر در ابتدا مقدار پایین برای آن در نظر بگیریم
یک نقشه کلی می‌دهد و بعد با افزایش آن جزئیات بیشتری را می‌توانیم داشته باشیم و در واقع بسط پیدا می‌کند
در نواحی. ابتدا با ۴ نورون شروع میشه و به مقداری که جلوی رود مشخص می‌شه نورون خطای بالایی دارد از آن جا بسط پیدا می‌کند
بسط به این صورت است که در تمام جهت‌های آزاد نورون که دارای خطای بالاست، نورون اضافه می‌شود.

فرآیند آموزش

1. تعیین مقادیر اولیه: ۴ تا نورون در شبکه قرار می‌دهیم. باید وزن‌های اولیه‌ای بهشان بدهیم. می‌توانیم رندوم انتخاب کنیم
همچنین می‌توانیم ۴ تا از بردارهای ورودی را انتخاب کنیم و به عنوان وزن اولیه بدهیم.
بهتر است روی داده‌ها یک عمل Normalization انجام بدهیم تا همه را در فاصله‌ی ۰ و ۱ بیاوریم و مقیاس‌هایی
که از نظر مقیاس با هم خیلی فرق دارند را در یک محدوده قرار بگیرند و این‌ها را به عنوان وزن اولیه بدهیم.

Scanned with CamScanner

۲ فازبسط (Growth phase)

ورودی ها را به شبکه می دهیم و هرکدام به یک نورون کمپین پیدا می کند.

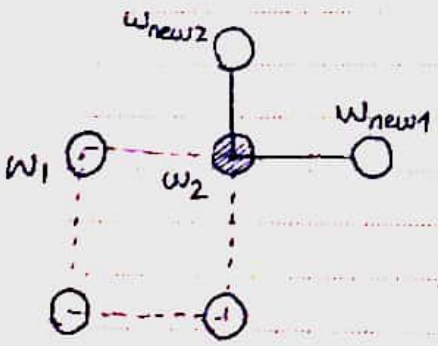
می توانیم در هر مرحله خطای مجتمع را برای هر نورون پیدا کنیم. خطای مجتمع یعنی یک نورون داریم که وزن هایش به عوان ورودی خودش حساب می شه، و فاصله هرکدام از ورودی هایی که به این خوشه اختصاص پیدا کرده اند تا مرکز خوشه حساب می کنیم و با هم جمع می کنیم. جایی که بزرگترین خطای مجتمع را داریم باید برسی بشه برای اضافه کردن نورون.

در ابتدا که ۲ تا نورون داریم اگر بگیم که فضای ورودی ما را به ۲ ناحیه تقسیم می کنند. نورونی که بیشترین خطا را دارد بیان می دهد که ورودی هایی که به ناحیه ی مربوط به آن می آیند پراکندگی شان زیاد است و ناحیه ای است که پتانسیل آن را دارد که شبکه از آن عبور کند.

خطای مجتمع اگر از یک ترشولد (GT) growth threshold) بزرگتر باشد، می گوییم شبکه باید در این قسمت رشد بکند. یعنی یک دسته های آزاد مربوط به این نورون، نورون اضافه شود.

وزن های اولیه ی نورون های جدید را چگونه بدهیم؟

وزن های اولیه برای نورون های تازه اضافه شده باید به گونه ای باشد که تغییرات وزن در ابتدا اثر زیادی (عمودی حالت هموار داشته باشد و نوسان نکند. (کم و زیاد نشود)



توصیه کرده اند: اگر بین w_2 از w_1 کوچکتره وزن w_{new1} با هم کوچکتر از w_2 باشد. همانند که w_2 از w_1 کوچکتره، w_{new1} هم از w_2 کوچکتر باشه.

if $w_1 < w_2 \rightarrow w_{new1} = w_2 + (w_2 - w_1)$

if $w_1 > w_2 \rightarrow w_{new1} = w_2 - (w_1 - w_2)$

Scanned with CamScanner

ابتدا کار را با وزن‌ها داریم و هرکدام وزن‌ها شروع می‌شوند چون هرکدام $\frac{1}{4}$ صفحه را پوشش می‌دهد و وزن‌های یکه هرکدام انتخاص پیدا میکند عکس حیلی متفاوت باشند.

نتیجه این مسئله این است که اگر از قانون کمتگام سازی وزن‌ها استفاده کنیم وزن‌ها عکس نوسان کنند به طوری که وزن‌ها حیلی متفاوت اند و وزن‌ها همراه آن‌ها کشیده می‌شوند.

در نتیجه گفته اند نرخ یادگیری را تغییر کنند براساس اینکه در چه مرحله‌ای از آموزش هستیم. اندازه شبکه چقدر است پیشنهاد دادند η که نرخ یادگیری است

$$\eta(t+1) = \alpha \varphi(n) \eta(t)$$

را اینگونه آپدیت کنیم:

α یک ضریب ثابت است $(0 < \alpha < 1)$ تعداد نورون‌های شبکه

$\varphi(n)$ یک تابع است که مقدار آن وابسته به اندازه شبکه است و رابطه مستقیم با اندازه شبکه دارد. یعنی وقتی اندازه شبکه کوچک، کوچک و برعکس. یک پیشنهاد برای φ:

$$\varphi(n) = 1 - \frac{R}{n(t)}$$

در ابتدا این ضریب کوچک است و با افزایش $n(t)$ افزایش پیدا می‌کند

R را خودشان پیشنهاد دادند که 3.8 بگیریم. n تعداد نورون‌ها در قدم t است. در ابتدا $1 - \frac{3.8}{4}$ عدد کوچکی است. وقتی ضریب در ابتدا کوچک یعنی جابجا شدن بردار وزن نورون با آمدن ورودی، کم است اما وقتی نورون‌ها زیاد شد بزرگ حرکت می‌کند.

در شبکه SOM گفتیم که در ابتدا همسایگی را تا حای ممکن بزرگ می‌گیریم به طوری که شامل حداکثر همسایه‌ها شود و بعد دفعه به دفعه آن را کوچک می‌کنیم. گفتیم به شبیک فاز ordering داریم که نورون‌ها جابجا می‌شوند براساس اینکه ورودی‌ها کجا هستند قرار می‌گیرند. و بعد هم فاز convergence داریم که دقیق تنظیم می‌شود.

اما این فاز ordering نداره شبکه چون همین که وزن اضافه می‌کنیم خودمان ترتیب را رعایت می‌کنیم و تا حدود زیادی نورون‌ها خودشان مرتب هستند. در این حالت همسایگی‌های کوچکی در نظر می‌گیریم مثلا ۱ و ۲ و ۳ و ۴ در نظر می‌گیریم و بعد کاهش می‌دهیم به ۱.

روز گفت و گو و تعامل سازنده با جهان

Scanned with CamScanner

گفتیم که در این مسئله بسط از طریق نورون های مرزی صورت می گیرد حال اگر خطای مجتمع زیاد برای یکی از نورون های مرکزی رخ دهد باید چه کار کرد ؟ در این حالت در دادن وسط نقشه ی ما خلل رسد و نورون های همسایه باید کار بیشتری انجام دهند که ما این فشار را از این محل کم کنیم.

برای کاهش این فشار پیشنهاد شد که فشار را منتقل کنیم به همسایه هایش. برای همسایه ها هم این روند تکرار شود تا همسایه مرزی برسد.

برای انجام این کار، نورون که خطای مجتمعش بیشتر از GT شده، خطای مجتمع آن را برابر نصف آستانه قرار دهید

$$E^{w}_{t+1} = \frac{GT}{2}$$

و دوتا از همسایه ی آن را در نظر گرفته و خطاهایشان را با کسری از خطا جمع کنید

$$E^{i}_{t+1} = E^{i}_{t} + 2E^{i}_{t}$$

با این ترتیب اندازه ی نورون منتقل می شود به همسایه هایش.

2 عددی بین 0 و 1 است

تیر | ۲۵
جمعه
16 Jul ■ ۵ ذی الحجه

گفتیم که Growth threshold تکلیف این که شبکه رشد بکند یا نه را مشخص می کند و کنترل می کند. یک عامل گسترش یا SF تعریف می کنیم: Spread factor (SF) عددی بین 0 و 1 است. تعیین کننده ی زمان است که شبکه رشد می کند و مرتبط با GT است

SF باید با GT در ارتباط باشد چون هر دو رشد شبکه را کنترل می کنند:

$$GT = -D \ln(SF)$$

D ← تعداد داده های ورودی $\quad 0 < SF < 1$

$-D \ln(SF)$ ← عددی مثبت است که حریص از بعد ورودی است

براساس این رابطه وقتی SF کوچک باشد مجبور می شود که GT بزرگ تر شود و وقتی GT بزرگ باشد طول زمان شبکه گرفته می شود بنابراین SF کوچک یعنی رشد کمتر. وقتی SF بزرگتر شود GT کوچکتر می شود در نتیجه رشد آسانتر انجام می شود. شبکه بیشتر رشد می کند جزئیات بیشتری می شود

Scanned with CamScanner

فاز رشد به این صورت ادامه پیدا می کند اما زمانی می رسد که دیگر نیازی نیست شبکه رشد پیدا کند و این حالت پیش می آید که خطای مجتمع نورون ها به GT برسد یا اینکه می رسد به حداکثر تعداد پایین اتفاق می افتد. در این حالت می گوییم رشد دیگه تمام شده و وارد فاز دیگری می شویم.

فاز هموارسازی (Smoothing)

در این فاز آموزش را ادامه می دهیم با یک نرخ یادگیری پایین. هدفش اینه که به حرکات جزئی اتفاق نیاز داشتند بتونه ای ام سبشه و تنظیمات ظریف شبکه صورت بگیرد.

همسایگی در این فاز فقط شامل خود نورون و ۴ تا همسایه ای اطرافش می شود.

مثال: Zoo dataset

در ابتدا SF=0.1 ، تنها یک صفحه شبکه ابتدایی طرح کلی درست آورد. مشاهده می شه در ۳ جهت اشتباه کرده.

دوباره با SF=0.25 روشن دهد نقشه ای با جزئیات بیشتر به دست می آید. در هر شافل کلی به سراغ خوشه های جدید می شود.

یکار دیگه که میشه ای ام داد: یه کپی قبل قبلی از نقشه را با SF بالا بسط بدهیم

وقتی شبکه را استپ دهیم اگر داریم ورودی را بالا می بریم. لازم نیست مقدار نورون ها دقیقا برابر مقدار خوشه هایشه

بعد از این بحث وارد آموزش supervised می شویم دوباره

Scanned with CamScanner

جلسه پانزدهم - ۱۴۰۰/۸/۲۳

جدایی پذیری الگوها

وقتی مسئله خیلی پیچیده می شود، آن را میبریم به یک فضای دیگر که احتمال میدهیم در آن فضا مسئله راحت تر حل شود. یک تبدیل غیرخطی به داده ها اعمال می کنیم و آن را به فضایی با ابعاد بزرگتر مسادی فضای قبلی می بریم. برای یک مسئله پیچیده اگر آن را به فضای با بُعد بالا با تبدیل غیرخطی ببریم این احتمال وجود دارد که بتواند به صورت خطی جدایی پذیر شود. (و این احتمال در فضای با بعد بالاتر بیشتر است تا فضای با بعد پایین تر) به شرطی که فضا خیلی پرچگال نباشد (نقاط به صورت dense نباشند)

χ (χ_1 / χ_2)

فرض کنید χ مجموعه‌ای از x_i ها بردار باشد که هرکدام متعلق به χ_1 یا χ_2 که زیرمجموعه‌های χ اند، می باشند.

$$\phi(x) = [\varphi_1(x), \varphi_2(x), \dots, \varphi_m(x)]^T \qquad \phi(x): R^n \longrightarrow R^m \qquad m \gg n$$

برای هر بردار x یک بردار تعریف می کنیم که شامل مقادیر حقیقی توابعی است که این توابع را با φ_i نشان می دهیم. کاری که این بردار $\phi(x)$ انجام می دهد این است که x را از فضای n بُعدی می برد به یک فضای جدید که با ϕ نشان داده می شود و m بُعدی است.

نام دیگر فضای ϕ فضای ویژگی است. می توان گفت φ_i ها ویژگی های x هستند.

یک دسته بندی X به دو مجموعه χ_1 و χ_2 را می گوییم φ separable (φ جدایی پذیر) است اگر بردار m بُعدی w وجود دارد به طوری که:

$$w^T\phi(x) > 0 \qquad x \in \chi_1$$

$$w^T\phi(x) < 0 \qquad x \in \chi_2$$

یعنی با محاسبه‌ی $w^T\phi(x)$ و بررسی علامت آن می توانیم تصمیم بگیریم x متعلق به دسته χ_1 یا χ_2 است.

مرزی که این ۲ دسته را در فضای φ از هم جدایی کند با معادله روبه رو مشخص می شود: $w^T\phi(x) = 0$

در فضای اولیه همان هم x هایی که در $w^T\phi(x) = 0$ صدق می کنند مرز را تشکیل می دهند.

Scanned with CamScanner

۱۴۰۰

این ایده می تواند در شبکه های عصبی به کار گرفته شود و شبکه ای که این کار را می کند RBF است.

Function approximation with radial basis functions

ما می توانیم یک تابع را با یک سری توابع دیگر تخمین بزنیم که این توابع یک سری توابع پایه هستند و یک نوع از آن ها توابع پایه شعاعی است

فرض کنید تا تا نقطه ی x_i داریم و تا مقدار متناظر این x_i ها

که با d_i نشان می دهیم:

$$\{x_i \in \mathbb{R}^n \quad i=1,2,3,\dots,L\}$$

$$\{d_i \in \mathbb{R} \quad i=1,2,3,\dots,L\}$$

x_i ها n بعدی اند d_i اسکالر حقیقی است و مقدار تابعی است که می خواهیم تخمین بزنیم.

هدف این است که تابع f را بیابیم به گونه ای که:

$$f(x_i)=d_i \qquad i=1,2,3,\dots,L$$

یک راه که پیشنهاد شد : f را برابر مجموع وزن دار یک سری توابع نشان دهیم که به این توابع، توابع پایه شعاعی می گوییم و آن ها را با φ نشان می دهیم

$$f(x)=\sum_{j=1}^{L} w_j \, \varphi(\|x-x_j\|)$$

بنابراین مجموعه توابعی که به کار گرفته می شود را به این شکل نشان می دهیم:

$$\{\varphi(\|x-x_j\|),\ j=1,2,\dots,L\}$$

$\|.\|$ ← نرم اقلیدسی

داده های ما x_j ها هستند که آن ها را به عنوان مرکز این توابع در نظر می گیریم. یعنی علت اینکه به جای $\varphi(\|x\|)$ داریم $\varphi(\|x-x_j\|)$ این است که مراکز روی نقاط x_j باشند.

$$f(x_i)=\sum_{j=1}^{L} w_j \, \varphi(\|x_i-x_j\|)=\sum_{j=1}^{L} w_j \varphi_{ij} \qquad i=1,2,\dots,L$$

اگر برای i های مختلف این رابطه را بنویسیم، تا تا رابطه به دست می آید که به جای $f(x_i)$ هم می توان d_i را در آن ها قرار داد.

تا تا معادله را می توان در یک فرم ماتریسی به شکل روبه رو ترکیب کرد:

$$\begin{bmatrix} \varphi_{11} & \varphi_{12} & \cdots & \varphi_{1L} \\ \cdot & \cdot & \cdots & \cdot \\ \cdot & \cdot & \cdots & \cdot \\ \varphi_{L1} & \varphi_{L2} & \cdots & \varphi_{LL} \end{bmatrix} * \begin{bmatrix} w_1 \\ \cdot \\ \cdot \\ w_L \end{bmatrix} = \begin{bmatrix} d_1 \\ \cdot \\ \cdot \\ d_L \end{bmatrix}$$

Scanned with CamScanner

رابطه‌ی ماتریسی: (بردار مقادیر مطلوب) desired values — weight — interpolation matrix

$$w = \Phi^{-1} d$$

Φ — w — d

d برای ما معلوم است، اگر φ ها هم معلوم باشند w به دست می‌آید. با مشخص شدن w، f به دست می‌آید.

قضیه Micchelli

فرض کنید مجموعه‌ای از نقاط متمایز $\{x_i, i = 1, 2, \dots, L\}$ را در فضای n بعدی داریم، در این صورت ماتریس interpolation: $\Phi : L \times L$ که عضو φ_{ij} ام آن φ_{ij} است: $\varphi_{ij} = \varphi(\|x_i - x_j\|)$ $i, j = 1, 2, \dots, L$ nonsingular (غیر منفرد) است یعنی معکوس دارد و برای تعیین معکوس Φ در رابطه‌ی ماتریسی که بالا گفته شد مشکلی نداریم.

توابعی که می‌توانند به عنوان تابع RBF انتخاب شوند:

① Multiquadrics — $\varphi(r) = (r^2 + c^2)^{1/2}$ — $c > 0, r \in \mathbb{R}$

② Inverse Multiquadrics — $\varphi(r) = \frac{1}{(r^2 + c^2)^{1/2}}$ — $c > 0, r \in \mathbb{R}$

③ Gaussian functions — $\varphi(r) = \exp\left(-\frac{r^2}{2\sigma^2}\right)$ — $c > 0, r \in \mathbb{R}$

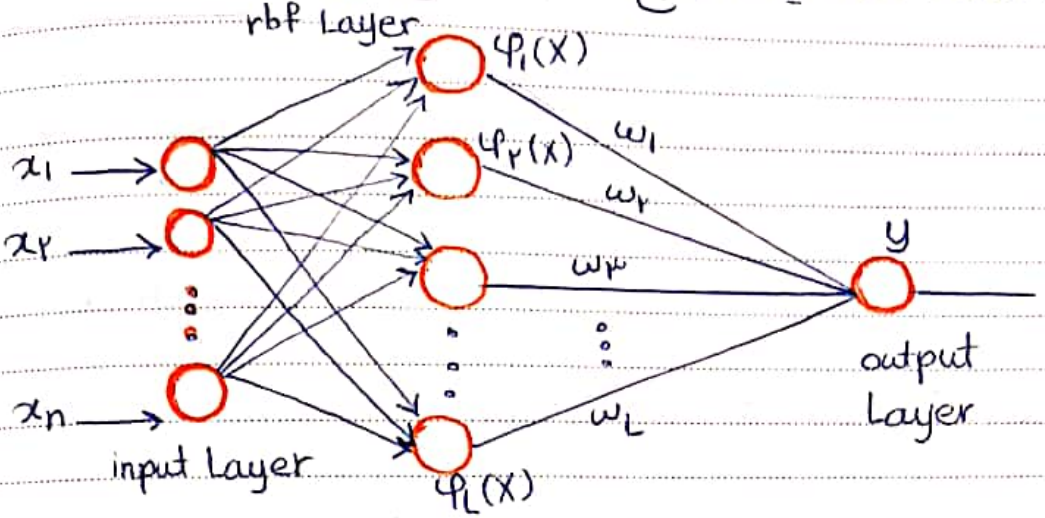
مثال (در اسلاید)

در شبکه RBF ابتدا در لایه‌ی مخفی ای که شبکه دارد (لایه دوم شبکه)، x ها را به فضای Φ ی می‌بریم در نتیجه معماری آن به این صورت است: ① یک لایه ورودی داریم که همه‌ی ورودی‌های x را به لایه‌ی بعدی می‌دهد. ② لایه‌ی مخفی که عمل تبدیل به فضای Φ در آن صورت می‌گیرد و خروجی آن به این صورت است: $\varphi_1(X), \varphi_2(X), \dots, \varphi_L(X)$ و بعد این φ ها با یک سری وزن‌هایی باهم

Scanned with CamScanner

ترکیب می شوند و خروجی را به ما می دهند.

بنابراین این شبکه ساده است و ۳ لایه دارد و تابع f را برای ما تخمین می زند



n تا نود در لایه ی ورودی داریم چون X ، n بُعدی است. L تا نود در لایه ی مخفی داریم که با تعداد نمونه های آموزشی ما است. یعنی شبکه به تعداد نمونه های آموزشی نورون دارد که مرکز تابع φ در هرکدام از نورون ها، روی آن داده قرار می گیرد.

این توابع σ (سیگما) هم دارند که پهنا را نشان می دهد. معمولا σ را در همه نورون ها یکسان می گیرند.

معمولا چون یک خروجی را می خواهیم تخمین بزنیم در لایه خروجی ۱ نورون کافی است.

به لایه ی مخفی، لایه ی RBF هم گفته می شود.

$$\varphi_j(X) = \varphi(\|X - X_j\|)$$

$$= \exp\left(-\frac{1}{2\sigma_j^2}\|x - x_j\|^2\right)$$

گفتیم که مراکز توابع RBF روی داده ها قرار می گیرد و به ازای هر داده یک نورون RBF خواهیم داشت.

- اگر داده ها زیاد باشند تعداد نورون های لایه RBF خیلی زیاد می شود و این می تواند مشکل ساز شود.
- از طرفی ممکن است داده ها نویزی باشند و باعث شوند نتایج نامربوط و ناخواسته ای از شبکه حاصل شوند. انگار داریم به نویز همان قدر اهمیت می دهیم که به داده های اصلی اهمیت می دهیم

بنابراین این طرح که به ازای هر داده یک نورون RBF داشته باشیم جالب نیست. پس چه کاری می توانیم به جای آن انجام بدهیم؟

Scanned with CamScanner

جواب این است که تعداد نورون های RBF را کمتر از تعداد داده ها بگیریم ($K < L$)

$$f(X) = \sum_{j=1}^{K} w_j \varphi(\|X - \mu_j\|) = \sum_{j=1}^{K} w_j \exp\left(-\frac{1}{2\sigma_j^2}\|X - \mu_j\|^2\right)$$

بنابراین ضابطه $f(X)$ به صورت بالا درمی آید. j از ۱ تا K جلو می رود. به جای X_j، یه μ ای گذاشتیم μ مرکز تابع گاوسی است که بین داینم و μ چیست. اگر φ را تابع گاوسی در نظر بگیریم (که معمولا اینطور هست) رابطه‌ی تابع گاوسی را جایگذاری می کنیم که در بالا آورده شده.

در این ها K تا سیگما (σ) و K تا μ داریم که پارامترهای ما هستند. این ها باید مشخص بشوند تا توابع پایه هم مشخص شوند. بعد از مشخص شدن توابع پایه، وزن ها هم باید مشخص شوند تا مسئله حل شود. (پس برای حل مسئله μها برای K تا نورون، σها و وزن ها باید تعیین شوند)

یک ایده این است که K تا نورون که توابع RBF را مشخص می کنند را حالی قرار بدهیم که داده ها خوشه شده اند یعنی هر جا که یک خوشه مشاهده می شود یک نورون قرار بدهیم. بنابراین راه همان اینکه ها چه چیزی باید باشند این است که داده ها چگونه در فضای ورودی خوشه می شوند.

به همین دلیل سراغ الگوریتم K_means می رویم

مرداد | ۱
جمعه
23 Jul ■ ۱۲ ذی الحجه

الگوریتم K میانگین (K_means)

فرض کنید مجموعه نقاطی که در فضای n بعدی حقیقی داریم X_i ها هستند: $\{X_i \in \mathbb{R}^n, i = 1, 2, \dots, L\}$

می خواهیم این X_i ها را به K تا خوشه، خوشه بندی کنیم. پس تعداد خوشه ها K است و به عنوان ورودی به الگوریتم داده می شود.

$d(x_i, x_{i'})$ ← معیار شباهت جفت جفت داده ها

یک نگاشت کننده یا encoder در نظر می گیریم که عنوان های i را به یکی از K تا خوشه map می کند در واقع تعیین می کند که هر عنوان به کدام خوشه باید متعلق باشد و این را بر اساس معیار شباهت

Scanned with CamScanner

$j = C(i) \quad i = 1, 2, \dots, L \rightarrow$ many_to_many mapper (encoder)

انجام می‌دهد.

↳ داده‌ی i ام را به خوشه‌ی ژام assign می‌کند

وقتی خوشه‌بندی انجام شود مرکز خوشه‌ها پیدا می‌شوند و مرکز خوشه‌ها را همان μها در نظر می‌گیریم

یعنی برای مراکز توابع RBF، مرکز خوشه‌ها را در نظر می‌گیریم.

الگوریتم K_means سعی دارد تابع زیر را کمینه کند:

$$J(C) = \frac{1}{2} \sum_{j=1}^{K} \sum_{C(i)=j} \sum_{C(i')=j} d(x_i, x_{i'})$$

d را برابر فاصله‌ی اقلیدسی می‌گیریم و می‌خواهیم

مجموع فواصل را حداقل کنیم. به عبارتی دقیق‌تر فاصله‌ی بین دو نمونه حداقل شود یعنی شبیه‌هستان حداکثر شود

و این مسئله باعث می‌شود تا نمونه‌های شبیه به هم در یک خوشه قرار بگیرند.

بنابراین هدف ما این است که $J(C)$ را min کنیم $d(x_i, x_{i'}) = \|x_i - x_{i'}\|^2 = \|x_{i'} - x_i\|^2$

با قرار دادن d در $J(C)$ داریم:

$$J(C) = \frac{1}{2} \sum_{j=1}^{K} \sum_{C(i)=j} \underbrace{\sum_{C(i')=j} \|x_i - x_{i'}\|^2}$$

$\hat{\mu}_j \leftarrow$ تخمین بردار مرکز خوشه‌ی ژ

$$J(C) = \sum_{j=1}^{K} \sum_{C(i)=j} \|x_i - \hat{\mu}_j\|^2$$

بنابراین مجموع فواصل تا مرکز را min می‌کند.

می‌توان نشان داد با احتساب یک ضریب برابر واریانس است. چون $x_i - \mu$ یک تخمین از واریانس در خوشه‌ی ژ است پس مجموع واریانس‌ها را داریم حداقل می‌کنیم.

$$\hat{\sigma}_j^2 = \sum_{C(i)=j} \|x_i - \hat{\mu}_j\|^2$$

Scanned with CamScanner

الگوریتم K_means :

ابتدا یک مقدار اولیه برای μ انتخاب می‌کنیم ← تخمین مراکز خوشه‌ها

تکرار قدم های زیر:

برای داده ها از 1 تا L تصمیم بگیر که هر داده به کدام مرکز از همه نزدیکتر است. (نزدیکترین مرکز به هر داده را مشخص کن) و این مقدار را برای این داده ثبت کن: Set b(i)=q

و این کار را برای همه‌ی داده ها تکرار کن.

با انجام این عمل هر کدام از داده ها به خوشه‌ای تعلق می‌گیرند، حالا x های هر خوشه را میانگین بگیر و اسم آن را μ برای آن خوشه بگذار

انقدر این مراحل را تکرار کن که مراکز دیگر تغییری نکند ← در این صورت مراکز خوشه ها پیدا می‌شود

الگوریتم K_means در واقع یک نگاشت غیرخطی به ورودی X اعمال می‌کند

توصیه می‌کنند الگوریتم را چندین بار برای مقادیر اولیه مختلف μ اجرا کنید و J(C) را در آن ها مقایسه کنید.

هرکدام J(C) کوچک تری داشت آن را انتخاب کنیم.

بنابراین μ ها را اینگونه حساب می‌کنیم. مسئله دیگر این است که w های لایه آخر که در φ ها ضرب می‌شدند را هم می‌خواهیم محاسبه کنیم.

یک روش ارائه شده که با الگوریتم تکراری متوسط مربع خطا را حداقل کنیم:

$$\Phi(X_i)=\left[\varphi(X_i,\mu_1),\varphi(X_i,\mu_2),\dots,\varphi(X_i,\mu_K)\right]^T$$

$$\varphi(x_i,\mu_j)=\exp\left(-\frac{1}{2\sigma_j^2}\|x_i-\mu_j\|^2\right) \quad j=1,2,\dots,K$$

نمونه‌های آموزشی که برای لایه خروجی به کار می‌گیریم:

$$\{(\Phi(X_i),d_i),\ i=1,2,\dots,L\}$$

$$\{(\Phi(i),d(i)),\ i=1,2,\dots,L\}$$

Scanned with CamScanner

الگوریتم RLS (Recursive least Squares)

$$p(n) = P(n-1) - \frac{P(n-1)\phi(n)\phi^T(n)P(n-1)}{1+\phi^T(n)P(n-1)\phi(n)}$$

با عنونه‌های $(\phi(i), d(i))$ که داریم

این محاسبات را از 1 تا L انجام می‌دهیم:

$$g(n) = P(n)\phi(n)$$

$$\alpha(n) = d(n) - \hat{w}^T(n-1)\phi(n) \rightarrow$$

$\hat{w}(n-1)$ را داریم، $d(n)$ را هم داریم پس α را داریم

$$\hat{w}(n) = \hat{w}(n-1) + g(n)\alpha(n) \rightarrow$$

w ای که به دنبال آن هستیم $\hat{w}(n)$ است که بر حسب مقدار قبلی اش و مقادیر g و α محاسبه می‌شود که مقادیر α و g قابل محاسبه اند

↓

این مرحله‌ای است که برای اصلاح w بکار برده می‌شود

در اولین قدم $\hat{w}(0) = 0$ و $P(0) = \lambda^{-1}I$ که λ یک عدد مثبت کوچک است

الگوریتم K_means, RLS

لایه ورودی: بعد لایه ورودی توسط بعد بردار x مشخص می‌شود که n است

لایه مخفی: ① تعداد نورون‌ها در لایه مخفی برابر تعداد خوشه‌ها که K است، می‌باشد

② میانگین خوشه‌ی i ام که $\hat{\mu}_i$ است را با استفاده از الگوریتم K_means که روی نمونه‌های X_i $i=1,2,\dots,L$ کار می‌کند، به دست می‌آوریم. این $\hat{\mu}_i$ ها مرکز توابع φ هستند.

③ انحراف معیار توابع گاوسی

$$\sigma = \frac{d_{max}}{\sqrt{2K}}$$

برای همه‌ی نورون‌های rbf یکسان است. K تعداد خوشه‌ها یا همان تعداد نورون‌های rbf است و d_{max} بزرگترین فاصله‌ی بین هفت هفت مراکز خوشه‌هاست. پس تا این جا μ_i و σ مشخص شدند و توابع پایه مشخص شدند

لایه خروجی: وزن‌های این لایه، خروجی نورون‌های مخفی است

$$\phi(x_i) = \left[\varphi(x_i,\mu_1), \varphi(x_i,\mu_2), \dots, \varphi(x_i,\mu_K)\right]^T$$

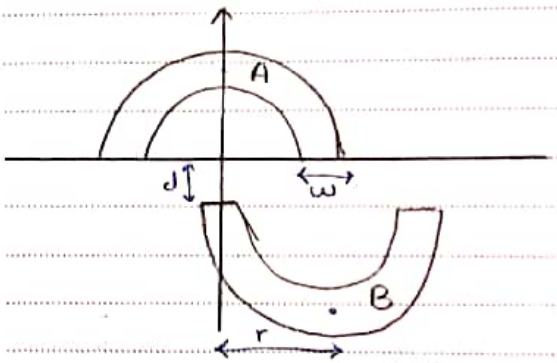
مجموعه‌ی آموزشی که به کار می‌رود $(\phi(x_i), d_i)$ $i=1,2,\dots,L$ است و الگوریتم RLS برای آموزش استفاده می‌شود.

Scanned with CamScanner

یک حُسن این روش این است که محاسباتش خیلی پیچیده نیست و نسبتاً سریع است. خودت که هم ساده است و اینکه دو تا لایه کارشان جدا از هم انجام می‌شود و تداخلی ندارند به سرعت کمک می‌کند.

یک سوالی که مطرح می‌شود: ما در مورد لایه rbf و لایه خروجی به صورت جداگانه تصمیم می‌گیریم و یک بهینه‌سازی در کل شبکه انجام نمی‌شود بلکه در هرکدام از این لایه‌ها سعی می‌کنیم جداگانه به صورت بهینه عمل کنیم. ولی به هرحال جواب‌های خوبی می‌دهد این شبکه، سریع است و خوب کار می‌کند.

مثال: مسئله‌ی دسته‌بندی ۲ دسته‌ای، داده‌های دسته A را از ناحیه A و داده‌های دسته B را از ناحیه B برمی‌دارد.



w ← پهنای نیم حلقه
r ← فاصله‌ی مرکز دو تا نیم حلقه
d ← اینکه حلقه‌ی پایینی چقدر زیر محور افقی است

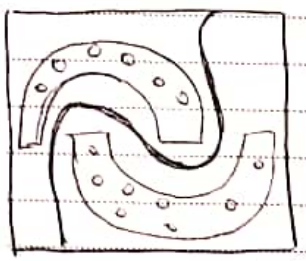
در این مثال: $r=10$ $w=6$
برای d دو حالت را در نظر می‌گیریم: $d=-6$ $d=-5$

تعداد نمونه‌های آموزش: ۱۰۰۰ تعداد نمونه‌های تست: ۲۰۰۰

حالت اول: $d=-5$ معماری این شبکه ← 2_20_1 2 تا ورودی چون داده‌ها ۲ بعدی هستند

۲۰ خروجی برای اینکه ۲ دسته داریم، ۲۰ تا هم نورون rbf در نظر گرفته. واحدهای مخفی را هم گاوسی در نظر می‌گیریم

تعداد خوشه‌ها = $K=20$ پس از اجرای K-means می‌توان σ را حساب کرد که 2.6 بدست آمده



برای یافتن وزن‌های w الگوریتم RLS را به کار می‌برد.

دایره‌ها توابع rbf اند، مرکز آن‌ها مرکز تابع گاوسی را نشان می‌دهد و شعاع هم σ را نشان می‌دهد.

حالت دوم $d=-6$ هم مشابه اول است. سپس همین دسته‌بندی را با شبکه MLP هم آموزش داده و معماری را همان 2,20,1 گرفته. learning rate را از 10^{-1} تا 10^{-5} کاهش داده است

عملکرد rbf کمی بهتر بوده هم از نظر سرعت هم صحت. (مثال از قسمت تابع توسط rbf در اسلایدها آمده)

Scanned with CamScanner

جلسه شانزدهم - ۱۴۰۰/۸/۲۵

شبکه های کانولوشنی (پیچشی) Convolutional NNs

مسائلی که تا این جا مطرح کرده ایم: ① دسته بندی (classification)

② تقریب تابع (function approximation)

③ خوشه بندی (clustering)

مسئله ای که الان می خواهیم بث کنیم: ④ استخراج ویژگی (feature extraction)

استخراج ویژگی اگر به صورت اتوماتیک در یک فرآیند یادگیری صورت بگیرد خیلی ارزشمند است زیرا در گذشته قبل از اینکه بتوانیم استخراج ویژگی را در یک شبکه یاد بگیریم، باید خودمان تصمیم می گرفتیم که چگونه ویژگی استخراج کنیم و ویژگی باید چگونه باشد که برای مسئله ی ما خوب باشد که بث مفصل و مشکلی است شبکه های کانولوشنی برای استخراج ویژگی مورد استفاده قرار می گیرند و می توانند ویژگی های با درجه های تجرید مختلف استخراج کنند. لایه های شبکه ی کانولوشنی یک دسته بندی کننده هم قرار دارد بنابراین در خروجی این شبکه ما شاهد حل مسئله مان خواهیم بود. هم باز استخراج ویژگی و هم باز تصمیم گیری براساس آن ویژگی را دارد.

مزیت شبکه کانولوشنی این است که نسبت به انتقال، مقیاس شدن، و اعوجاج در ورودی مقاوم است. بحث استخراج ویژگی و representation داده ها یکی از هدف ها این است که تا حد ممکن نسبت به یک سری تغییرات غیرحساس باشد. مثلا دو تا تصویر از یک شی که فقط اندازه شی در آن ها متفاوت است یا تصویر چرخیده یا کج شده یا... لازم است که representation یکسانی داشته باشند.

آموزش این شبکه به صورت supervised است.

Scanned with CamScanner

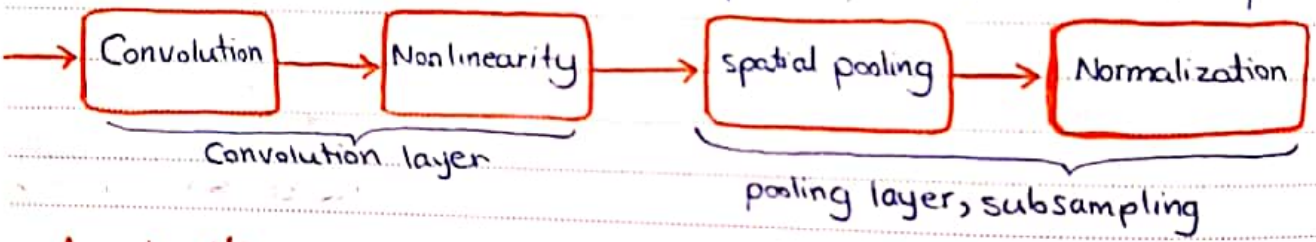
مرداد | ۷ | 29 Jul 2021
پنجشنبه | ۱۸ ذی الحجه ۱۴۴۲

معماری شبکه

معماری شبکه دو بخش اصلی دارد: ① استخراج ویژگی ← شامل N تا مرحله است که ویژگی استخراج می‌کند
② دسته‌بندی ← بعد از بخش استخراج ویژگی وجود دارد، مثلاً می‌تواند یک شبکه MLP باشد. یک شبکه FF ساده مثلاً ۳ یا ۴ لایه.

در بخش استخراج ویژگی که N تا stage داریم وقتی داده‌ی ورودی اعمال می‌شود همین طور که در این مراحل جلو می‌رود، ویژگی‌ای که استخراج می‌کند خردتر می‌شود. وقتی به سمت خروجی می‌رویم درجه‌ی تجرید ویژگی استخراج شده بالاتر می‌رود یا به عبارتی ویژگی استخراج شده سطح بالاتری می‌شود.

هرکدام از stage ها شامل همه یا برخی از واحدهای زیر است:



مرداد | ۸ | جمعه
30 Jul ■ ۱۹ ذی الحجه

ابتدا یک عمل Convolution انجام می‌شود، سپس یک تابع غیرخطی به نتیجه اعمال می‌شود. بعد ادغام مکانی و در آخر Normalization صورت می‌گیرد.

دو مرحله‌ی اول به عنوان یک لایه شناخته می‌شود که به آن لایه‌ی کانولوشن گفته می‌شود. دو مرحله‌ی بعدی به عنوان لایه‌ی ادغام (pooling) یا لایه‌ی subsampling (زیرمجموعه برداری) شناخته می‌شود

stage 1: Convolution layer 1 + pooling layer 2

stage 2: Convolution layer 2 + pooling layer 2

⋮

Stage N: Convolution layer N + pooling layer N

روز بزرگداشت شیخ شهاب الدین سهروردی (شیخ اشراق)

Scanned with CamScanner

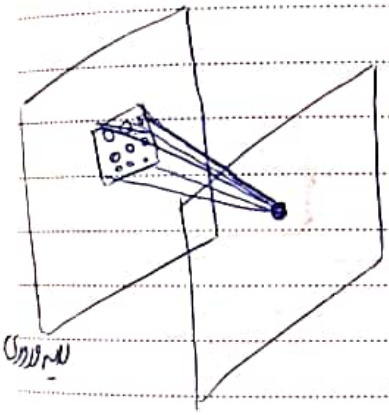
Convolution

در اطراف هر نورون محدوده ای را در نظر می گیرد و در ورودی آن، یک عمل کانولوشن را در آن همسایگی receptive field انجام می دهد. همسایگی ای که برای این نورون اثر گذار است به همین دلیل به این همسایگی می گویند که به معنای محدوده ای است که این نورون می بیند و ورودی های این محدوده روی این نورون اثر می گذارند.

بنابراین در شبکه کانولوشن اینگونه نیست که هر نورون از تمام ورودی ها، ورودی بگیرد. بلکه هر نورون از تعدادی از ورودی ها، ورودی می گیرد. پس در این شبکه اتصالات کامل نیستند بلکه تعدادشان خیلی کمتر از اتصالات کامل است.

$$I_j^{(l)} = \sum_{i \in \text{receptive field}} w_{ij}^{(l)} x_i^{(l-1)} + b_j^{(l)}$$

کانولوشن برای یک نورون



وزن های اتصالاتی که برای هر نورون داریم با سایر نورون ها مشابه است.

weight sharing

برای مثال عاتریس وزن همه ی نورون ها در شکل بالا، ۳×۳ و یکسان خواهد بود. به این ویژگی گفته می شود. بنابراین اگر یک مجموعه وزن داشته باشیم برای تمام نورون های صفحه می توانیم I را حساب کنیم. می توان به جای اینکه یکی از این مجموعه وزن ها را داشته باشیم یک تعدادی از این مجموعه وزن ها را داشته باشیم در اثر به کارگیری هر مجموعه یک صفحه ویژگی یافت می شود. مثل این است که ویژگی های مختلفی از ورودی استخراج می شود. این عمل که با یک مجموعه از این وزن ها انجام می دهیم معادل این است که یک فیلتر به ورودی اعمال شود و وزن هایی که استفاده می شد در واقع همستی فیلتر را تشکیل می دهند.

مثلا اگر ورودی تصویر باشد، یک فیلتر می تواند برای تشخیص لبه های افقی اشیاء ایجاد شود. یک فیلتر برای

Scanned with CamScanner

لبه های عمودی، یکی برای لبه‌های با زاویه ۴۵ درجه و ...

مطلب دیگر اینکه خود ورودی می تواند بیشتر از یک ورودی باشد. مثلا اگر تصویر رنگی داشته باشیم ۳ تا کانال دارد ← R, G, B و این عملیات برای هر کانال جداگانه اعمال می شود و تعداد صفحات ویژگی که استخراج می شود ۳ برابر می شود.

فرض کنیم فیلتری که استفاده می کنیم F×F باشد ← انتخاب آن با خود طراح شبکه است

موردی که باید مشخص کنیم این است که فیلتر با receptive field روی ورودی چگونه حرکت کند. یعنی وقتی الان یک receptive field داریم، بعدی چگونه تعیین شود. ممکن است بعدی را بین ۱ تا F خانه جلو ببریم و قرار بدهیم. وقتی فقط ۱ خانه جلو ببریم هم پوشانی زیادی داریم و وقتی F تا جلو ببریم هم پوشانی نداریم اینکه چند قدم ببریم و بعدی را قرار بدهیم stride نام دارد که می تواند ۱، ۲، ۳، ... و F باشد. (اندازه قدم)

وقتی میدان های دریافت هم پوشانی داشته باشند نتایج بهتری حاصل می شود.

هرچقدر stride بزرگتر باشد مقدار کمتری فاصله نیاز است و در نتیجه در لایه‌ی کانولوشن مقدار کمتری گره یا نورون خواهیم داشت. (و برعکس) stride کوچکتر به معنای محاسبات بیشتر و نقشه های ویژگی بزرگتر است.

وزن های فیلترها با آموزش supervised یادگرفته می شود. قانون آموزش آن هم BP است.

Non Linearity

روی نتیجه‌ی کانولوشن یک تابع غیرخطی اعمال می کنیم. این تابع می تواند Sigmoid، Tanh، ReLU و ... باشد. خوبی که ReLU نسبت به بقیه دارد این است که آموزش و انتشار خطا به عقب را آسانتر می کند، یادگیری با آن سریعتر صورت می گیرد و توابع دیگر که به اشباع می روند مشکلاتی را ایجاد می کنند که این تابع ایجاد نمی کند.

خروجی لایه کانولوشن ← $x_j^{(l)} = f(I_j^{(l)})$

(مثال در اسلاید)

Scanned with CamScanner

Spatial Pooling

گفتیم که اگر ورودی t تا کانال داشته باشد و k تا صفحه ویژگی بخواهیم استخراج کنیم، t x k تا صفحه خواهیم داشت و تعداد ورودی های لایه ی بعدی می شود t x k تصویر و همین طور تعداد این ها در هر لایه زیاد می شود. بنابراین باید از یک راهی این مورد را کنترل کنیم که یکی از وظایف لایه ی ادغام مکانی همین است که حجم این داده ها را کم کند و آن ها را فشرده کند.

لایه ادغام یک ناحیه را می گیرد و به جای مقادیر آن ها یک مقدار جایگزین می کند. این مقدار مثلا می تواند max مقادیر آن ناحیه باشد یا مثلا مجموع یا میانگین آن ها.

به عنوان مثال اگر ناحیه ای که در نظر می گیریم ۲x۲ باشد، هر سلول ۲x۲ در خروجی با ۱ عدد جایگزین می شود. پس هر بعد داده تقسیم بر ۲ می شود و در کل داده ها $\frac{1}{4}$ می شود برای هر کدام از صفحات.

معمولا این ناحیه دارد بدون اشتراک کامل، از ناحیه ی قبلی ورودی می گیرد یعنی یک receptive field دارد که همان نورون هایی هستند که از لایه ی قبلی در نظر گرفته می شوند (مثلا ۲x۲)

در این جا هم میدان های دریافت می توانند هم پوشانی داشته باشند یا نداشته باشند. (stride)

معمولا receptive field را ۲x۲ و stride را ۲ می گیرند که هم پوشانی نداشته باشند.

کاری دیگری که pooling انجام می دهد و به جایی های کوچک غیر حساس می شود چون برای یک ناحیه max یا مجموع و ... را در نظر می گیرد.

نکته دیگر اینکه چون مثلا یک ناحیه ۲x۲ رو در نظر می گیره، در حقیقت مثل این است که میدان های دریافت بزرگتری را در نظر گیرد و ناحیه ی بزرگتری از ورودی ها دارند روی گسوای که در لایه ی pooling قرار دارد اثر می گذارند.

(مثال از max pooling در اسلاید)

Scanned with CamScanner

مبنق max pooling ← در واقع جاهایی که فیلتر با پیکسل قوی تری رو می بینه برای ما مهم است و این جاها همان max و عددهای بزرگتر هستند. این نقاط هستند که برای استخراج ویژگی مفیدند.

Normalization

برای کنترل مقادیر و اینکه contrast بالا برود یعنی تمایزها بیشتر بشود. یعنی داده های خروجی pooling، بگونه ای باشد که خیلی مقادیر متفاوتی پیدا کنند و در بین نتایجی که حاصل شده یک تمایز خوب بتوانیم ایجاد بکنیم.

نکته: اگر ورودی N×N باشد و فیلتر F×F و stride داشته باشیم اندازه خروجی به این صورت محاسبه می شود:

$$(N - F) / stride + 1$$

مثال: $N=7, F=3, stride=1$ ⟵ output size $= \frac{7-3}{1} + 1 = 5$

$N=7, F=3, stride=2$ ⟵ $= \frac{}{} = 3$

$N=7, F=3, stride=3$ ⟵ $= = 2.33$ ↘

این حالت را نباید انتخاب کنیم

نکته: اگر تعداد مراحل در شبکه کانولوشنی زیاد باشد چه اتفاقی می افتد؟ مثلا اگر pooling ۲×۲ داریم در مرحله اول اندازه ورودی تقسیم بر ۲ می شود، در مرحله دوم دوباره تقسیم بر ۲ می شود و $\frac{1}{4}$ می شود و مرحله سوم $\frac{1}{8}$ می شود... در حالی که می خواهیم حجم داده ها را کنترل کنیم در این حال می خواهیم داده ها نابود نشوند.

یک راه این است که در برخی مراحل pooling نداشته باشیم. یعنی ۲ تا لایه کانولوشن و یک pooling و... یک راه دیگر: چون علاوه بر pooling خود کانولوشن هم داده ها را دچار تغییر می کند، بیاییم به ورودی سطرها و ستون های صفر اضافه کنیم به طوری که اندازه ی خروجی کانولوشن برابر اندازه ی ورودی این باشد. به آن zero padding می گویند.

Scanned with CamScanner

۱۴۰۰

zero padding کمک می‌کند که مرکز receptive field روی تمام نقاط داده‌ی ورودی قرار بگیرد. به‌عنوان مثال اگر ورودی 7×7 و فیلتر 3×3، stride=1 باشد، یک padding با اندازه‌ی مناسب است. یعنی یک سطر و یک ستون صفر اضافه شود.

F=5 → zero pad = 2
F=7 ⇒ zero pad = 3

تعداد سطر و ستونی که نیاز داریم $\frac{F-1}{2}$ است.

فرمول‌های کلی‌تر:

ورودی: $W_1 \times H_1 \times D_1$ ← تعداد کانال‌ها

K: تعداد فیلترها — اندازه فیلترها: F×F

stride = S — تعداد سطر یا ستون صفر = P

اگر خروجی لایه کانولوشن $W_2 \times H_2 \times D_2$ باشد:

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$$D_2 = K$$

$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

(مثال در اسلاید)

آموزش شبکه:

با الگوریتم BP. این شبکه به مجموعه داده بزرگ نیاز دارد به‌خصوص اگر تعداد لایه‌ها زیاد باشد.

نتیجه این است که آموزش این شبکه بسیار کند است.

بعضی از مسائل مشکل ما این است که این حجم داده را اصلا نمی‌توانیم داشته باشیم (لیبل هم باید داشته باشد) یک راه برای بالا بردن تعداد داده‌ها این است که همین داده‌های موجود را تغییر بدهیم و به‌عنوان داده جدید اضافه کنیم. مثلا برای تصویر: چرخش یا زاویه‌های مختلف.

یک کاری که انجام شده این است که ابزارهایی که به سیستم‌های اصلی حوزه دسترسی داشته‌اند شبکه‌هایی را آموزش داده‌اند و در اختیار عموم گذاشته‌اند. حتی برای مجموعه داده‌های متفاوت هم تا حدی می‌تواند کمک کند.

Scanned with CamScanner

میتوانیم از شبکه‌های پیش آموزش دیده (pre-trained Network) استفاده کنیم. که عمدتا آموزش دیده‌اند، این‌ها را با داده‌های خودمون دوباره آموزش (re-training) می‌دهیم و به مقداری برای داده‌های ما آموزششان بهتر می‌شود که این کار یک عنوان Transfer learning یادی شود. که به این معناست که شبکه‌ای با یک مجموعه داده آموزش دیده و حالا در مسئله‌ی دیگری با مجموعه داده دیگری مورد استفاده قرار می‌گیرد.

جلسه هفدهم ـ ۱۴۰۰/۸/۳۰

تابع زیان در این شبکه، تابع cross entropy است:

$$J(\theta)=\sum_i y^{(i)} \log(\hat{y}^{(i)})$$

مثلا دوست دارم روی ارقام دست‌نویس فارسی کار کنم بهتر است از شبکه‌ای که با MNIST پیش آموزش دیده کار کنم یا مثلا برای تشخیص مکان‌ها خوب است با شبکه‌ای که با دیتاست places آموزش دیده کار کنم.

روز خانواده و تکریم بازنشستگان

مثال: شناسایی حروف دست‌نویس

لایه ورودی ما لایه‌ای است که متوجه یک حرف به آن داده می‌شود

یک ماتریس ۲۸×۲۸ است، تصویر حرف در وسط تصویر است و از نظر اندازه هم حروف نرمالیزه شده‌اند

از ۶ تا فیلتر برای استخراج ویژگی استفاده می‌کنیم، هر فیلتر ویژگی خاصی را استخراج می‌کند.

ابعاد خروجی فیلترها ۲۴×۲۴ است زیرا receptive Field = 5×5

به اندازه‌ی $\frac{F-1}{2}$ از ابعاد تصویر کم می‌شود

در مرحله‌ی اول که استخراج ویژگی انجام می‌شود یک pooling صورت می‌گیرد که ۲×۲ است. در نتیجه ۶ تا نقشه‌ی ۲۴×۲۴ تبدیل به ۶ تا نقشه‌ی ۱۲×۱۲ می‌شود.

Scanned with CamScanner

در مرحله‌ی بعد کانولوشن ۵×۵ صورت می‌گیرد که باعث می‌شود ۴ تا از بعد کم بشود.
۱۲ تا نقشه‌ی ۸×۸ خواهیم داشت، بعد از pooling، ۱۲ تا نقشه‌ی ۴×۴ خواهیم داشت.
بعد از این مرحله، این ۱۲ تا نقشه را به عنوان ورودی به لایه‌ی out put ی دهیم. خود این کانولوشن صورت می‌گیرد که ۴×۴ است.
۲۶ تا نقشه‌ی ۱×۱ ای خواهیم خروجی output باشد. ۲۶ تا عدد که اولین عدد احتمال اینکه ورودی A باشد، دومی احتمال اینکه B باشد و... ۲۶ ای احتمال اینکه Z باشد را نشان می‌دهد.
این شبکه حدود ۱۰۰ هزار تا احتمال دارد. به دلیل وجود اشتراک گذاری وزن ها، این اتصالات به ۲۶۰۰ کاهش پیدا می‌کند.

popular CNNs

① LeNet: اولین پیاده سازی از CNN را انجام داد. برای کاربرد خواندن ارقام و Zipcode و... استفاده شد.

② AlexNet: توسط یک تیم ۳ نفره معرفی شد. در سال ۲۰۱۲ در یک مسابقه این شبکه را شرکت دادند و مشخص شد شبکه شان به مراتب بهتر از بقیه است. همان LeNet بود که تغییراتی در آن داده بودند

③ ZF Net: یک نسخه دیگر از CNN ارائه دادند که هیپرپارامترهای در AlexNet داده بودند.

④ GoogleNet: در ۲۰۱۴ این شبکه برنده شد که توسط فردی که در گوگل کار می‌کرد معرفی شد. از ماژول‌هایی به نام Inception استفاده کرد که تعداد پارامترها را کاهش می‌داد. مثلا این شبکه

⑤ VGGNet: در همان سال 2014 این شبکه دوم شد. در دانشگاه oxford این شبکه توسعه داده شده بود. کاری که کردند این بود که عمق را زیاد کردند و نشان دادند که افزایش عمق تاثیر زیادی دارد علاوه بر این، با اینکه این شبکه به GoogleNet باخت اما نشان داده شد که این شبکه ویژگی های بهتری

Scanned with CamScanner

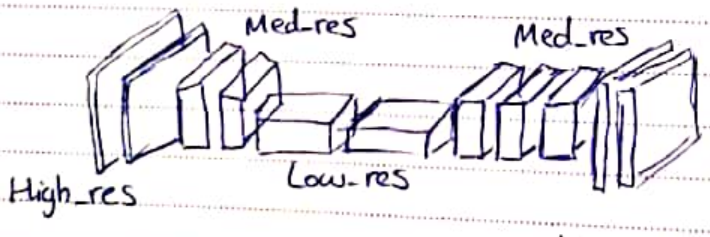
را استخراج می کند به همین دلیل در Transfer learning خیلی مفید است و هنوز هم افراد ترجیح می دهند که از VGG استفاده کنند. هزینه ای که این شبکه دارد تعداد پارامترهای زیاد آن است.

دو از سخه های اولیه، شروع به استفاده از CNN شد و سخه های جدیدی هم ارائه شد. مثلا یک سخه R_CNN (Region CNN) است که در واقع روی نواحی تصویر کار می کند و مثلا برای یافتن شرح تصویر به کار گرفته شده. کاربردهای segmentation و تقطیع تصویر، object detection، عنوان نویسی برای تصاویر

Semantic Segmentation : FCNs

ابتدا ورودی را کاهش می دهد بعد دوباره افزایش می دهد. FCN = Fully Connected

برای Semantic Segmentation هم استفاده شده که پیکسل های مربوط به هر شی را با ناحیه از تصویر را به یک رنگ خاص در می آورد.



تشخیص های خودران از Segmentation استفاده می شود

علاوه بر Semantic Segmentation علاقه خواهیم عمق را مشخص کنیم (تخمین بزنیم) یا علاقه خواهیم جدا هایی از یک شی را که در تصویر وجود دارد جداگانه تشخیص بدهیم که به آن Instant Segmentation می گوییم

در Image Captioning یک شبکه CNN تصویر را تحلیل می کند و نتیجه را به یک RNN می دهد که کپشن را تولید می کند

در کاربردهای پزشکی هم استفاده می شود. مثلا از روی تصویر چهره احتمالات را تشخیص می دهند. خروجی شبکه احتمال ابتلای فرد به بیماری های مختلف را نشان می دهد.

روز خبرنگار

Scanned with CamScanner

Convolution over volumes

یک عکس یا تصویر RGB سه تا کانال دارد. اگر بخواهیم کانولوشن انجام بدهیم باید فیلتر را روی هرکدام از کانال‌ها قرار بدهیم و عمل کانولوشن را انجام بدهیم.

یک راه دیگر این است که تعداد کانال‌ها را به عنوان مولفه سوم تصویر در نظر بگیریم و تصویر حجم داشته باشد فیلترها را هم می‌توانیم به صورت حجم در نظر بگیریم که مولفه‌ی سوم آن همان تعداد کانال‌ها است مثلا یک فیلتر ۳×۳ برای ۳ کانال می‌شود یک حجم ۳×۳×۳

حجم فیلتر در داخل حجم تصویر قرار می‌گیرد و حرکت می‌کند و کانولوشن محاسبه می‌شود.

اگر بیشتر از یک فیلتر داشته باشیم، هرکدام از فیلترها و حتی با تصویر convolve می‌شوند یک خروجی در خروجی ایجاد می‌کنند. ابعاد صفحات خروجی را می‌توان به صورت حجم در نظر گرفت که مثلا بعد سوم آن ۲ است

فرض کنید ورودی $n \times n$ و تعداد کانال‌ها n_c باشد:

ورودی : $n \times n \times n_c$

فیلتر $f \times f$ است:

فیلتر : $f \times f \times n_c$

ابعاد صفحات خروجی $n-f+1$ است:

خروجی : $(n-f+1) \times (n-f+1) \times n_c'$

↓ این جا اضافه کردن ۰ نداریم

n_c' ← تعداد کانال در خروجی = تعداد فیلترها

جلسه هجدهم - ۹/۲، ۱۴۰۰

وقتی کانولوشن ۱×۱ را با عمق تعداد کانال‌ها استفاده کنیم تصویر حاصل انگار خلاصه‌ای از همه‌ی کانال‌ها را در خودش دارد. انگار اطلاعات همه‌ی کانال‌ها را با هم ترکیب کرده و عصاره‌ای از همه‌ی آن‌ها است.

وقتی کانولوشن ۱×۱ را حرکت می‌دهیم در واقع ترکیب وزن‌دار پیکسل‌ها در کانال‌های مختلف محاسبه می‌شود و در خروجی ظاهر می‌شود

Scanned with CamScanner

لایه‌ی سوم فیلتر ۱×۱ در این با تعداد کانال‌ها هماست
تعداد خروجی‌ها در خروجی ← تعداد فیلترها

اگر بخواهیم از یک فیلتر ۱×۱ با عمق تعداد کانال‌ها داشته باشیم، صفحات خروجی هم بیش از ۱ می‌شود

تعداد فیلترها، صفحه‌ی خروجی داریم.

← همان دستورد اطلاعات است، اطلاعات ۱۹۲ کانال در ۳۲ کانال خلاصه می‌شود

حجم بیس برای کاهش تعداد کانال‌ها می‌توان از کانولوشن ۱×۱ استفاده کرد. به عنوان مثال تصویر ۲۸×۲۸

با ۱۹۲ کانال داریم: ۲۸ × ۲۸ × ۱۹۲ ← می‌خواهیم تعداد کانال‌ها را به ۳۲ کاهش دهیم

بین از ۳۲ تا فیلتر ۱×۱ استفاده می‌کنیم: ۱۹۲ × ۱×۱ ← خروجی: ۲۸ × ۲۸ × ۳۲

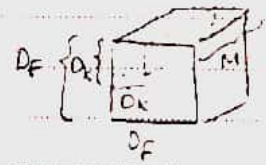
کانولوشن عمق جدایی‌پذیر

تصویر: $D_F \times D_F \times M$

فیلتر: $D_K \times D_K \times M$ $\xrightarrow[N \text{ filters}]{\text{Conv}}$ خروجی: $D_G \times D_G \times N$

D_G از روی D_F و D_K محاسبه می‌شود

محاسبه‌ی تعداد ضرب‌ها:
در حالت استاندارد



$$\text{Mults ones} = D_K^2 \times M$$

$$\text{Mults per kernel} = D_G^2 \times D_K^2 \times M$$

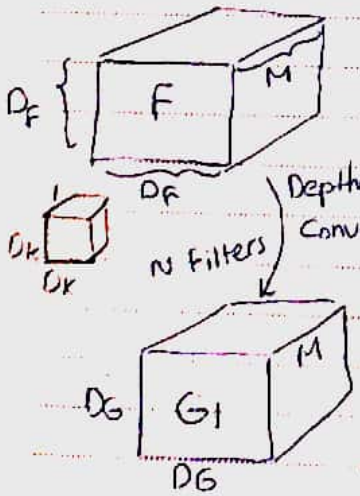
$$\text{Mults N kernel} = N \times D_G^2 \times D_K^2 \times M$$

کانولوشن جدایی‌پذیر عمق:
در این حالت فیلتر را $D_K \times D_K \times 1$ در نظر می‌گیریم و کانولوشن را ابتدا

برای این فیلتر انجام می‌دهیم. تعداد دفعاتی که فیلتر در آن می‌گذرد

$D_G \times D_G$ است و یک صفحه $D_G \times D_G$ در خروجی نوشته می‌شود

اگر M تا فیلتر داشته باشیم M تا از این صفحات در خروجی داریم

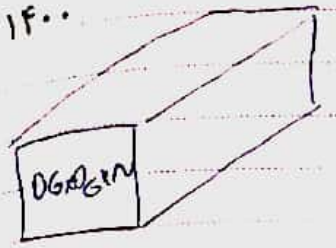


در ادامه‌ی این کار یک کانولوشن ۱×۱ عمق هم انجام می‌دهیم.

به این صورت که G_1 را که نتیجه‌ی مرحله‌ی قبل بود با فیلترهای $1 \times 1 \times M$ کانوالو می‌کنیم. در اینجا

Scanned with CamScanner

۱۴۰۰



بارای هم این کار، یک صفحه‌ی $D_G \times D_G$ خواهیم داشت و کلا

N تا از این فیلترها داشتیم پس می‌شود $D_G \times D_G \times N$

حالا نتیجه‌ی قبلی به دست می‌آید. با این تفاوت که این جا ۲ مرحله داریم. یک بار کار اعمال فیلترر تک تک انجام می‌شه و بعد کانال‌ها با کانولوشن ۱×۱ ترکیب می‌شوند

محاسبه‌ی تعداد ضرب‌ها در این حالت:

$$\text{Mults ones} = D_K^2$$

برای یک کانال ←

$$\text{Mults 1 channel} = D_G^2 \times D_K^2$$

تعداد ضرب‌های مرحله‌ی اول ← برای M کانال ←

$$\text{DC Mults} = M \times D_G^2 \times D_K^2$$

مرحله دوم:

$$\text{Mults once} = M$$

و تمام این فیلترها در M تا ضرب داریم

$$\text{Mults 1 kernel} = D_G \times D_G \times M$$

$$\text{PC Mults} = N \times D_G \times D_G \times M$$

$$\text{تعداد کل ضرب} = \text{DC Mults} + \text{PC Mults} = M \times D_G^2 \times D_K^2 + N \times D_G^2 \times M$$

مقایسه حالت جدایی پذیر شده با حالت استاندارد:

$$\frac{M \times D_G^2 (D_K^2 + N)}{N \times D_G \times D_G \times D_K \times D_K \times M} = \frac{D_K^2 + N}{D_K^2 \times N} = \frac{1}{N} + \frac{1}{D_K^2}$$

مثلا اگر $D_K = 3$ و $N = 1024$ ← 0.112

تعداد ضرب‌های این روش حدود یک دهم تعداد ضرب‌های روش استانداردی شود (در این مثال)

افزایش سرعت قابل توجهی به دست می‌آید

Scanned with CamScanner

مرداد | ۲۱ | 12 Aug 2021
پنجشنبه | ۳ محرم ۱۴۴۳

مقایسه‌ی روند پارامترها

param 1 kernel = $D_k^2 \times M$

param N kernels = $N \times M \times D_k^2$

دسته‌بندی می‌شود که مقدار ضرب‌ها بود

یکی از کاربردهای شبکه‌های کانولوشن در شبکه‌های Multi Modal است. یعنی شبکه‌هایی که ورودی و خروجی‌شان نوعاً مختلف باشند.

مثلاً ورودی تصویر باشد صوت هم باشد.

روز حمایت از صنایع کوچک

مرداد | ۲۲ | 13 Aug
جمعه | ۴ محرم

Xception

MobileNet ← همان‌طور که گفتیم شبکه کانولوشن را کاهش حجم می‌دهند تا در موبایل به‌کار گرفته شود. عملکرد دقت پیدا می‌کند اما حجم را کاهش می‌دهند که روی موبایل بتوانند کار کند.

underfitting ← در مسئله‌ی رگرسیون یا کلاسیون یعنی با این وضعیت روبه‌رو شویم که مدل تعمیم شایسته‌ای طوری که باید منعطف نیست high bias

overfitting ← تغییرات مدل خیلی زیاد است، داده‌ها را خوب جدا کرده ولی تغییراتش خیلی بالاست نویز روی مدل تعمیم خیلی اثر دارد high variance

Scanned with CamScanner

Bias/Variance

high variance ← overfitting

high bias ← underfitting

برای ما هیچ کدام از این دو حالت مطلوب نیست

Training Set error	1 %	15 %	15 %	0.5 %
Testing Set error	11 %	16 %	30 %	1 %
	overfitting واریانس بالا	underfitting بایاس بالا (خطا بالاست)	بایاس بالا واریانس بالا	بایاس پایین واریانس پایین

مطلوب /

چگونه می‌توان در یک حالت هم بایاس بالا هم واریانس بالا باشد؟

در طراحی شبکه هم موقعی که داریم آموزش می‌دهیم باید این سوال را بپرسیم

براساس خطای داده‌های آموزش مشخص می‌شود که بایاس بالا هست یا نه و بعد براساس خطای داده‌های تست آیا واریانس بالا هست یا نه؟ اگر نیست حزبه.

اگر بایاس بالاست / اگر بایاس بالا نیست

برای پایین آوردن بایاس می‌توان شبکه را بزرگتر کرد یا مدت طولانی‌تری آموزش را ادامه دهید و یا اینکه معماری شبکه را عوض کنید.

اگر واریانس بالا باشد توصیه می‌شود ① سعی کنید داده‌های بیشتری به کار ببرید

② Regularization ⑭ معماری شبکه را تغییر دهید

مسئله‌ی Memorization ، Generalization را در شبکه‌های FF بررسی کردیم زمانی که شبکه کم‌عمق بود این مسئله را داشتیم. این مسئله در شبکه‌های عمیق حل شده و ما می‌توانیم حالتی را داشته باشیم که در عین اینکه در آموزش خطا پایین، خطای تست هم پایین بماند. این یکی از دلایل موفقیت شبکه‌های عمیق است.

روز مقاومت اسلامی

Scanned with CamScanner

Regularization (تنظیم)

تغییری در تابع هدف می دهیم و عملکرد روش مان تغییر می کند.

تعریف Regularization: هر تغییری که در الگوریتم یادگیری می دهیم به طوری که خطای تعمیم پذیری را بدون ایجاد بایاس
پایین بیاوریم ولی خطای آموزش تأثیر چندانی نگذارد.
Generalization یا خطای روی داده تست

عملی وقت خواندن که پیدای کنیم یک مدل بزرگ هست که توانسته خطای آموزش را پایین بیاورد
به طور عملی Regularization روی آن انجام شده است.

روش های Regularization:

کاری که در Regularization انجام می شود به این شکل است:

$$\tilde{J}(\theta; X, y) = J(\theta; X, y) + \alpha \Omega(\theta)$$

یک جمله ای با تابع هدف جمع می شود
پارامترها ورودی

که این جمله معمولا یک ضریب ثابت هست و میدد پارامترهای مسئله (تابعی از پارامترها)

α که معمولا به عدد ثابت وزن جمله ی Regularization را نسبت به تابع بهینه سازی اصلی کنترل می کند

وقتی α را min می کنیم هم min ی J و هم $\Omega(\theta)$ ـ به نوعی پارامترها را کنترل می کند. معمولا داخل ـ وزن ها را
نقطای گذاریم و بایاس ها را این گذاریم به اشتباه دارد

L^2 Regularization (weigh decay)

$$\Omega(\theta) = \frac{1}{2}\|w\|_2^2 \qquad \tilde{J}(w; X, y) = \frac{\alpha}{2} w^T w + J(w; X, y)$$

نرم 2 وزن ها در $\tilde{J}$ قرار می گیرد

$$\nabla_w \tilde{J}(w; X, y) = \alpha w + \nabla_w J(w; X, y)$$

$$w \leftarrow w - \epsilon(\alpha w + \nabla_w J(w; X, y)) \rightarrow w \leftarrow (1 - \epsilon\alpha) w - \epsilon \nabla_w J(w; X, y)$$

Scanned with CamScanner

L1 Regularization

$$\Omega(\theta) = \|w\|_1 = \sum_i |w_i|$$

از نرم L1 وزن‌ها را استفاده می‌کنیم

$$\tilde{J}(w; X, y) = \alpha \|w\|_1 + J(w; X, y)$$

$$\nabla_w \tilde{J}(w; X, y) = \alpha \text{sign}(w) + \nabla_w J(X, y; w)$$

منظور از sign، element wise به آن اعمال می‌شود

وقتی از L1 استفاده می‌کنیم به سری وزن‌ها در نتیجه اجبار صفر شده‌اند. پاسخ به دست آمده اسپارس است به خصوص اگر α بزرگ باشد

مقدار زیادی صفر

وقتی یک سری وزن‌ها صفر باشند یعنی به سری اتصالات نیاز نیست، یعنی ورودی‌های مربوط به این اتصالات هم می‌توانند نباشند. یعنی آن ورودی‌هایی که صفر شده‌اند خیلی کافی هستند و بیزن‌ها خیلی کمک‌کننده‌اند و این کار انتخاب ویژگی Feature Selection انجام می‌دهد

Data Augmentation

برای آموزش شبکه‌های زیاد نیاز داریم. همیشه داده‌ها آن‌قدر که فکر می‌کنیم زیاد نیستند یا به دلیل فنی یا اقتصادی هم نمی‌توانیم داده‌ها را زیاد کنیم و مسائل خیلی هزینه‌بر می‌شوند.

توصیه می‌شود از داده‌هایی که دارید داده‌های جدیدی تولید کنید و به کار ببرید. مثلاً در تشخیص اشیاء یا در گفتار. (روش‌های تولید داده‌های مصنوعی) مثلاً در گفتار داده‌ها را کش بدهید یا در تصویر با مثلاً چرخاندن تصاویر یا scale کردن

در مسئله‌ای مثل تشخیص دست‌خط این کار را می‌توانیم انجام دهیم. در این موارد یک راه این است که یک نویز مناسب به داده‌ها اضافه کنیم و نویز را خود داده در نظر بگیریم اگر خوب اعمال شده باشد.

Scanned with CamScanner

نکته: می توانیم چند نوع Regularization را باهم به کار بریم

۲۶ | مرداد | سه‌شنبه | 17 Aug 2021 | ۸ محرم ۱۴۴۳

early Stopping

یعنی خطاهای training set و Validation Set را همین طور که جلو می‌رویم کنترل کنیم و [illegible] داریم تا ببینیم که هر دو دارن کاهش پیدا می‌کنن اما از یه جایی به بعد خطای ولیدیشن شروع به افزایش می‌کنه. همین جا آموزش رو متوقف می‌کنیم

early stopping هم نوعی Regularization است. در الگوریتم Regularization به جز early stopping برای یک مدت چک می‌کنه که خطای Validation افزایش پیدا نکنه. اگر برای مدت مشخصی افزایش داشت آموزش را متوقف می‌کند.

برای early stopping نیاز به validation set داریم یعنی کسری از داده‌ها را شبکه ندیده و از آن‌ها در بین یادگیری شبکه استفاده نشده.

یک راه این است که پس از پایان آموزش، آموزش را با داده‌های Validation ادامه بدهیم. تا شبکه داده‌هایی که ندیده بود را ببیند. می‌توانیم به این شکل انجام بدهیم:

شبکه را با دوباره از اول [illegible] با همه‌ی داده‌ها آموزش بده با همان تعدادی که early stopping تعیین کرده بود.

یک راه دیگه: دوباره از اول آموزش ندهیم بلکه آموزش را ادامه بدهیم و فقط این داده‌های جدید را به شبکه بدهیم و این آموزش را تا وقتی ادامه بدهیم که مقدار خطا به زیر مقدار متوسط خطا در داده‌ی ولیدیشن برسد

Scanned with CamScanner

Dropout (حذف تصادفی)

برای هر نود یک مسئله پرتاب می کنیم که آن را نگه داریم یا نه! (در این بخش از آموزش (در آموزش))

هر نودی که می آید به صورت تصادفی تصمیم می گیریم یک بخش هایی از شبکه در آموزش استفاده نشود

می تواند برای همه یا برخی از لایه ها انجام شود. احتمال حذف برای لایه های مختلف می تواند متفاوت باشد.

مشابه عملکرد L2 Regularization را دارد. تردیک به آن ها است. کار آسانی است و هزینه کمی دارد

Dropout در شبکه های دیگر مثل بولتزمن، RNN، FF می تواند به کار برود.

از طرفی روش ارزانی هست و فاکتورات سنگین ندارد.

مثال از Dropout در شبکه CNN:

$$a_j^{(l+1)} = pool(a_1^{(l)}, \dots, a_i^{(l)}, \dots, a_n^{(l)}) \quad i \in R_j^{(l)}$$

اتفاقی که می خواهیم روی آن ها pooling انجام بدهیم را در یک $m^{(l)}$ ضرب می کنیم و اسمش را $\hat{a}$ می گذاریم و بعد pooling را روی آن انجام می دهیم.

m^l در واقع یک ماسک باینری که هر عضوش از یک توزیع برنولی گرفته شده، یعنی به صورت تصادفی یک سری از این مقادیر صفر می شوند و چون max pooling انجام می دهیم حذف می شوند و انتخاب می شوند

$$\hat{a}^{(l)} \sim m^{(l)} * a^{(l)}$$

$$a_j^{(l+1)} = pool(\hat{a}_1^{(l)}, \dots, \hat{a}_i^{(l)}, \dots, \hat{a}_n^{(l)}) \quad i \in R_j^{(l)}$$

در لایه کانولوشن هم Dropout به همین صورت با ماسک باینری انجام می شود

$$m_k^{(l)}(i) \sim Bernoulli(p) \qquad z_j^{(l+1)} = \sum conv(w_j^{(l+1)}, \hat{a}_k^{(l)})$$

$$\hat{a}_k^{(l)} = a_k^{(l)} * m_k^{(l)} \qquad a_j^{(l+1)} = f(z_j^{(l+1)})$$

در فارسی ← در هر دور بولتزمن دکانولوشن

$$a_j^{(l+1)} = p \times \max(a_1^{(l)} \dots a_i^{(l)} \dots a_n^{(l)})$$

p ← احتمال اینکه خروجی واحد فعال بماند

جلسه نوزدهم - ۱۴۰۰/۹/۷

صحبت در مورد شبکه های کانولوشنی را به صورت غیر رسمی ادامه می دهیم. در حوزه این شبکه توسعه هایی داشته ایم و کارهایی در جهت بهبود آن ها انجام شده است. از جمله آن ها: شبکه ی باقی مانده ای

Deep Residual Neural Networks (ResNets)

در شبکه ی کانولوشنی اگر تعداد لایه ها خیلی زیاد شود و شبکه خیلی عمیق شود، آموزش آن دچار مشکل می شود. شبکه ResNet برای حل این مشکل معرفی شد.

عملکردی که این شبکه در کاربردهای مختلف مثل دسته بندی تصاویر، تشخیص اشیاء، تقطیع موضوعی تصاویر از خودشان داده، بهترین بوده است.

ایده ای که پیشنهاد دهنده های این شبکه داشتند این بود که با افزایش تعداد لایه ها قدرت توصیف و ارائه ای که شبکه پیدا می کند قوی تر می شود و هم چنین قدرت تعمیم هم بالاتر می رود.

سوالی که در این جا مطرح شد این بود که تا چند لایه اگر افزایش بدهیم نتایج بهتر می شود؟ آیا حدی دارد؟

جواب این بود که بله حدی دارد و از یک جایی به بعد عملکرد افت می کند. شبکه ای که خودشان در مسابقات ارائه دادند ۱۵۲ لایه داشت.

منظورمان از لایه، لایه های کانولوشنی است یعنی batch Normalization، pooling و ... را نشمردیم.

vanishing / exploding gradient problems

مسئله ای که گفتیم در شبکه کانولوشنی وجود دارد و در شبکه ResNet حل شده این است: وقتی شبکه FF

مرداد | ۲۹ | جمعه
20 Aug ■ ۱۱ محرم

Scanned with CamScanner

۱۴۰۰

با لایه های خیلی زیاد داریم، آموزش این شبکه ها با روش BP صورت می گیرد که از گرادیان استفاده می کند، در این حالت به دلیل وجود تعداد لایه های زیاد آموزش شبکه دچار مشکل می شود.

مشکلی که ایجاد می شود بیشتر به این شکل است:

لایه های ابتدایی یعنی لایه هایی که به ورودی نزدیک تر هستند، آموزششان خیلی کندی می شود. خیلی دیر یاد می گیرند یا به عبارتی تغییرات وزن هایشان خیلی کندی می شود.

دلیل این مسئله این است:

همان طور که قبلا گفتیم اصلاح وزن ها متناسب با گرادیان خطا صورت می گیرد در قانون BP.

فرمول اصلاح وزن ها:

$$w_{ij}^{new(l)} = w_{ij}^{old(l)} + \eta\, \delta_{pj}^{(l)}\, y_{pi}^{(l-1)}$$

$$\delta_{pj}^{(l)} = f_j'(I_{pj}^{(l)}) \cdot \sum_k \delta_{pk}^{(l+1)} \cdot w_{jk}^{(l+1)}$$

$$-\frac{\partial E_p}{\partial w_{ij}^{(l)}} = -\nabla_w E_p = \delta_{pj}^{(l)}\, y_{pi}^{(l-1)}$$

وقتی در شبکه توابع فعالیتی مانند Tanh داریم، تغییراتشان در یک محدوده ی خاصی است و در یک نواحی هم اشباع می شوند یعنی تغییراتشان خیلی جزئی می شود. گرادیان این توابع در محدوده ۱- و ۱ قرار می گیرد. وقتی تعداد زیادی لایه داریم، خطای خروجی لایه به لایه در گرادیان ها که اعداد کوچکتر از ۱ هستند ضرب می شود و به سمت ورودی می آید و باعث می شود هرچه به سمت ورودی می آییم این عدد کوچکتر شود.

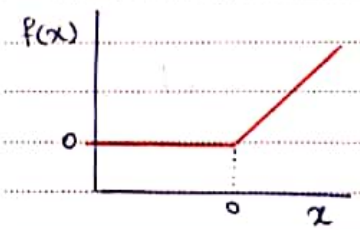
بنابراین گرادیانی که در لایه های ابتدایی به کار می رود بسیار جزئی است در نتیجه اصلاحی که در وزن ها صورت می گیرد بسیار جزئی خواهد بود یعنی جمله ی $\eta \delta y$ در لایه هایی که به ورودی نزدیک هستند خیلی جزئی

Scanned with CamScanner

خواهد بود. به طوری که ارتباط اصلا برقرار نمی‌گیرد. به این مشکل Vanishing gradient problem گفته می‌شود. یعنی گرادیان آنقدر ناچیز می‌شود که ارتباط از بین می‌رود.

اگر از تابع فعال سازی استفاده کنیم که گرادیانش بزرگتر از ۱ باشد، همینطور که به سمت ورودی می‌آییم یک سری عدد بزرگتر از ۱ در هم ضرب می‌شوند و وقتی به لایه‌های اولیه می‌رسیم عددی که از این حاصل ضرب‌ها ایجاد شده عدد خیلی بزرگی است. بزرگی این عدد باعث می‌شود در محاسبات وزن‌ها ناپایداری به وجود بیاید که به آن exploding gradient problem گفته می‌شود. که این مسئله هم آموزش را دچار مشکل می‌کند.

Hinton پیشنهاد داد که مشتق نه بزرگتر از ۱ و نه کوچکتر از ۱ باشد و دقیقاً خود ۱ باشد در نتیجه تابع ReLU پیشنهاد داد.



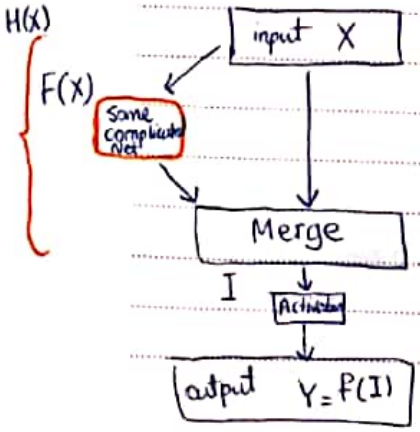
$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$

$$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$

ReLU کمک می‌کند که این مشکل حل شود

معماری ResNET

این شبکه از تعدادی بلوک تشکیل شده که به هرکدام از آن‌ها یک بلوک پایه‌ی ResNet گفته می‌شود. (ResNet Basic Block). ساختار بلوک‌ها به این صورت است:



ورودی وارد یک کنشی مشکل از تعدادی عملیات می‌شود. خود این کنش یک شبکه است، مثلا می‌تواند تعدادی لایه کانولوشنی داشته باشد. تابعی که این شبکه پیاده می‌کند را F(x) می‌نامیم. ورودی و F(x) باهم ترکیب می‌شوند و به نتیجه‌ی آن یک تابع فعالیت اعمال می‌شود و خروجی بلوک را تولید می‌کند.

$$Y = f(m(x, F(x)))$$

m ترکیب بین x و F(x) را نشان می‌دهد:

Scanned with CamScanner

مثلا اگر merge عمل جمع کردن و f یعنی تابع فعال سازی بلوک، ReLU باشد، داریم:

$$Y = ReLU(X + F(X))$$

حالا اگر نتیجه‌ی بلوک را H(X) بنامیم داریم: $Y = ReLU(H(X))$

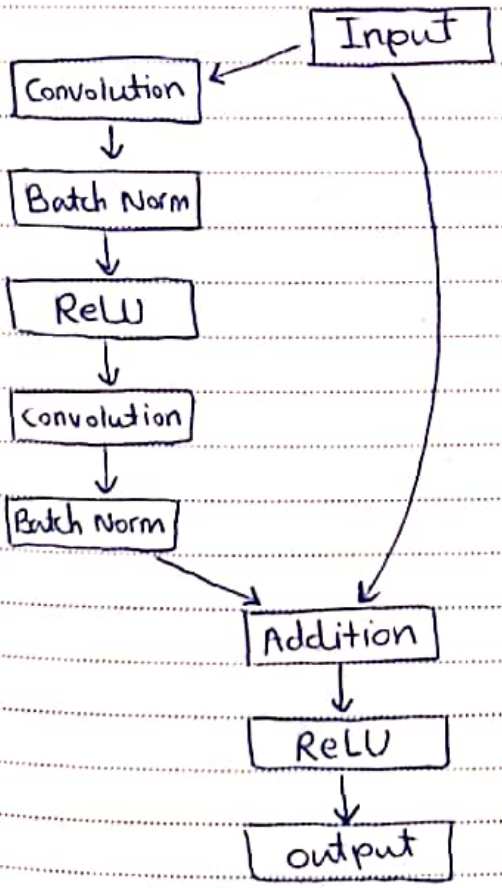
در این آموزش به دلیل ساختاری که انتخاب کرده‌ایم در حقیقت F(X) را یاد می‌گیریم که برابر است با:

$$F(X) = H(X) - X$$

یعنی باقی‌مانده‌ی این تفاضل را در حین آموزش یاد می‌گیریم

به همین دلیل اسم Residual Neural Net را روی آن گذاشته‌اند.

بلوک F(x):



یک نمونه اولیه از آن به این صورت است:

(یک بلوک پایه‌ی ReNet)

فرق عمده‌ی آن با شبکه‌ی کانولوشنی انتقال مستقیم

است که از Input به Addition داریم و امتیازاتی

را به وجود می‌آورد.

این عامل Batch Norm را قرار داده‌اند که سعی

دارد مسئله‌ای را برای ما حل کند. نام این مسئله:

internal covariate shift problem

موجب مشکل در آموزش می‌شود.

کاری که Batch Norm می‌کند این است که خروجی لایه‌ی کانولوشن را نرمالیزه می‌کند.

Scanned with CamScanner

این مسئله این است: همین طور که آموزش جلو می‌رود تغییرات وزن‌ها موجب می‌شود توزیع مقادیر عوض شود، این تغییر توزیع برای آموزش نامطلوب است زیرا باعث می‌شود آموزش کند شود و در یک مسیر مشخص نتواند حرکت کند. بنابراین ترجیح می‌دهیم خروجی لایه‌ی کانولوشن را نرمالیزه کنیم که جلوی این تغییرات را تا حدی بگیریم و آن را کنترل کنیم.

عملی که انجام می‌گیرد به این صورت است: فرض کنید یک Batch داریم: $B=\{x_{1....m}\}$

میانگین آن را محاسبه می‌کنیم:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

واریانس را محاسبه می‌کنیم:

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_B)^2$$

خروجی نرمالیزه کردن:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Scale & Shift

خروجی نرمالیزه شده‌ی مطلوب:

$$y_i = \gamma \hat{x}_i + \beta \equiv BN_{\gamma,\beta}(x_i)$$

در حقیقت xها را به yهایی تبدیل می‌کنیم که میانگین و واریانس آن‌ها را کنترل می‌کنیم

مقایسه شبکه FF با ۲ لایه با شبکه رزنتی که ۲ تا بلوک دارد: (شکل ساختار در اسلایدها)

FF Network:

$$h_1(X) = Sigm(X.W_1 + b_1)$$

$$Y_1 = h_2(h_1(X))$$

$$h_2(X) = Sigm(X.W_2 + b_2)$$

$$Y_1 = Sigm(h_1(X).W_2 + b_2)$$

ResNet Network:

$$g_1(X) = Sigm(X.W_1 + b_1 + X)$$

$$g_2(X) = Sigm(X.W_2 + b_2 + X)$$

$$Y_2 = g_2(g_1(X))$$

$$Y_2 = Sigm(g_1(X).W_2 + b_2 + g_1(X))$$

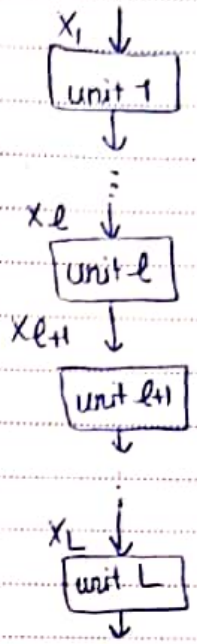
Scanned with CamScanner

تحلیل ResNet

می خواهیم ببینیم اگر بلوک ها را پشت سر هم قرار بدهیم به چه نتیجه ای می رسیم حالت کلی مدل این است که به جای اینکه خود x مستقیماً انتقال پیدا کند، یک تابعی روی آن اعمال شود:

$$y_\ell = h(x_\ell) + F(x_\ell, w_\ell)$$

$$x_{\ell+1} = f(y_\ell)$$



w_ℓ ← مجموعه‌ی وزن‌های مربوط به ℓ امین بلوک شبکه

$$w_\ell = \{w_{k,\ell} \mid 1 \le k \le K\}$$

K (بزرگ) ← تعداد لایه‌هایی که در واحد پایه داریم (۲ یا ۳ است)

F ← تابعی که بلوک پیاده می‌کند، مثلاً ممکن است شامل دو تا کانولوشن 3×3 که پشت سر هم هستند باشد.

f ← تابع فعال‌سازی مثلا ReLU (the operation after element_wise addition)

حالا یک مقدار روابط را ساده‌تر می‌کنیم: فرض کنید $f(x)=x$ و $h(x_\ell)=x_\ell$

آن‌گاه داریم: $x_{\ell+1} \equiv y_\ell$

$$y_\ell = h(x_\ell) + F(x_\ell, w_\ell) \Rightarrow x_{\ell+1} = x_\ell + F(x_\ell, w_\ell)$$

$$x_{\ell+2} = x_{\ell+1} + F(x_{\ell+1}, w_{\ell+1}) = x_\ell + F(x_\ell, w_\ell) + F(x_{\ell+1}, w_{\ell+1})$$

بنابراین داریم:

$$x_L = x_\ell + \sum_{i=1}^{L-1} F(x_i, w_i)$$

یعنی در این شبکه برای واحدی در عمق L خروجی می‌تواند برحسب هرکدام از خروجی‌های بلوک‌های قبلی نوشته شود و خروجی لایه‌ی ابتدایی تر را با مجموع F هایی که در این مسیر دیده شد جمع می‌کنیم.

Scanned with CamScanner

حالا می خواهیم خواصی که این شبکه مبتنی بر این رابطه دارد را بررسی کنیم:

رابطه ای که وجود دارد این است که می توانیم $l=0$ قرار بدهیم. در نتیجه x_0 را داریم که ورودی به شبکه است یعنی اولین ورودی است. پس می توانیم در هر بلوکی رابطه را بر حسب ورودی شبکه بنویسیم:

$$x_L = x_0 + \sum_{i=0}^{L-1} F(x_i, w_i)$$

اگر در شبکه FF که این اتصالات مستقیم وجود ندارد می خواستیم این کار را انجام بدهیم می شد حاصلضرب تعدادی ماتریس. یعنی بلوک بلوک که جلو می آمدیم ضرب می شد در یک ماتریس W دیگر و یک تجمعی ماتریس ضرب شونده در هم را داشتیم که کار را مشکل می کند اما این جا جمله ی ثابتی که داریم به حل مشکلات کمک می کند. حالا چگونه کمک می کند؟ خطا را بررسی می کنیم:

$$\frac{\partial \varepsilon}{\partial x_l} = \frac{\partial \varepsilon}{\partial x_L}\frac{\partial x_L}{\partial x_l} = \frac{\partial \varepsilon}{\partial x_L}\left(1 + \frac{\partial}{\partial x_l}\sum_{i=l}^{L-1} F(x_i, w_i)\right)$$

یعنی در مشتقی که برای خطا حساب می کنیم علاوه بر مشتقی که به صورت معمول در شبکه های FF داشتیم یک جمله ی دیگر هم پیدا می شود که به ما اجازه می دهد بتوانیم مشتق را مستقل کنیم در کنار بخشی که مربوط به F ها مستقل می شود. این قسمت مستقیماً مشتق را به لایه قبلی منتقل می کند و باعث می شود مشتق نتواند در لایه های اولیه هم بزرگ باشد و مشکل vanishing حل می کند. (این جمله ی 1 در مشتق)

شهریور | ۵ | 27 Aug
جمعه | ۱۸ محرم

مقایسه ی شبکه های کانولوشنی مختلف:

صحت: ResNet > VGG-16 > AlexNet > LeNet-5

تعداد پارامترها: VGG-16 > ResNet > AlexNet > LeNet-5

برای بلوک پایه پیشنهادهای دیگری هم داده شد. مثلا Constant Scaling ← در هر دو شاخه مقیاس شدن اعمال می شود.

Scanned with CamScanner

۲) بلوک پایه دیگر: exclusive gating

۳) shortcut only gating ← در این مورد قبلی $h(x)$ دیگر برابر x نیست بلکه تابع دیگری است

نکته: در این شبکه چون X را با F(X) جمع می‌کنیم باید دقت کنیم که ابعادشان باید یکی باشد.

ممکن است بُعد F(X) متفاوت از X باشد در این حالت می‌توانیم یک ضرب ماتریسی در X انجام بدهیم به گونه‌ای که هم بُعد با F(X) شود:

$$F(X) + W_s X$$

درایه‌های ماتریس W_s باید مشخص شود که می‌توانیم در طول آموزش اجازه بدهیم که تعیین شود، یعنی درایه‌های این ماتریس را به عنوان پارامترهایی که در حین آموزش باید مشخص شود در نظر بگیریم.

یک روش دیگر هم این است که با اضافه کردن صفر بُعد X را به بُعد F(X) برسانیم.

Densely Connected Convolutional NNs (DenseNets)

توسعه‌ی دیگری که در شبکه‌های کانولوشنی داده شد و باعث بهبود در آن‌ها شد DenseNet است

در این جا برای حل مسئله‌ی vanishing gradient اینگونه عمل می‌کنیم:

فرض کنیم L تا لایه داشته باشیم و هرکدام از این لایه‌ها تابع H_ℓ را پیاده کند. در خروجی هرکدام از این لایه‌ها یک سری نقشه ویژگی (صفات خروجی) ایجاد می‌شود.

و قرار به لایه‌ی بعدی می‌دهیم این ویژگی‌ها را هم به عنوان ورودی به لایه‌های بعدی می‌دهیم و این کار را در همه‌ی لایه‌ها اجرا می‌دهیم.

بنابراین همینطور که جلو می‌رویم تعداد ورودی‌ها در این شبکه زیاد می‌شود.

لایه‌ی ℓ تابع H_ℓ را پیاده می‌کند که شامل Batch Norm، ReLU و کانولوشن ۳×۳ است.

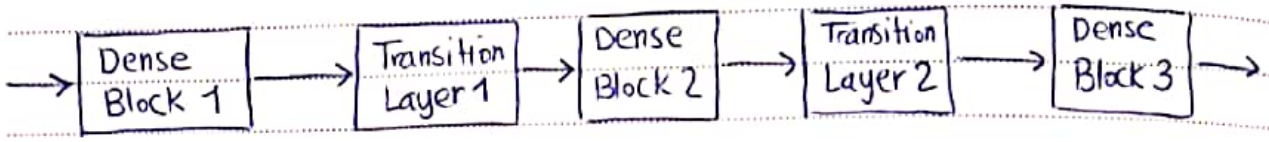
مثلا در اولین لایه این ۳ عمل را داریم، تعدادی صفات ویژگی در خروجی این لایه تولید می‌شود. حالا برای لایه

Scanned with CamScanner

بودن، این خروجی ها می شود ورودی و با نحوه طرح مشابه در خروجی لایه دوم این صفحات و دیگر نیز تولید می شود.
دقت به لایه سوم می دهیم نه فقط صفحات خروجی لایه دوم بلکه صفحات دیگری که در لایه اول هم تولید شده را به
عنوان ورودی به لایه سوم می دهیم.

$$X_\ell = H_\ell([X_0, X_1, \dots, X_{\ell-1}])$$

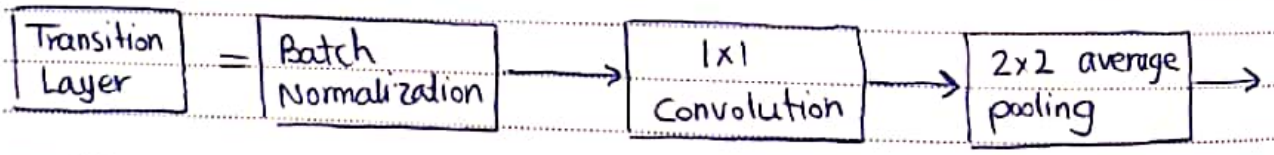
معماری شبکه



بین هر دو تا Dense Block یک Transition Layer قرار می گیرد. کاری که Transition Layer انجام
می دهد:

1) Batch Normalization
2) 1x1 Convolution
3) 2x2 Average pooling

یعنی به نوعی اتفاقاتی که در Dense Block ها می افتد را کنترل می کند.



(شکل معماری DenseNet در اسلاید)

Growth Rate

یک نکته دیگر این است که هر کدام از لایه ها چند تا فیشه تولید کند؟
اگر فرض کنیم هر کدام از لایه ها K تا فیشه تولید کند، تعداد ورودی های لایه ی ℓ ام برابر است با:

$$K_0 + K(\ell-1)$$

K = نرخ رشد شبکه
ℓ = تعداد Feature map هایی که هر لایه تولید می کند

Scanned with CamScanner

گفته شده که معمولا برای به دست آوردن بهترین نتایج لازم نیست که نرخ رشد خیلی بالا باشد

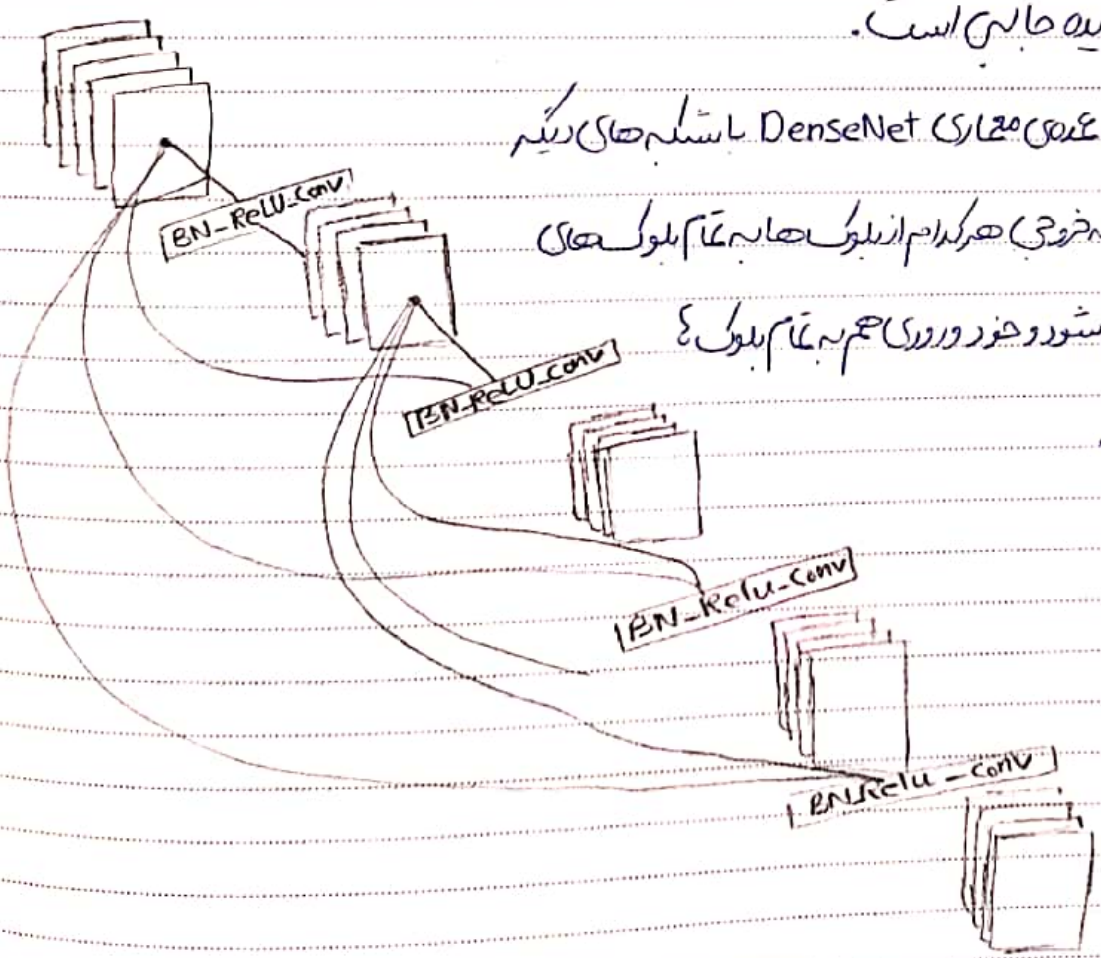
مثلا در حد $k=12$ کافی است.

نکته دیگر این است که این Feature Map ها در جاهای مختلف شبکه با همدیگر، حالت شبکه (global state) را مشخص می کند. شرایطی که شبکه دارد را به ما نشان می دهد. جمع اطلاعاتی که می توانیم از همه ی شبکه بگیریم در این ها هست. می توان گفت که این Feature Map ها به ما ویژگی هایی با درجات تجرید مختلف می دهند. هرچقدر جلوتر می رود، درجه تجریدش بالاتر می رود.

به این ترتیب این ها جمعا اطلاعات خیلی خوبی می دهند.

اصولا این ایده که به هرکدام از لایه های شبکه ویژگی هایی با درجات تجرید مختلف به عنوان ورودی داده شود، ایده جالبی است.

نکته: تفاوت عمده معماری DenseNet با شبکه های دیگر در این است که خروجی هرکدام از بلوک ها به تمام بلوک های بعدی وارد می شود و خود ورودی هم به تمام بلوک ها وارد می شود.



Scanned with CamScanner

جلسه بیستم - ۱۴۰۰/۹/۹

Growth rate

هر لایه K تا نقشه تولید می‌کند و به ورودی های لایه های بعد اضافه می‌شود

تعداد نقشه‌هایی که به لایه l وارد می‌شود: $K_0 + K(l-1)$ ← تعداد نقشه‌های ورودی

K ← نرخ رشد در این شبکه اگر K حدود ۱۲ باشد کافی است که عملکرد این شبکه برسد به بهترین عملکردهای شبکه‌های دیگر.

Battleneck Layers

یک نسخه دیگر از این شبکه: می‌خواهیم به جوری حجم داده‌ای که در شبکه تولید می‌شود را کنترل کنیم

یک لایه گلوگاه می‌گذاریم که اطلاعات را فشرده می‌کند

با قرار دادن یک کانولوشن ۱x۱ قبل از کانولوشن ۳x۳ این کار را انجام می‌دهند

ابتدا اطلاعات فشرده می‌شود و بعد وارد عملیات سه‌گانه‌ی Batch Norm ، ReLU و Conv(3x3) می‌شود

بنابراین یک Conv(1x1) داریم و بعد از آن Batch Normalization ReLU داریم و بعد از آن Batch Norm, Relu, Conv(۳x۳)

اسم این نسخه از شبکه: DenseNet_B

در این حا Conv(1x1) طوری اعمال می‌شود که feature Map تولید شود. 4K مقدار صفات دیگر بود

که هر لایه تولید می‌کرد در این جا می‌خواد که 4K برابر آن باشد

Compression

سومین این شبکه ← باز هم مقدار فشرده‌سازی روی اطلاعات انجام بدهد. اگر فرض کنیم یک بلوک denseNet ، m تا نقشه داشته باشد، سعی می‌کنیم که در Transition layer بعدی تعداد این‌ها به $\lfloor \theta m \rfloor$ برسد.

Scanned with CamScanner

Compression Factor ← $0 < \theta < 1$

DenseNet_C ← نام این نسخه. بنابراین تعداد feature Map ها کاهش داده می‌شود.

DenseNet + Bottleneck layer = DenseNet_B
DenseNet + Compression = DenseNet_C
DenseNet + Bottleneck + Compression = DenseNet_BC

اگر هم Bottleneck layer را بگذارد و هم این Compression را اعمال کند به آن DenseNet_BC می‌گویند.

جدول مقایسه DenseNet با سایر روش‌ها در اسلاید

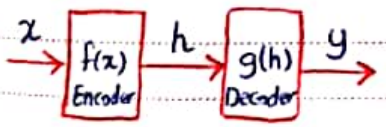
در این جدول نشان داده شده که DenseNet حتی از ResNet هم کمتر عمل می‌کند.

خودرگذارها (Auto Encoders)

هدف این شبکه: بتواند ورودی را دوباره در خروجی ایجاد کند، یعنی خروجی اش برابر ورودی اش باشد.

این شبکه ۲ بخش دارد: ① کدگذار (کدکننده) Encoder ② کدگشا Decoder

وقتی ورودی x به شبکه داده می‌شود Encoder آن را می‌گیرد و خروجی h = f(x) را می‌دهد. بعد Decoder h را ورودی می‌گیرد و y را خروجی می‌دهد که برابر x ورودی است.



فایده این شبکه چیست؟

فایده این شبکه در y یا همان خروجی ای که تولید می‌کند نیست ولی این شبکه با گرفتن ورودی x ارائه‌ای از آن را پیدا می‌کند که با h نشان می‌دهد و این ارائه است که برای ما مهم است. خود این شبکه این ارائه را چجوری پیدا می‌کند؟ یعنی h یک representation از x است که برای ما ارزشمند است. بتوان از روی آن، x اولیه را ساخت.

در Auto Encoder های جدید تر یک نگاه احتمالاتی هم به کار رفته و بازبان احتمالات ارائه داده اند. در این حالت Encoder میاد یه نگاشت تصادفی انجام میده:

$P_{encoder}(h|x)$

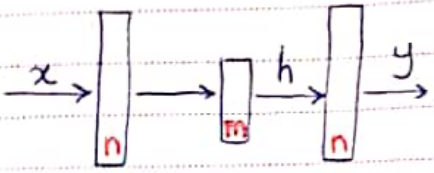
decoder هم اینطور بیان می‌شود:

$P_{decoder}(x|h)$

Scanned with CamScanner

معماری

۲ حالت کلی می‌تواند داشته باشد:



(m<n) undercomplete Autoencoder (۱

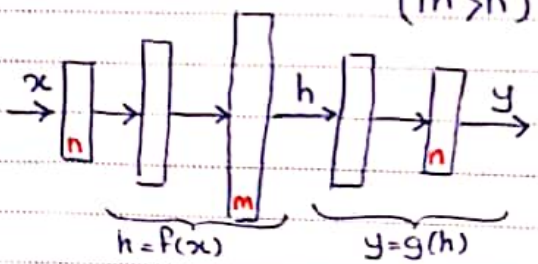
x ورودی با بُعد n به لایه ورودی وارد می‌شود، بعد توزیع می‌شود به لایه‌ی بعدی که m تا نورون دارد و h را تولید می‌کند این لایه. h به لایه‌ی بعدی که n تا نورون دارد داده می‌شود و y را تولید می‌کند. اگر m<n ← underComplete ← یعنی تعداد نورون‌ها در Encoder از بُعد x کوچکتر است در این حالت.

xهایی که وارد می‌شوند وقتی در h ارائه می‌شوند، فشرده شده‌اند (m<n). می‌توان گفت که در این حالت شبکه یاد می‌گیرد که ویژگی‌های مشخص ورودی‌ها را استخراج کند و در ارائه‌ی h بیاورد. چون این ویژگی‌ها لازم‌اند برای اینکه x مجدداً بازسازی شود پس می‌توان گفت ویژگی‌های اصلی هستند.

در این حالت به صورت ۳ لایه رسم شده اما در حالت کلی Encoder و decoder می‌توانند بیش از ۱ لایه باشند.

۱۲ | 3 Sep

(m>n) overComplete Autoencoder (۲



بررسی‌هایشان داده این شبکه به دلیل اینکه m>n می‌تواند x را در خروجی حفظ کند بدون اینکه h به درستی در خروجی Encoder یاد گرفته باشد. گفته می‌شود این شبکه ظرفیت بالایی دارد به خاطر ابعادش.

حتی می‌توانید با یک رابطه‌ی خطی این کار را انجام بدهید و دقیقاً x را تولید کند بدون اینکه h چیزی باشد که ما می‌خواهیم. این ایرادی است که در معماری overcomplete وجود دارد.

این شبکه‌ها FF هستند و با BP آموزش داده می‌شود. تابع Loss آن به این صورت است:

Scanned with CamScanner

$$J_{AE}(\theta) = \sum_{x \in D_n} L(x, g(f(x)))$$

g عملی است که decoder انجام میدهد

f عملی است که encoder انجام می دهد

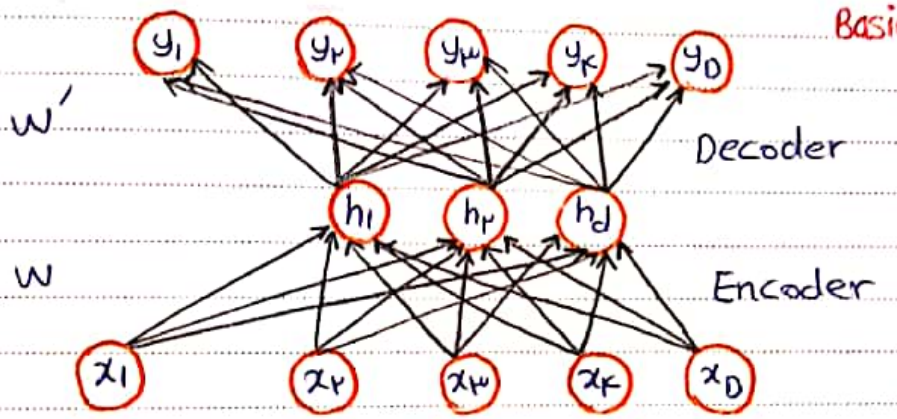
تابع Loss براساس تفاوت ورودی و خروجی تعریف می شود

Autoencoder اگر decoder در این شبکه خطی باشد و L را هم برابر با میانگین مربع خطا بگیریم، آن گاه یک شبکه ی

undercomplete همان فضایی را یادی گیرد که PCA یادی گیرد.

اگر Encoder و Decoder غیرخطی باشند یک تعمیمی از PCA را یادی گیرد که قوی تر است.

Basic AutoEncoders



x ها در لایه ورودی اعمال می شوند و توزیع می شوند به لایه ی مخفی که همان لایه ی Encoder است (با وزن های W توزیع می شوند). بردار خروجی لایه مخفی با وزن های لایه Decoder وارد می شوند و خروجی نهایی تولید می شود.

Encoder

$$h = f(x) = S_f(Wx + b_h)$$

(Sigmoid ↑, bias ↑)

$$S_f = S_g = S(z) = \frac{1}{1+e^{-z}}$$

Decoder

$$y = g(h) = S_g(W'h + b_y)$$

(Sigmoid ↑)

Training: Gradient Descent

objective function:

$$J_{AE}(\theta) = \sum_{x \in D_n} L(x, g(f(x)))$$

D_n : Training Set

پارامترهای شبکه $= \theta = \{W, b_h, b_y\}$

Scanned with CamScanner

تابع Loss می‌تواند موارد مختلفی باشد مثلا:

① squared error: $L(x,y) = \|x-y\|^2$

② cross_entropy Loss: $L(x,y) = -\sum_{i=1}^{d_x} x_i \log(y_i) + (1-x_i)\log(1-y_i)$

نکته: وزن‌های w و w' می‌توانند گونه‌ای انتخاب شوند که: $w' = w^T$. در این حالت تعداد پارامترها کمتر می‌شود و به این حالت Tied weights (وزن‌های به هم گره خورده) گفته می‌شود.

Regularized AutoEncoders

برای اینکه خودکدگذار overComplete را مجبور به تولید یک h مفید کنیم می‌توانیم از Regularization استفاده کنیم. باید در تابع زیان شبکه جمله‌ای اضافه کنیم که چیزی که مورد نظر ماست را تحمیل کند به شبکه.

مثلا می‌توانیم ارائه را به سمت تنک شدن ببریم (sparsity of representation)

یا مثلا می‌توانیم کوچک شدن مشتق ارائه را دنبال کنیم. یا می‌توانیم مقاومت در برابر نویز یا در برابر ورودی‌هایی که نادیده شده اند را به وجود بیاوریم در این شبکه. با این کارهایی می‌توانیم شبکه overcomplete را تبدیل کنیم به شبکه‌ای که کار مفیدانه‌ای انجام می‌دهد.

یک روش ساده برای Regularization این است که weight_decay انجام بدهیم.

$$J_{WDRAE}(\theta) = \sum_{x \in D_n} L(x, g(f(x))) + \lambda \sum_{i,j} W_{ij}^2$$

در این حالت در واقع حین فرآیند آموزش روی وزن‌ها اعمال می‌کنیم که وزن‌ها نباید بزرگ شوند.

WDRAE → weight Decay Regularized Auto Encoder

در نسبت جمله‌ی regularization به جمله‌ی اصلی را کنترل می‌کند در واقع میزان منظم‌سازی را کنترل می‌کند.

ارزیابی‌ها نشان داده است که با انجام این کار مدلی به دست می‌آید که هم ظرفیت بالایی دارد و هم اینکه یادگیری مفید h در آن انجام می‌شود.

Scanned with CamScanner

Sparse Autoencoders

تعبیر دیگری که می‌توان در خودکدگذار overComplete داد این است که تابع زیان را به گونه‌ای تغییر دهیم که ارایه‌ای که حاصل می‌شود تقریبا قابل توجهی از درایه‌هایش صفر باشند. (یعنی sparse یا تُنُک باشد)

$$J_{SAE}(\theta) = \sum_{x \in D_n} L(x, g(f(x))) + \eta \sum_j KL(\rho | \rho_j)$$

$$KL(\rho|\rho_j) = \rho \log \frac{\rho}{\rho_j} + (1-\rho) \log \frac{1-\rho}{1-\rho_j}$$

Kullback Leibler distance ← KL
← فاصله‌ی دو تا توزیع را با آن می‌سنجند

ρ_j ← میانگین خروجی نورون j روی m تا داده‌ی ورودی

ρ ← یک عدد ثابت که معمولا می‌گیریم که خیلی کوچک است مثلا ۰/۰۵

در واقع می‌خواهیم میانگین خروجی نورون j روی ورودی‌های مختلف به سمت ۰/۰۵ (ρ) برود. یعنی به سمت اینکه بیشتر خروجی‌هایش صفر شود. یعنی representation ای که یاد می‌گیرد بیشتر درایه‌هایش صفر خواهند بود.

η هم وزن جمله‌ی اضافه شده را کنترل می‌کند

از ابتدای شبکه اگر چند لایه داشته باشیم تا h تولید شود خروجی هرکدام از این لایه‌ها نسبت sparse هستند ولی ویژگی‌های داده را به‌خوبی encode می‌کند. اما وقتی h که sparse است تولید می‌شود نشان می‌دهد که بعضی از درایه‌ها که صفر شده‌اند تحمیم نیستند و در ارائه‌ی داده‌ی ورودی نقشی ندارند.

به نوعی feature selection انجام می‌دهد به گونه‌ای که نیچرهایی که صفر نیستند مهم‌ترند و آن‌هایی که صفر هستند فیچرهای کم اهمیت‌اند.

Scanned with CamScanner

Denoising Autoencoders

در این جا داده‌ها را نویزی می‌کنیم و می‌خواهیم شبکه همان ورودی را بدون نویز خروجی بدهد. برای این کار ورودی‌ها را با نمونه‌هایی که از یک توزیع گاوسی isotropic گرفته می‌شود، نویزی می‌کنیم:

در رابطه‌ی زیر، ϵ از یک توزیع نرمال با میانگین ۰ و واریانس σ^2 گرفته می‌شود

$$\tilde{x} = x + \epsilon, \quad \epsilon \sim N(0, \sigma^2 I)$$

یک راه دیگر، یک ماسک باینری در نظر بگیریم با مقادیر ۰ و ۱ که در ورودی ضرب می‌شود و بعضی از ورودی‌ها را صفر می‌کند یعنی بعضی از ورودی‌ها را نداریم و ورودی ناقص است. اما در خروجی می‌خواهیم ورودی کامل را داشته باشیم. ۰ و ۱ هایی که در ماسک است به صورت رندوم انتخاب می‌شود.

$\tilde{x}$ با یک فرآیند تخریب $q(\tilde{x}|x)$ ارائه می‌شود. تابعی که برای آموزش شبکه به کار می‌رود:

$$J_{DAE}(\theta) = \sum_{x \in D_n} \mathbb{E}_{\tilde{x} \sim q(\tilde{x}|x)} \left[L(x, g(f(\tilde{x}))) \right]$$

(شکل دراسلاید)

optimization با استفاده از SGD صورت می‌گیرد. این کار هم باعث می‌شود که h مفید شود علیرغم overcomplet بودن.

stacked Denoising Autoencoders

می‌توانیم پشته‌ای از Denoising AE ها داشته باشیم و این h ای که AE اول یاد می‌گیرد ورودی AE دوم باشد، h ای که دومی یاد می‌گیرد ورودی سومی باشد و... به این صورت می‌توانیم representation ها را همین طور که جلو می‌روند بهبود ببخشیم و می‌توانیم تعدادی از این AE ها را داشته باشیم که ما را به یک ارائه خیلی خوب از داده‌ها برسانند.

بعد از آن در یک ساختار supervised دوباره همه‌ی شبکه را آموزش می‌دهیم برای اینکه شبکه عمل دسته‌بندی را یاد بگیرد در عین حال یک آموزش خوبی وزن‌های Encoder ها ببیند.

Scanned with CamScanner

۱۴۰۰

وزن Autoencoder ها را تک به تک آموزش می دهیم هرکدام به طور جداگانه آموزش دیده اند و بدین ها را به هم می چسبانیم در حالی که هنوز بهم آموزش ندیده اند. به همین دلیل آموزش مرحله دوم که گفتیم نیاز است که به آن fine_tuning گفته می شود. ممکن است در آن وزن ها تغییرات کوچکی بکنند. (شکل را ببینید)

Contractive Autoencoders (خودکدگذار انقباضی)

گفتیم در Denoising Autoencoder قصد داشتیم که $g(f(x))$ نسبت به ورودی های نویزی مقاوم باشد. یعنی اگر در ورودی x نویزی بدهیم در خروجی همان x را بدون نویز دریافت کنیم.

به راه دیگری که گفته شده بیاییم خود h را نسبت به نویز مقاوم کنیم. تابع هدف را به این صورت تعریف می کنند:

$$J_{CAE}(\theta) = \sum_{x \in D_n} \left(L(x, g(f(\hat{x}))) + \lambda \left\| J_f(x) \right\|_F^2 \right)$$

$J_f \leftarrow$ مشتقات درجه ۱ تابع f ← Jacobian f

جمله ی Regularization ← نرم فروبنیوس ماتریس ژاکوبین f (Frobenius norm of Jacobian of f)

داریم اندازه ی مشتق را سعی می کنیم که minimize کنیم. مشتق یعنی تغییرات وچون می خواهیم تغییرات $f(x)$ یا h نسبت به تغییرات x کم باشد مشتق f نسبت به x را minimize می کنیم. یعنی تغییرات h را minimize می کنیم نسبت به نویزی که در ورودی اضافه شده.

$$\Omega(h, x) = \lambda \sum_i \left\| \nabla_x h_i \right\|^2$$

علت اینکه به آن Contractive می گویند این است که کاری که در فرآیند Minimization انجام می دهیم در حقیقت معادل این است که یک همسایگی از ورودی ها را به یک همسایگی کوچکتر نگاشت می کند.

Scanned with CamScanner

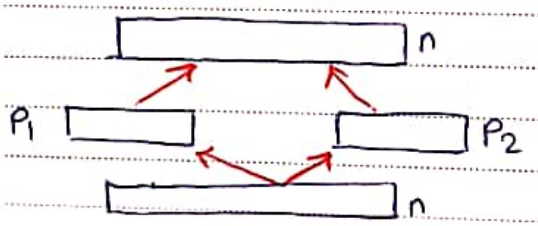
انگاه یک انقباض در اندازه‌ی همسایگی داریم.

استفاده از Encoder و Decoder های deep خوب است چون کمک می‌کند تقریب بهتری بزنیم و نگاشت مناسب‌تری داشته باشیم. حتی محققین می‌شوند که هزینه‌ها در کل کاهش پیدا کند چون با deep کردنشان آموزش راحت‌تر انجام می‌شود و می‌تواند با داده‌های کم‌تر هم صورت بگیرد.

Combined Representations

می‌توانیم نمایش‌های ارائه از auto encoder به دست آوریم و این‌ها را با هم ترکیب کنیم و پشت سر هم قرار دهیم. (Concat) در این صورت می‌توان یک ارائه‌ی قوی‌تری پیدا کرد.

فرض کنید دو تا auto encoder با معماری‌های $n/p_1/n$ و $n/p_2/n$ داشته باشیم. در انتها آموزش آن‌ها دو تا ارائه به دست می‌آید که می‌توان با ترکیب آن‌ها یک ارائه‌ی قوی با سایز p_1+p_2 داشته باشیم.



فرق این دو تا representation که به دست می‌آوریم می‌تواند در این موارد باشد:
① الگوریتم یادگیری
② مقداردهی اولیه (initialization)
③ مجموعه آموزشی
④ نرخ یادگیری
⑤ معیارهای اندازه‌گیری اعوجاج (distortion measures)

هم‌چنین پیش‌نهاد شده نورون‌های encoder تدریجا کم و زیاد شود، به این‌گونه نورون اضافه یا از آن کم کنیم.

نکته: در حالت خطی که نورون‌ها خطی‌اند و در فضای حقیقی باشیم، اگر یک واحد مخفی قرار دهیم اولین principal component را می‌یابد. وقتی را یک قرار دهیم دومین PC را می‌یابد.

Scanned with CamScanner

جلسه بیست و یکم - ۱۴۰۰/۹/۱۴

تاکنون ۴ مسئله را مورد بررسی قرار دادیم: دسته‌بندی، تخمین تابع، خوشه‌بندی، استخراج ویژگی

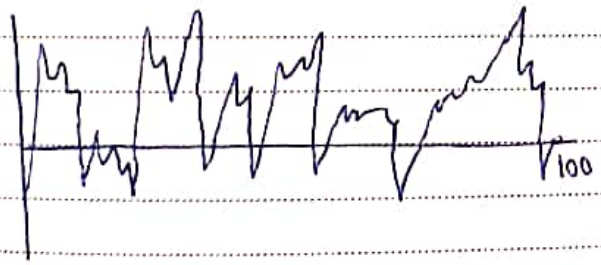
مسئله‌ی بعدی که بررسی می‌کنیم: پیش‌بینی سری‌های زمانی با شبکه‌های عصبی

این مسئله بسیار مهم است و در مسائل عملی خیلی پیش می‌آید. مثلا: پیش‌بینی بارش باران، پیش‌بینی مصرف برق، پیش‌بینی بازار سهام و ...

در همه‌ی این‌ها یک متغیری داریم که به صورت منظم مقادیر آن را از بازه‌های زمانی گذشته داریم و می‌خواهیم مقادیر آینده‌ی آن را پیش‌بینی کنیم.

تاکنون در شبکه‌هایی که داشتیم نمی‌توانستیم ورودی‌ای داشته باشیم به صورت دنباله‌ای از اعدادی که بازمان تغییر پیدا کرده‌اند. بنابراین می‌خواهیم یک دیتاگر جدیدی را وارد شبکه‌های عصبی کنیم.

مثال:



مثلا تغییرات یک متغیر را در ۱۰۰ روز داریم و می‌خواهیم مقدار آن را در قدم بعدی پیش‌بینی کنیم.

با توجه به این سری‌هایی توان متوجه شد که این مسئله، مسئله‌ی دشواری است.

برای حل مسئله از یک دیتاگر که در سری‌های زمانی وجود دارد استفاده می‌کنیم: خودهمبستگی (Auto correlation)

یعنی هم‌بستگی بین نمونه‌ای که الان داریم و نمونه‌های قبلی ← مبنای روش‌های پیش‌بینی

این هم‌بستگی در مورد نمونه‌هایی که در گذشته‌ی نزدیک هستند بیشتر قابل اعتنا است تا آن‌هایی که مربوط به گذشته‌های دور هستند. به همین دلیل نمونه‌های قبلی هم در تاثیرگذاری در نظر گرفته می‌شوند و به آن‌ها lags گفته می‌شود

مدل زمانی: مدلی است که سعی می‌کند مقادیر آینده‌ی یک متغیر را براساس نمونه‌های قبلی که از این متغیر داریم پیش‌بینی کند

Temporal Models

Scanned with CamScanner

Spatio_Temporal Modelling : مقدار آینده‌ی متغیر را با استفاده از مقادیر گذشته‌ی خودش و یک متغیر دیگر پیش بینی می‌کند

یعنی در روش قبل فقط خود این متغیر نیست، بلکه متغیرهای دیگر هم هستند

پیش بینی می‌تواند بر مبنای مدل‌های خطی یا غیرخطی باشد.

Linear Forecasting

1) Autoregressive (AR) model of order p

مدل خودپس رونده از درجه‌ی p

$$y_t = a_0 + a_1 y_{t-1} + a_2 y_{t-2} + \dots + a_p y_{t-p} + \varepsilon_t$$

در این جا یک ترکیب خطی از نمونه‌های قبلی در نظر گرفته می‌شود و p قدم به عقب برمی‌گردد. یعنی به مقدار نمونه‌های موثر را

p در نظر گرفته‌ایم به همین دلیل می‌گوییم این مدل از درجه‌ی p است.

در حقیقت در هر زمانی که هست، مقدار را برای قدم بعدی از ترکیب خطی و نیز از مقادیر قبلی پیش بینی می‌کند.

ε_t خطا در زمان t است، یک متغیر تصادفی که از یک تابع گاوسی با میانگین صفر و واریانس σ^2 نمونه برداری می‌شود

Moving Average (MA) model of order q

مدل میانگین متحرک از درجه‌ی q

$$y_t = a_0 + \varepsilon_t - b_1 \varepsilon_{t-1} - b_2 \varepsilon_{t-2} - \dots - b_q \varepsilon_{t-q}$$

بازسازی مقدار بعدی به جای اینکه بر اساس نمونه‌های قبلی باشد، بر اساس خطاهای پیش بینی‌های قبلی انجام می‌شود

Autoregressive Integrated Moving Average (ARIMA) model of order (p, q)

$$y_t = a_0 + a_1 y_{t-1} + a_2 y_{t-2} + \dots + a_p y_{t-p} + \varepsilon_t - b_1 \varepsilon_{t-1} - b_2 \varepsilon_{t-2} - \dots - b_q \varepsilon_{t-q}$$

ε_i ها متغیرهای تصادفی مستقل از هم و دارای توزیع یکسان هستند

مدل‌های غیرخطی

$$y_t = f(y_{t-1}, y_{t-2}, \dots, y_{t-p}) + \varepsilon_t$$

این چیزی که مدل را غیرخطی می‌کند تابع f است

چگونه می‌توانیم داده‌هایی که دارای زمان هستند را وارد شبکه‌های عصبی کنیم؟

برای انجام این کار باید بتوانیم حافظه کوتاه مدت شبکه ایجاد کنیم. به دو راه می‌توان این کار را انجام داد:

① یک مکانیزمی به نام tapped delay Line (خط تاخیر انتقال دار) به شبکه اضافه کنیم

Scanned with CamScanner

② اضافه کردن اتصالات فیدبک دار به شبکه (feedback)

در مقابل حافظه‌ی کوتاه مدت، حافظه‌ی بلندمدت را داریم که در شبکه توسط وزن‌ها مشخص می‌شود.
حافظه‌ی کوتاه مدت اثرش با زمان از بین می‌رود ولی حافظه‌ی بلندمدت ماندگاری دارد.

STM structures

اگر یک سری زمانی داشته باشیم که با یک دنباله $\{x(n)\}$ نشان داده می‌شود و یک سری زمان گسسته است. تبدیل Z برای سری‌های گسسته به کار می‌رود:

$$X(Z) = \sum_{n=-\infty}^{\infty} x(n) z^{-n}$$

z^{-1} = unit delay operator

↓

اپراتور تأخیر واحد، یعنی یک قدم تأخیر

$$y(n) = \sum_{k=-\infty}^{\infty} h(k)\, x(n-k)$$

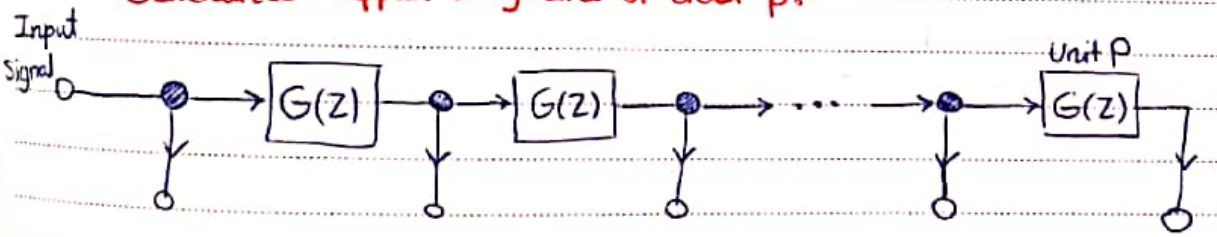
حوزه‌ی Z ↓

$$Y(z) = H(z)\, X(z)$$

برای یک سیستم گسسته زمان این روابط را داریم:

خروجی را می‌توان براساس ورودی و تابع h نوشت که h(k) پاسخ ضربه‌ی سیستم است

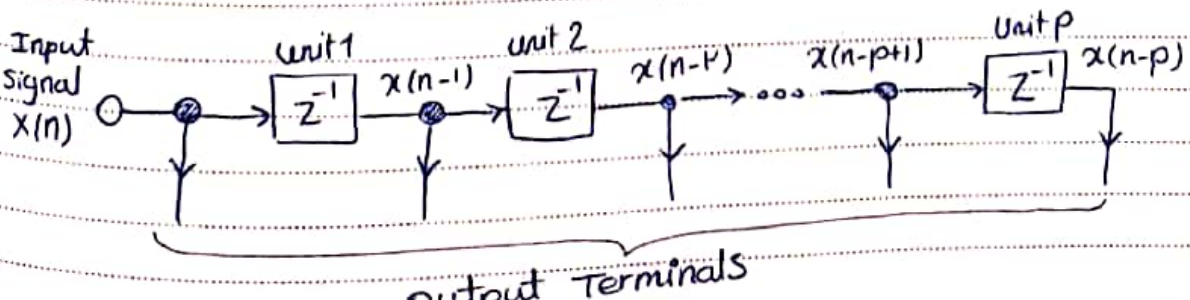
Generalized Tapped Delay-Line of order p:



output Terminals

حالت کلی خط تأخیر یک سری بلوک داریم که تابع G(z) را انجام می‌دهد، p تا از این‌ها را به صورت سری پشت سر هم داریم. چون G را کلی گذاشتیم به آن Generalized می‌گوییم. حالا اگر بخواهیم خاصش کنیم:

$G(z) = z^{-1}$



output Terminals

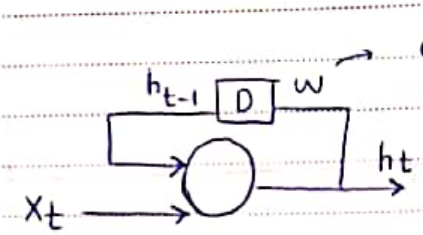
Scanned with CamScanner

وزن $z^{-1}G(z)$ یعنی هر unit فقط یک قدم تأخیر ایجاد کند و هیچ عمل دیگری روی سیگنال انجام ندهد جز اینکه یک قدم آن را نگه دارد. شبیه shift register است.

در اتصالات بعد از هر unit خروجی ها را نگه میداریم و همه‌ی این ها می‌شوند خروجی های این ساختار مان.

به آن Tapped Delay-Line of order p گفته می‌شود.

Feedback Connection



در شکل روبه رو خروجی نورون را با یک تأخیر واحد به ورودی نورون برگردانده‌ایم. فرض کنیم نورون خطی باشد:

$f(x_t)=x_t$ $\quad h_t=f(x_t,h_{t-1})=x_t+wh_{t-1}$

x_1 $\quad h_1=x_1$

x_2 $\quad h_2=x_2+wh_1=x_2+wx_1$

x_3 $\quad h_3=x_3+wh_2=x_3+wx_2+w^2x_1$

... ...

x_m $\quad h_m=x_m+wx_{m-1}+w^2x_{m-2}+w^3x_{m-3}+\dots+w^{m-1}x_1$

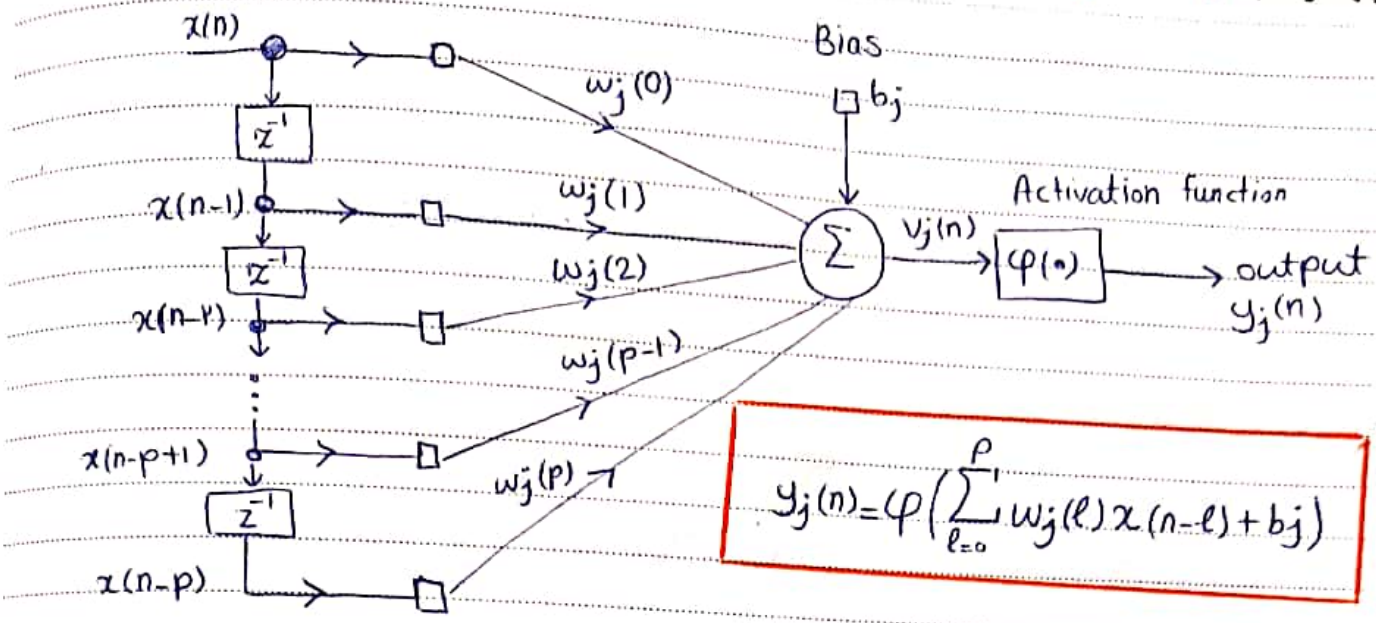
خروجی در هر زمان برابر است با ترکیبی از ورودی ها در این قدم و قدم های قبل.

اگر $|w|<1$ ← ضریب ها هر چقدر دورتر می‌شویم کوچکتر می‌شود. یعنی نمونه های نزدیکتر اثرگذاری بیشتری دارند.

حالا چگونه از این ها در شبکه ها استفاده کنیم؟

ابتدا فرض کنید یک نورون داشته باشیم که بخواهیم سری های زمانی را به عنوان ورودی به آن بدهیم. از ساختار Tapped Delay-line استفاده می‌کنیم و آن را به ورودی های نورون وصل می‌کنیم:

Scanned with CamScanner



$$y_j(n) = \varphi\left(\sum_{l=0}^{p} w_j(l)\, x(n-l) + b_j\right)$$

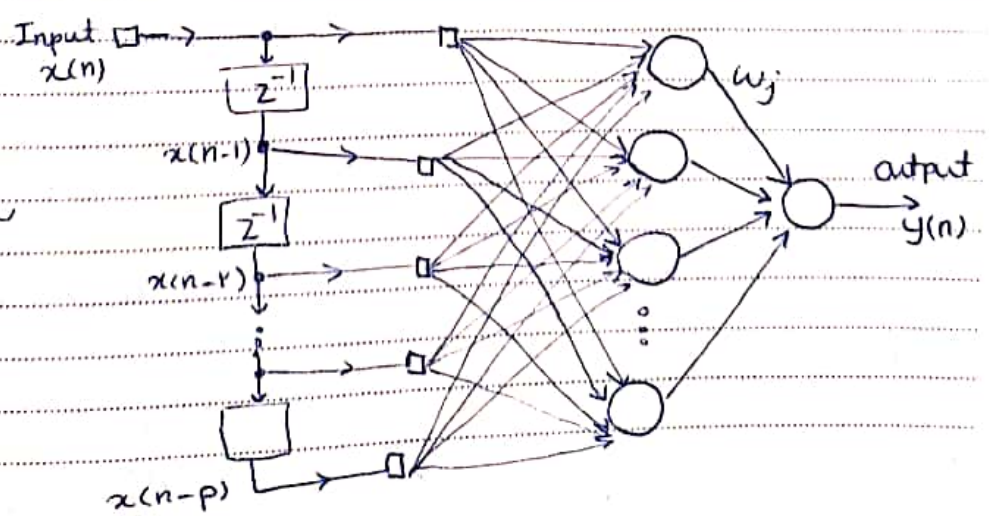
هرکدام از خروجی های Tapped Delay_Line مرتبه‌ی p را با یک وزن دلخواه به نورون بعدی می‌دهیم. در واقع برای اینکه بتوانیم یک سیگنال تابع زمان را به نورون بدهیم، نمونه‌ها زمان‌های مختلف را هم‌زمان به نورون می‌دهیم.

Focused Time Lagged FeedForward Network (TLFN)

می‌توانیم در لایه‌ی مخفی یک شبکه ۳ لایه مقداری از این نورون‌ها را به کار ببریم و در لایه‌ی دوم هم مکانیزم TDL نمونه‌های سری زمان را ایجاد می‌کند به گره‌های ورودی می‌دهد و توزیع می‌شود. یک نورون خطی هم در لایه خروجی می‌گذاریم

$$y(n) = \sum_{j=1}^{m_1} w_j y_j(n) = \sum_{j=1}^{m_1} w_j \varphi\left(\sum_{l=0}^{p} w_j(l)\, x(n-l) + b_j\right) + b_0$$

با بایس نورون خروجی



m_1 تا نورون مخفی داریم

آموزش: BP

Scanned with CamScanner

مثال: فرض کنید یک سری زمانی داریم: $\{x(n-l)\}_{l=0}^{p}$

راستگونه مسائل دقتی می خواهند بسیتیک مدل چقدر خوب کار می کند. با یک مسئله ای که می دانند خروجی هایش چه چیزی خواهد بود شروع می کنند. در این مسئله سری زمانی از رابطه ریاضی زیر تولید شده:

$$x(n) = \sin(n + \sin(n^2)) \qquad n = 0, 1, 2, \dots$$

تا بین که خروجی ها را برای مشاهدات نیاز داریم از روی همین تابع حساب می کنیم. دقتی هم که مدل پیش بینی را انجام داد، مقدار صحیح را با همین تابع حساب می کنیم و تفاضلش را با مقدار پیش بینی شده می سنجیم تا خطا به دست آید.

در این شبکه درجه ی خطا تاخیر را ۲۰ گرفته ایم ($p=20$)، تعداد نورون های لایه مخفی = ۱۰، تابع فعال سازی: لاجستیک

در لایه ی خروجی ۱ نورون خطی داریم. نرخ یادگیری هر دو لایه ← ۰.۱

Momentum هم نداریم.

(شکل مثال در اسلاید)

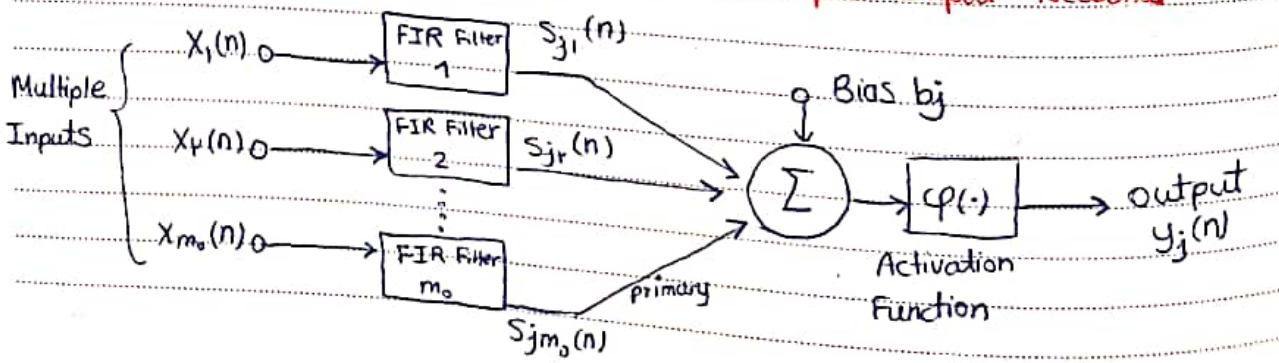
Training Data

۵۰۰ تا سری ۲۰ تایی تولید می کنیم با استفاده از رابطه ریاضی که داشتیم (رندم)

پس $\{x(n)\}$، ۲۰ تا مقدار دارد و ۵۰۰ تا از این ها را می کنیم. پس از آموزش مدل، خطای پیش بینی در بازه $[-0.2, 0.2]$ است.

حالا اگر مقادیر سری زمانی داشته باشیم که دوبعدی باشند (Spatio_Temporal):

Multiple Input Neuronal Filter



Scanned with CamScanner

۱۴۰۰

فرض کنید m_0 تا سری زمانی داریم که ورودی های شبکه اند: $x_1(n), x_2(n), \dots, x_{m_0}(n)$

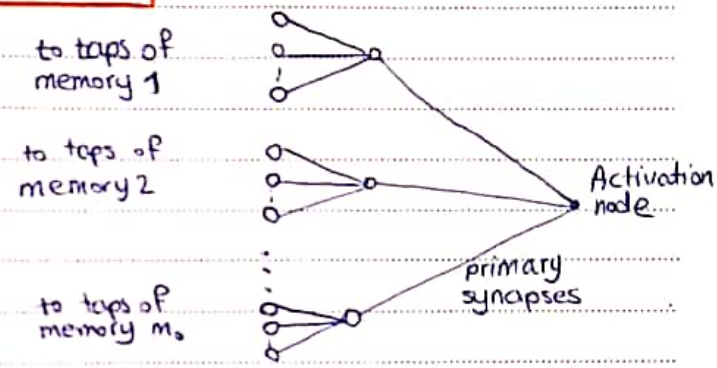
هرکدام از این ها با ساختاری مشابه شبکه ی قبلی یک خروجی تولید می کنند: $s_{j1}(n), s_{j2}(n), \dots, s_{jm_0}(n)$

ساختارها هم با عنوان FIR Filter در شکل هستان داده شده (همان ساختار شبکه ی قبلی است)

یک نورون داریم که نتایج را با هم ترکیب می کند، خروجی آن از تابع فعال سازی عبور می کند و خروجی نهایی را تولید می کند.

رابطه این شبکه:

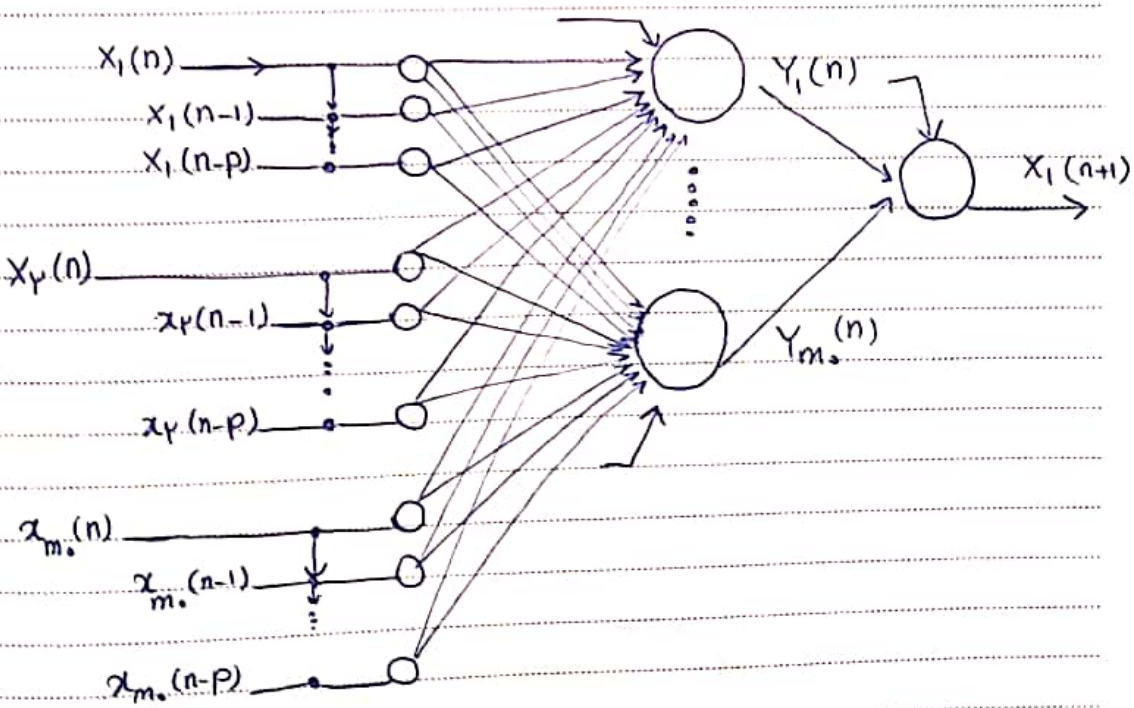
$$y_j(n) = \varphi\left(\sum_{i=1}^{m_0} \sum_{l=0}^{p} w_{ji}(l)\, x_i(n-l) + b_j \right)$$



حالا می خواهیم پیچیده ترکنیم و شبکه ای از نورون ها داشته باشیم:

که ورودی شان سری های زمانی مختلف است

Spatio_Temporal (Distributed) Time Lagged FF Network



Scanned with CamScanner

$$S_{ji}(n) = \sum_{l=0}^{p} w_{ji}(l)\, x_i(n-l)$$

$$v_j(n) = \sum_{i=1}^{m_0} S_{ji}(n) + b_j$$

$$y_j(n) = \varphi(v_j(n))$$

$$S(n) = \sum_{j=1}^{m_0} y_j(n)$$

$$X_1(n+1) = \varphi(S(n))$$

در این حالت هدف این است که نمونه‌ی بعدی x_1 را تولید کنیم ولی نمونه‌ی بعدی x_1 فقط تحت تاثیر نمونه‌های قبلی x_1 نیست بلکه تحت تاثیر نمونه‌های قبلی $x_2, x_3, \dots$ و x_{m_0} هم هست.

وزن‌های y_jها را ۱ داریم در نظر می‌گیریم چون نیازی نیست که این‌ها دوباره وزن یاد گرفته شود.

در این حالت بیشتر از یک سری زمانی پاسخ را تعیین می‌کند.

آموزش این شبکه مشابه شبکه‌های Feed Forward است.

این حالت یک گسترش غیرخطی از AR است، با ورودی‌های خارجی: exogenous Input variables

که به آن مدل NARx گفته می‌شود.

برای آموزش این شبکه سه مجموعه Train، Validation و Test نیاز است. توصیه می‌شود که Validation را از جایی نزدیک Train انتخاب کنیم و Test را از جایی دور از Train بردار برای اینکه بفهمیم چقدر خوب عمل می‌کند. چون رفتار سری‌های زمانی در حال تغییر است و ممکن است مقداری که در حال پیش‌بینی هستیم به طور کلی تغییر کند.

در این شبکه اتصالات کامل است.

Scanned with CamScanner

۱۴۰۰ X(n): Forecast values T(n): Desired values

معیارهای سنجش کیفیت پیش بینی:

$$MSE = \frac{1}{N}\sum_{n=1}^{N}(x(n)-T(n))^2 \qquad RMAE = MAE/\bar{T} \qquad \bar{T}: \text{میانگین مقادیر target}$$

$$MAE = \frac{1}{N}\sum_{n=1}^{N}|x(n)-T(n)| \qquad PI = 1 - \frac{\sum_{n=1}^{N}(x(n)-T(n))^2}{\sum_{n=2}^{N}(T(n)-T(n-1))^2}$$

↳ شاخص خطای تخمین نسبت به شاخص تفاضل عنوان‌های متوالی

Hybrid Linear and NonLinear Model

می‌توانیم مدل‌های ترکیبی خطی و غیرخطی داشته باشیم.

فرض کنیم L_t مولفه‌ی خطی و N_t مولفه‌ی غیرخطی مدل است. در این صورت y_t را می‌توانیم به صورت مجموع این دو کنش بنویسیم:

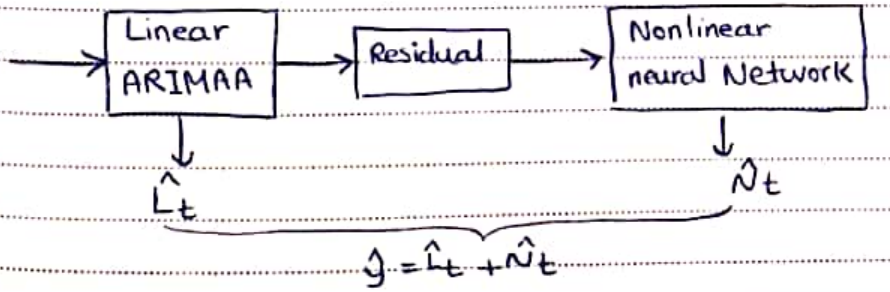
$$y_t = L_t + N_t$$

اگر کنش خطی را با یک مدل خطی تخمین بزنیم $\hat{L}_t$ به دست می‌آید. $\hat{L}_t$ را از y_t کم کنیم خطا پیدا می‌شود که مربوط به قسمت خطی است:

$$e_t = y_t - \hat{L}_t$$

حال e_t را با یک تابع غیرخطی تخمین می‌زنم. e_t در واقع تخمین N_t است:

$$e_t = \hat{N} + \varepsilon_t = f(e_{t-1}, e_{t-2}, \dots, e_{t-n}) + \varepsilon_t$$



توضیح شکل: ابتدا یک تخمین خطی می‌زنیم $\hat{L}_t$ پیدا می‌شود. باقی مانده‌اش را می‌دهیم دیگ مدل غیرخطی و $\hat{N}_t$ را می‌یابیم و خروجی جمع این دو تاست. نشان داده شده که این دو با یکدیگر عملکرد بهتری از مدل خطی تنها یا مدل غیرخطی تنها دارد و اینکه مدل‌های خطی و غیرخطی ساده‌تری هم نیاز است.

Scanned with CamScanner

آیا می‌توان از همین مدلی که داشتیم برای پیش بینی چند گام آینده استفاده کنیم؟

مثلا ممکن است پیشنهاد شود که از مقادیر پیش بینی شده توسط خود مدل استفاده کنیم و به عنوان ورودی به مدل بدهیم اما این کار ایراد دارد. وقتی یک پیش بینی را به عنوان یک مقدار مشاهده شده وارد مدل می‌کنیم، داریم خطایی را وارد مدل می‌کنیم و همین طور که جلوتر می‌رود خطا هم بیشتر می‌شود.

مثلا اگر یک قدم جلو برویم ۱ نمونه تقریبی اضافه شده، در ۲ قدم ۲ نمونه تقریبی و ...

بعد یک تاریخچه‌ای از مقادیر تقریبی خواهیم داشت که خطایی را طی کند و یک خطای مجتمع داریم.

همچنین هر چقدر اون زمانی که می‌خواهیم پیش بینی کنیم آینده‌ی دورتری باشد قدرت پیش بینی بیشتر کاهش پیدا می‌کند.

اگر مجموعه‌ی تستی داشته باشیم که شبیه مجموعه آموزش نباشد صحت در مورد آن افت می‌کند نسبت به نمونه‌ی تستی که نزدیک باشد به Training Set.

اگر چند سری زمانی ورودی داشته باشیم نمونه‌های آن‌ها را هم باید وارد کنیم که این جا باز هم تقریب وارد می‌شود و خطا خیلی زیاد می‌شود

⇐ مشاهده شد که اگر تمام سری زمانی که داریم (کل مشاهدات) را برای آموزش به کار بریم، پیش بینی‌های دراز مدت بهتر انجام می‌شود

جلسه بیست و دوم — ۱۴۰۰/۹/۱۴

مقایسه‌ی مدل‌های خطی و غیرخطی

به طور کلی قدرت پیش بینی مدل‌های غیرخطی بیشتر از مدل‌های خطی است به خصوص وقتی که پیش بینی دراز مدت باشد. اگر داده‌های تست شبیه داده‌های آموزش باشند مدل غیرخطی بهتر عمل می‌کند

Scanned with CamScanner

فقط در یک حالت مدل خطی بهتر عمل می‌کند: در حالتی که داریم قدم بعدی را پیش‌بینی می‌کنیم و داده‌ها از نظر مشخصات با داده‌های آموزش متفاوت باشند، مدل خطی به صورت جزئی بهتر از مدل غیرخطی عمل می‌کند.

اما همین پیش‌بینی اگر برای چند قدم دورتر باشد، باز مدل غیرخطی بهتر عمل می‌کند.

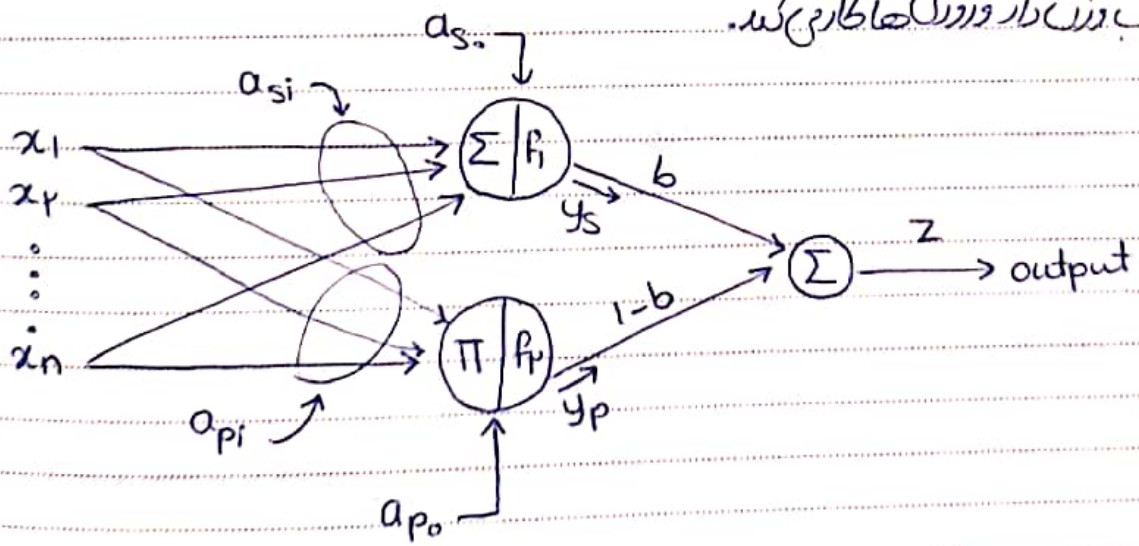
در هر دو حالت استفاده از تمام داده‌ها دقت پیش‌بینی را افزایش می‌دهد.

برای پیش‌بینی‌های بلندمدت مدل‌های غیرخطی بهتر عمل می‌کنند.

Generalized Neuron Network

یک مدل دیگر که پیشنهاد شده مدلی است که ترکیبی از حالات خطی و غیرخطی ورودی‌ها را به طریقی می‌گیرد.

شبکه دو تا شاخه دارد: یک شاخه روی مجموع وزن‌دار ورودی‌ها کار می‌کند و یک شاخه دیگر روی حاصل ضرب وزن‌دار ورودی‌ها کار می‌کند.



شاخه‌ی مجموع: $y_s = f_1\left(\sum_{i=1}^{n} a_{si}x_i + a_{s0}\right) = f_1(a_{s1}x_1 + a_{s2}x_2 + \dots + a_{s0})$

شاخه‌ی حاصل ضرب: $y_p = f_2\left(\left(\prod_{i=1}^{n} a_{pi}x_i\right)a_{p0}\right) = f_2(a_{p1}x_1 \times a_{p2}x_2 \times \dots \times a_{pn}x_n \times a_{p0})$

خروجی: $z = y_s b + y_p(1-b)$

Scanned with CamScanner

روابط اصلاح وزن‌ها برای یک نورون که دو ورودی دارد: (قسمت حاصل جمع)

$$\Delta a_{s1_new} = \eta \sum_{i=1}^{n} (t_i - z_i)\, b y_{si}(1-y_{si})\, x_{1i} + \alpha \Delta a_{s1_old}$$

$$\Delta a_{s2_new} = \eta \sum_{i=1}^{n} (t_i - z_i)\, b y_{si}(1-y_{si})\, x_{2i} + \alpha \Delta a_{s2_old}$$

$$\Delta a_{s0_new} = \eta \sum_{i=1}^{n} (t_i - z_i)\, b y_{si}(1-y_{si}) + \alpha \Delta a_{s0_old}$$

$$a_{si,new} = a_{si} + \Delta a_{si,new}$$

η ← نرخ یادگیری α ← ضریب momentum

اصلاح وزن‌ها برای قسمت حاصل ضرب:

$$\Delta a_{p1_new} = -2\eta \sum_{i=1}^{n} (t_i - z_i)(1-b)\, y_{pi} \frac{u_{pi}^2}{a_{p1}} + \alpha \Delta a_{p1,old}$$

$$\Delta a_{p2_new} = -2\eta \sum_{i=1}^{n} (t_i - z_i)(1-b)\, y_{pi} \frac{u_{pi}^2}{a_{p2}} + \alpha \Delta a_{p2,old}$$

$$\Delta a_{p0_new} = -2\eta \sum_{i=1}^{n} (t_i - z_i)(1-b)\, y_{pi} \frac{u_{pi}^2}{a_{p0}} + \alpha \Delta a_{p0,old}$$

$$a_{pi,new} = a_{pi} + \Delta a_{pi,new}$$

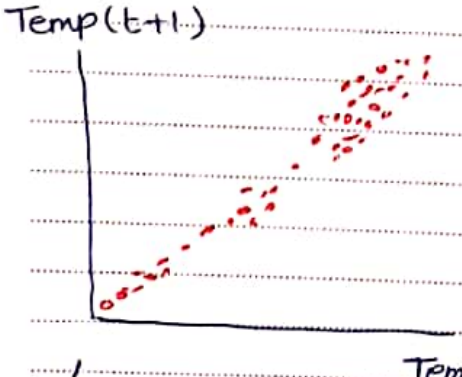
$$\Delta b_{new} = \eta \sum_{i=1}^{n} (t_i - z_i)(y_s - y_p) + \alpha \Delta b_{old}$$

$$b_{new} = b + \Delta b_{new}$$

۲ مهر | جمعه
24 Sep ■ ۱۷ صفر

مثال: پیش‌بینی درجه حرارت

ابتدا نمودار زیر از درجه حرارت در زمان $t+1$ نسبت به درجه حرارت در زمان t رسم کرده است و مشاهده شده که این دو تا رابطه‌ی خطی دارند.



به همین دلیل یک نورون خطی به صورت یک مدل ARX در نظر گرفته و فقط یک time lag را استفاده کرده (یعنی فقط نمونه‌ی قبلی)

Scanned with CamScanner